SIMATS ENGINEERING

Department of Information Technology

ITA05 – Computer Vision

COMPUTER VISION – ASSIGNMENT 2 (CO2)

OLD PHOTOGRAPH RESTORATION SYSTEM

CO2: Apply image enhancement and segmentation techniques to restore damaged photographs and identify meaningful image regions.

Assignment Details	Information	Assignment Details	Information
Student Name	K NISHALINI	Register Number	192521301
Department / Section	Information Technology	Academic Year / Semester	2025–2026
Faculty Name	DR P MURALI	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Date of Submission	05.11.2026	Date of Evaluation	

SCENARIO

A college archive has a collection of old family and institutional photographs that have become faded and damaged over time. The photographs show loss of contrast, yellow/sepia colour shifts, scratches, dust, stains, ink marks, creases and uneven fading. Some images are slightly blurred, while others contain small torn or missing regions. The archive wants an Old Photograph Restoration System that (1) improves brightness, contrast and visual clarity while preserving original facial features and fine details, and (2) identifies damaged regions such as scratches, stains, dust spots, creases and torn areas so that they can be repaired or reviewed. The restored result must retain the historical appearance of the photograph and must not invent or alter important details.

You are the vision engineer. For every task, three candidate techniques are provided. Implement all three techniques on the same set of old photographs, compare their performance, select the most suitable technique, and justify the selection with experimental evidence. Pay special attention to preserving faces, clothing, background details and original photographic tone. State and justify any additional assumptions.

OPERATING DATA

Parameter	Operating Data
Image resolution	A4, 2480 × 3508 pixels (300 DPI)
Image format	8-bit grayscale

Text stroke width	2–4 pixels
Contrast	Faded ink is 30–50 grey levels darker than paper
Lighting	Uneven illumination with one-sided shadows
Noise	Gaussian noise ($\sigma \approx 6$) and 1% salt-and-pepper noise
Scanner noise	Horizontal stripes every 6 pixel rows
Processing time	Maximum 3 seconds per page
Target accuracy	Text segmentation Dice score ≥ 0.90
Hardware	4 CPU cores, 16 GB RAM, no GPU
Software	Python, OpenCV and NumPy

TASK 1 – GREY LEVEL TRANSFORMATION ANALYSIS [4 Marks]

The old photographs contain faded areas and low contrast. Apply and compare three grey-level transformation techniques to recover visible details without losing highlights or shadow details.

Candidate Techniques

- A. Negative and Log Transformation
- B. Power-Law (Gamma) Transformation
- C. Piecewise-Linear Contrast Stretching

Questions

1. Implement all three transformations using Python/OpenCV on the same photographs.
2. Compare subject-to-background contrast, recovery of faint details, clipping effects and processing time.
3. Plot the transformation curves and corresponding input/output histograms.
4. Perform bit-plane slicing and identify which bit planes contain useful image details and which mainly contain noise or damage.
5. Analyse the effect of applying the same gamma value to both bright and dark photographs.
6. Compare a pretrained low-light or image-enhancement model with the selected transformation.

Evidence Expected: Transformation curves, before-and-after images, histograms, contrast comparison table and justified technique selection.

1. Introduction

Old photographs are valuable historical records that preserve memories of families, institutions, cultural events and important personalities. Due to ageing, sunlight, moisture, poor storage and environmental conditions, photographs may become faded and damaged. Common problems include low contrast, loss of brightness, yellow or sepia shifts, scratches, dust particles, stains, ink marks, creases, uneven illumination and slight blur.

Grey-level transformation is a spatial-domain image enhancement technique that changes pixel intensity values to improve visibility. In this task, three candidate methods—negative and logarithmic transformation, power-law (gamma) transformation, and piecewise-linear contrast stretching—are implemented and compared on the same photographs. The aim is to improve visible details while preserving faces, clothing, background objects and the original historical tone.

2. Objectives

- Improve brightness and contrast of faded grayscale photographs.
- Recover faint facial features, clothing patterns and background details.
- Implement and compare the three candidate grey-level transformation techniques.
- Study input and output histograms and transformation curves.
- Use bit-plane slicing to identify useful image information and possible noise.
- Analyse the effect of using the same gamma value on bright and dark photographs.
- Compare traditional transformations with a pretrained enhancement model.
- Select a method based on detail preservation, clipping, processing time and historical accuracy.

3. Operating Parameters and Assumptions

Parameter	Operating data
Image resolution	A4, 2480 × 3508 pixels (300 DPI)
Image format	8-bit grayscale; intensity range 0–255
Text stroke width	2–4 pixels
Faded ink contrast	30–50 grey levels darker than paper
Lighting	Uneven illumination with one-sided shadows
Noise	Gaussian noise ($\sigma \approx 6$) and 1% salt-and-pepper noise
Scanner noise	Horizontal stripes every 6 pixel rows
Processing time	Maximum 3 seconds per page
Target accuracy	Text segmentation Dice score ≥ 0.90 (not a direct Task 1 metric)
Hardware	4 CPU cores, 16 GB RAM, no GPU
Software	Python, OpenCV and NumPy

Additional assumptions: all images are converted to 8-bit grayscale before comparison; the same source image is supplied to every technique; no missing facial or historical details are generated; percentile limits are used to reduce the influence of isolated extreme pixels; processing time is measured on the same computer; and the complete pipeline must be timed separately from the individual transformation.

4. Basic Concept of Grey-Level Transformation

The general transformation is $s = T(r)$, where r is the input intensity, s is the output intensity and T is the transformation function. For an 8-bit image, both input and output values are restricted to 0–255. A suitable mapping increases useful differences between subject and background while avoiding unnecessary black or white clipping.

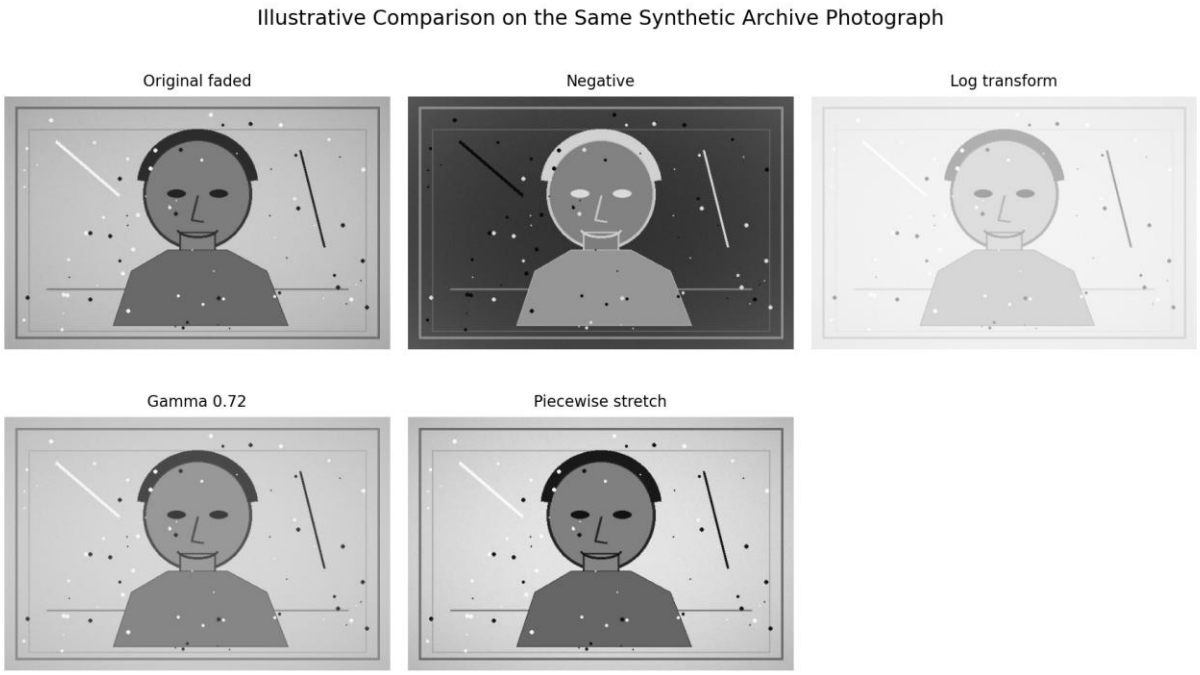


Figure 1. Illustrative output comparison using the same synthetic faded photograph. This is a demonstration image, not a measured archive photograph.

5. Technique A – Negative and Log Transformation

5.1 Negative Transformation

Negative transformation reverses the intensity values of an image. Dark pixels become bright and bright pixels become dark. The mathematical expression is s = L − 1 − r. For an 8-bit image, L = 256, so s = 255 − r.

Input r	Output s = 255 − r
0	255
50	205
100	155
150	105
200	55
255	0

Working procedure: load the grayscale image; subtract each pixel from 255; store the result as an 8-bit image; and save or display the negative output.

Advantages: simple implementation, very low processing cost and useful visual inspection of certain dark regions. Disadvantages: it reverses the original photographic appearance, changes the natural relationship between faces and background, and is not suitable as the final restoration output.

5.2 Log Transformation

Log transformation expands lower intensity values and compresses higher intensity values. It can reveal information in dark areas of an underexposed photograph. The equation is s = c log(1 + r), where c = 255 / log(256) for an 8-bit image. The scaling constant keeps the result within the display range.

Working procedure: convert pixels to floating point; apply log1p to avoid numerical problems at zero; multiply by c; clip to 0–255; and convert to uint8.

Advantages: improves dark-region visibility and compresses bright values. Disadvantages: may amplify noise, make an image look too bright and compress distinctions in already bright areas. It does not remove scratches, dust or stains.

```
import cv2
import numpy as np
import time

img = cv2.imread("old_photo.jpg", cv2.IMREAD_GRAYSCALE)

start = time.perf_counter()
negative = 255 - img
negative_time = time.perf_counter() - start

start = time.perf_counter()
c = 255 / np.log(256)
log_img = c * np.log1p(img.astype(np.float32))
log_img = np.clip(log_img, 0, 255).astype(np.uint8)
log_time = time.perf_counter() - start

cv2.imwrite("negative.jpg", negative)
cv2.imwrite("log.jpg", log_img)
print("Negative time:", negative_time)
print("Log time:", log_time)
```

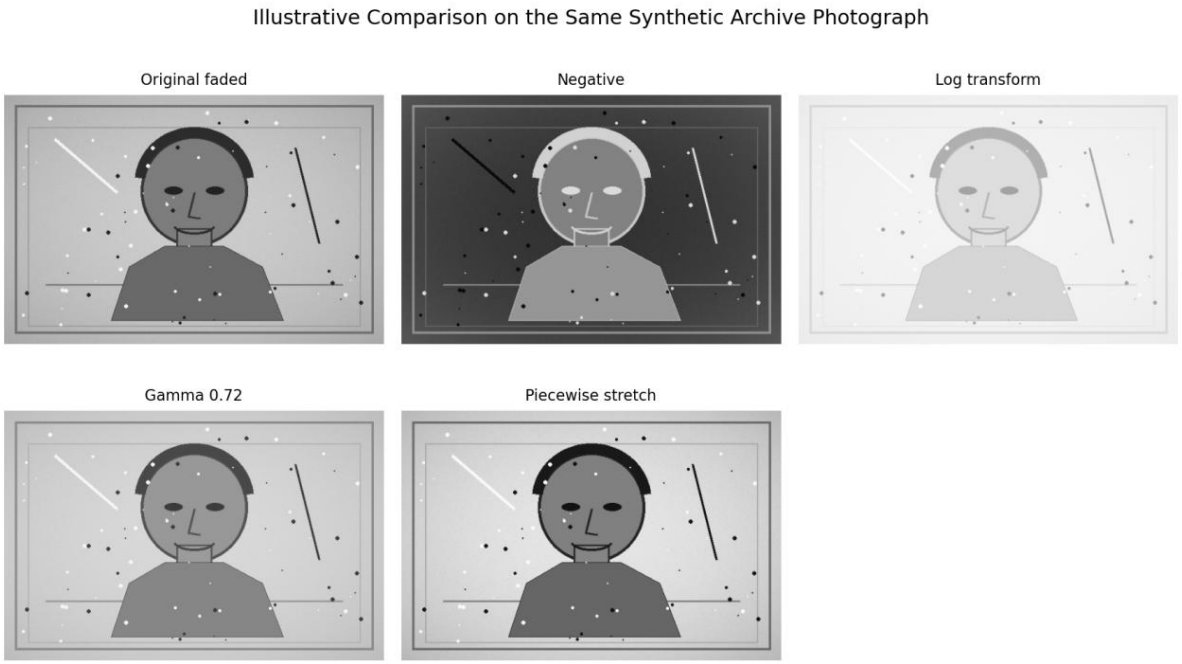


Figure 2. Negative and logarithmic transformation examples are shown beside the original for visual comparison.

6. Technique B – Power-Law (Gamma) Transformation

6.1 Introduction and Formula

Gamma transformation controls image brightness through a power-law mapping. The normalized form is $s = 255 \times (r/255)^{\gamma}$, where r is the input intensity and γ is the gamma value.

When $\gamma < 1$, dark values become brighter. When $\gamma = 1$, the image remains unchanged. When $\gamma > 1$, the image becomes darker. For example, $\gamma = 0.5$ can improve a dark photograph, while $\gamma = 1.5$ can reduce excessive brightness.

Input intensity	Gamma 0.5	Gamma 1.0	Gamma 1.5
25	80	25	8
50	113	50	31
100	159	100	99
150	195	150	180
200	226	200	226
255	255	255	255

Values are rounded illustrative examples. Gamma is a global mapping, so it affects every pixel according to the same curve.

6.2 Working Procedure

7. Read the original grayscale photograph.
8. Normalize intensity values to the range 0–1.
9. Select a gamma value based on image brightness.
10. Raise normalized pixel values to the gamma power.
11. Multiply by 255, clip to the valid range and convert to uint8.
12. Save the output and compare it with the original.

6.3 Python Implementation

```
import cv2
import numpy as np
import time

img = cv2.imread("old_photo.jpg", 0)

def gamma_transform(image, gamma):
    start = time.perf_counter()
    normalized = image.astype(np.float32) / 255.0
    result = 255.0 * (normalized ** gamma)
    result = np.clip(result, 0, 255).astype(np.uint8)
    elapsed = time.perf_counter() - start
    return result, elapsed

gamma_dark, t1 = gamma_transform(img, 0.5)
gamma_normal, t2 = gamma_transform(img, 1.0)
gamma_bright, t3 = gamma_transform(img, 1.5)

cv2.imwrite("gamma_05.jpg", gamma_dark)
cv2.imwrite("gamma_10.jpg", gamma_normal)
cv2.imwrite("gamma_15.jpg", gamma_bright)
print(t1, t2, t3)
```

Advantages: gamma correction is simple, fast and gives direct brightness control. Disadvantages: a fixed value may not suit all photographs; brightening can expose noise; darkening can hide faint details; and global gamma does not independently correct local shadows.

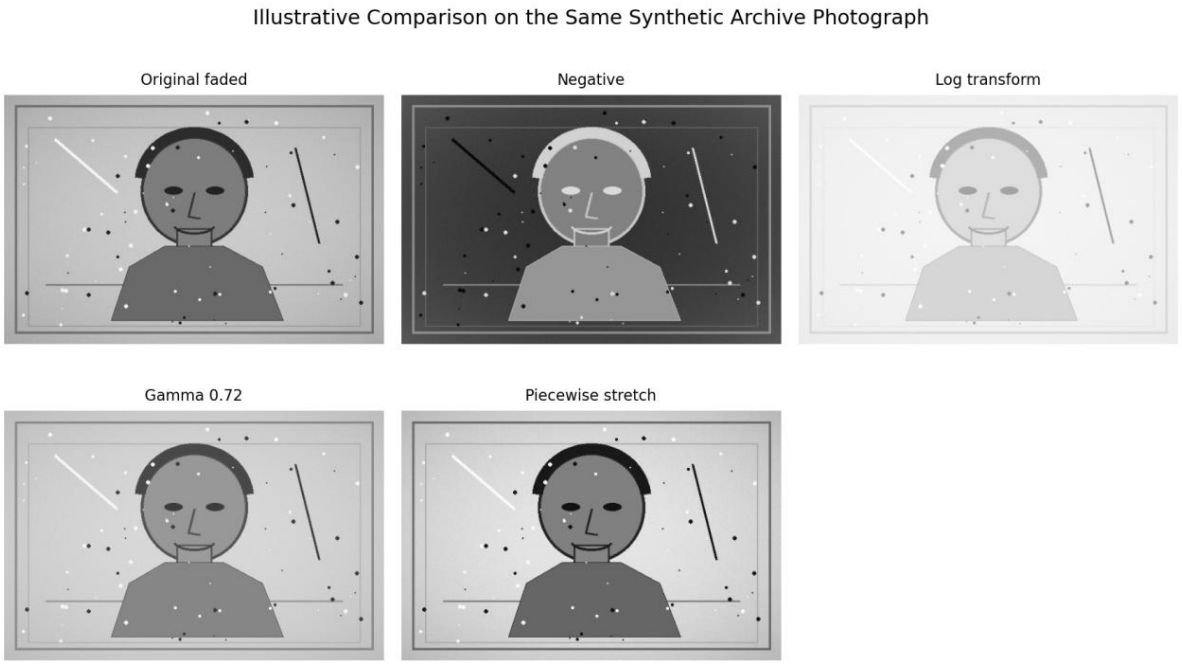


Figure 3. Illustrative gamma and other transformation outputs from the same synthetic photograph.

7. Technique C – Piecewise-Linear Contrast Stretching

7.1 Introduction

Piecewise-linear contrast stretching expands a narrow range of useful grey levels. It uses separate straight-line segments for dark, middle and bright intensity ranges. This allows the operator to increase contrast without reversing the image.

The transformation is defined by control points (r1,s1) and (r2,s2), along with the endpoints (0,0) and (255,255). The lower segment controls shadows, the middle segment controls the main contrast range, and the upper segment controls highlights.

For an illustrative example, r1 = 60, s1 = 25, r2 = 190 and s2 = 230. These values are examples only; actual control points should be selected from the input photograph.

Input intensity	Illustrative output
0	0
30	13
60	25
100	93
130	127
160	161
190	230
220	243

255	255

7.2 Working Procedure

13. Load the original grayscale photograph.
14. Estimate useful lower and upper intensity limits.
15. Choose control points for shadow, middle-tone and highlight regions.
16. Construct a piecewise-linear lookup table.
17. Apply the mapping to all pixels.
18. Inspect faces, clothing, background and clipped areas.

7.3 Python Implementation

```
import cv2
import numpy as np
import time

img = cv2.imread("old_photo.jpg", 0)

start = time.perf_counter()
r1, r2 = np.percentile(img, [2, 98])
x = np.arange(256)
mapping = np.interp(
    x, [0, r1, r2, 255], [0, 25, 230, 255]
)
result = mapping[img].astype(np.uint8)
elapsed = time.perf_counter() - start

cv2.imwrite("contrast_stretched.jpg", result)
print("Processing time:", elapsed)
```

Percentile-based control points reduce the influence of a small number of extreme pixels. However, the output still requires visual review because scratches and stains may influence local intensity distributions.

Advantages: flexible contrast control, good separation of useful intensity levels, preservation of original intensity ordering and low computational cost. Disadvantages: poor control points can create excessive contrast; noise may become more visible; and the method cannot reconstruct missing regions or remove physical damage.

8. Comparison of the Three Techniques

Parameter	Negative and Log	Gamma	Piecewise-linear
Contrast improvement	Log can improve dark range; negative reverses tones	Moderate; depends on γ	Flexible and often strong

Faint detail recovery	Log may reveal dark details	Useful in dark images with $\gamma < 1$	Good if details fall in expanded range
Highlight preservation	Log compresses highlights	Depends on γ	Can be adjusted separately
Noise behaviour	May reveal noise	May reveal noise	May reveal noise
Original appearance	Negative changes appearance	Usually maintained	Usually maintained
Control	Limited	One global parameter	Multiple control points
Computational cost	Low	Low	Low
Final restoration suitability	Limited	Useful supporting method	Strong candidate

Subject-to-background contrast can be estimated as $C = |\mu s - \mu b|$, where μs is the mean intensity of a selected subject region and μb is the mean intensity of a selected background region. Higher contrast is not automatically better: excessive contrast can erase natural facial gradations.

Clipping percentage is $Pc = (Nc/N) \times 100$, where Nc is the number of pixels at the black or white limit and N is the total number of pixels. Low clipping is preferred when preserving historical details.

9. Transformation Curves

Transformation curves show how every possible input intensity is mapped to an output intensity. The diagonal identity line represents no change. Negative transformation slopes downward. Log transformation rises rapidly at low values. Gamma curves move above or below the identity line depending on gamma. Piecewise stretching consists of connected linear segments.

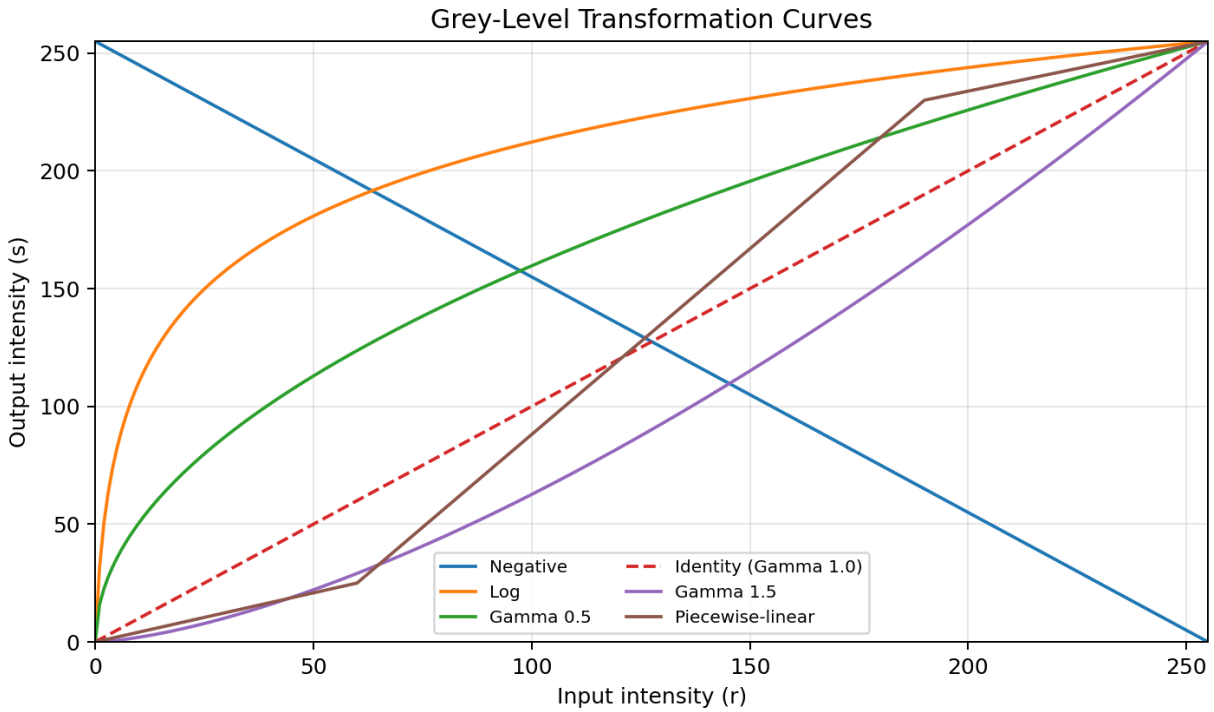


Figure 4. Transformation curves for negative, log, gamma and piecewise-linear mappings.

```
import numpy as np
import matplotlib.pyplot as plt

r = np.arange(256)
negative = 255 - r
log_curve = 255 * np.log1p(r) / np.log(256)
gamma05 = 255 * (r / 255.0) ** 0.5
gamma15 = 255 * (r / 255.0) ** 1.5
piecewise = np.interp(r, [0, 60, 190, 255], [0, 25, 230, 255])

plt.plot(r, negative, label="Negative")
plt.plot(r, log_curve, label="Log")
plt.plot(r, gamma05, label="Gamma 0.5")
plt.plot(r, gamma15, label="Gamma 1.5")
plt.plot(r, piecewise, label="Piecewise")
plt.xlabel("Input intensity")
plt.ylabel("Output intensity")
plt.title("Grey-level transformation curves")
plt.legend()
plt.grid()
plt.show()
```

10. Input and Output Histogram Analysis

A grayscale histogram counts the number of pixels at each intensity level from 0 to 255. A faded image may have most values concentrated in a narrow range. Transformation changes the distribution: negative reverses it; log expands darker values and compresses brighter values; gamma shifts values according to gamma; and piecewise stretching spreads the selected intensity range.

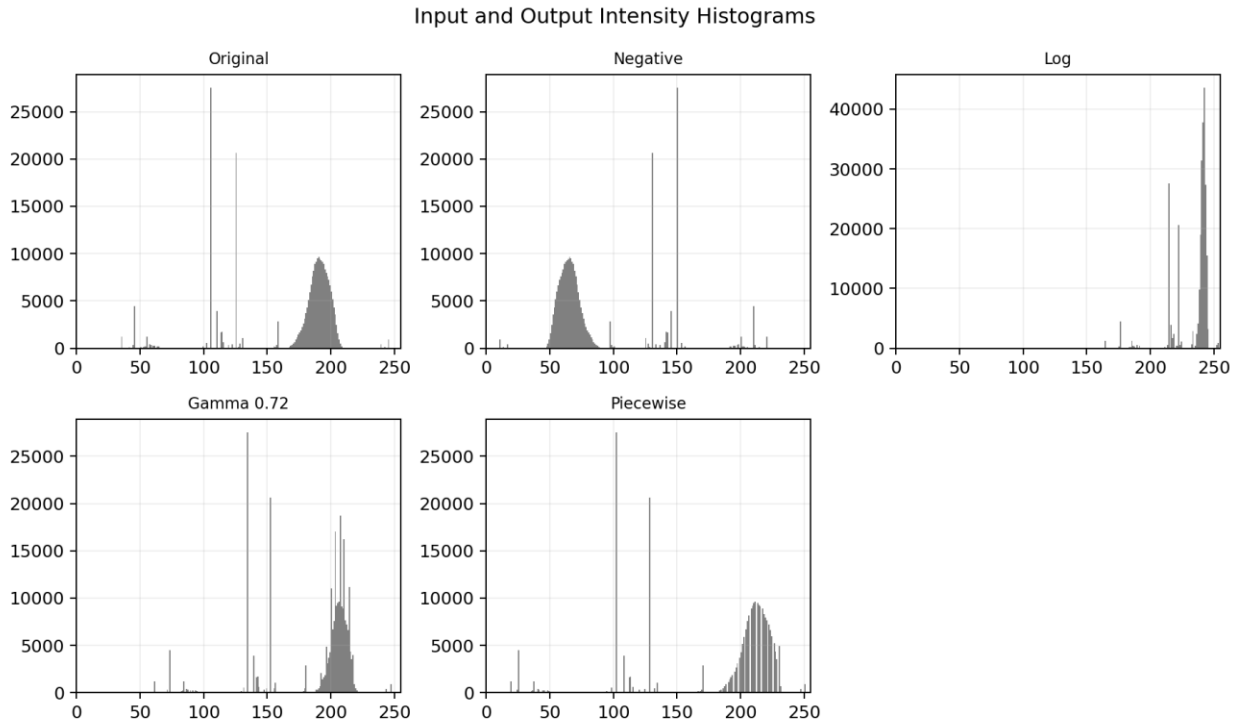


Figure 5. Histograms for the same illustrative image before and after transformations.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("old_photo.jpg", 0)
negative = 255 - img
log_img = np.clip(
    255 / np.log(256) * np.log1p(img.astype(np.float32)),
    0, 255
).astype(np.uint8)
gamma_img = np.clip(
    255 * (img / 255.0) ** 0.5, 0, 255
).astype(np.uint8)

r1, r2 = np.percentile(img, [2, 98])
mapping = np.interp(np.arange(256), [0, r1, r2, 255], [0, 25, 230, 255])
piecewise = mapping[img].astype(np.uint8)
```

```
images = [img, negative, log_img, gamma_img, piecewise]
titles = ["Original", "Negative", "Log", "Gamma", "Piecewise"]
for image, title in zip(images, titles):
    plt.figure()
    plt.hist(image.ravel(), bins=256, range=(0, 256))
    plt.title(title)
    plt.xlabel("Grey level")
    plt.ylabel("Pixel count")
    plt.show()
```

A wider histogram may indicate that contrast has increased, but histogram width alone cannot prove that the photograph is better. The output must also be inspected for noise, clipped highlights, blocked shadows and preservation of meaningful details.

11. Bit-Plane Slicing

An 8-bit grayscale pixel is represented by eight binary digits. Bit 7 is the most significant bit (MSB), while bit 0 is the least significant bit (LSB). Bit-plane slicing extracts one bit position from every pixel and displays the result as a binary image.

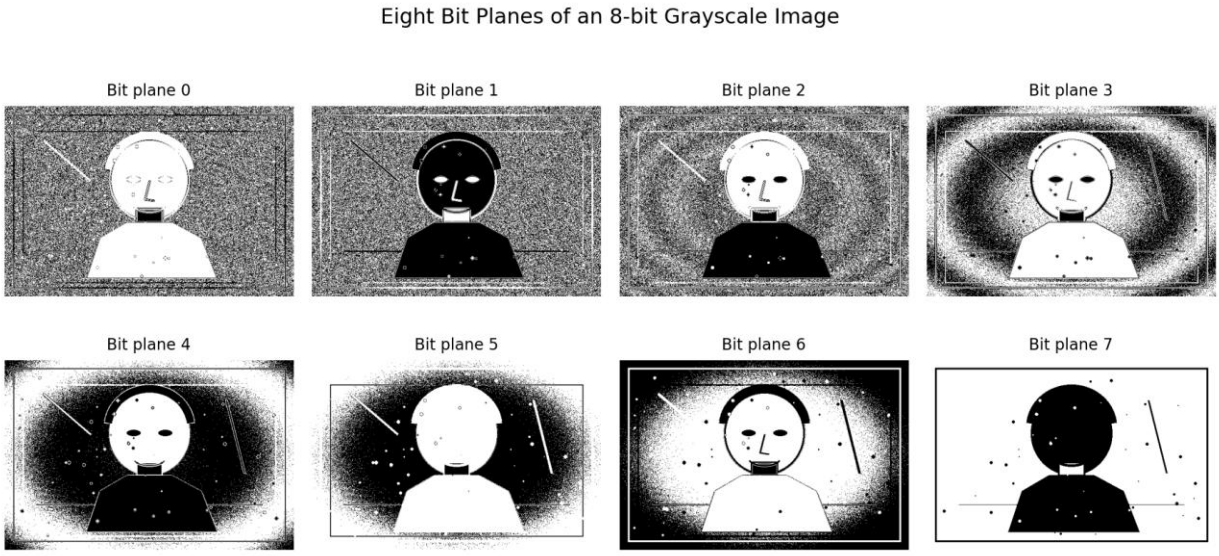


Figure 6. Eight bit planes of the illustrative grayscale photograph.

Bit plane	Typical information
7	Large structures and major intensity regions
6	Major subject and background structure
5	Medium-scale shapes and tonal information
4	Additional texture and local details
3	Fine texture and smaller variations
2	Small variations; may include noise

1	Low-level variations and noise
0	Least significant changes; often sensitive to noise

The higher bit planes commonly preserve the broad structure of a photograph. Middle bit planes may contain useful texture and fine details. Lower planes are more sensitive to Gaussian noise, scanner variations and small intensity disturbances. However, lower planes can also contain genuine fine details, such as thin hair, narrow lines and small facial features. Therefore, removing all lower planes would risk destroying historical information.

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("old_photo.jpg", cv2.IMREAD_GRAYSCALE)

for bit in range(8):
    plane = ((img >> bit) & 1) * 255
    plt.subplot(2, 4, bit + 1)
    plt.imshow(plane, cmap="gray")
    plt.title("Bit plane " + str(bit))
    plt.axis("off")

plt.tight_layout()
plt.show()
```

12. Effect of Applying the Same Gamma to Bright and Dark Photographs

A single gamma value affects all pixels according to the same global curve. Therefore, it cannot be expected to work equally well on photographs with very different exposure conditions.

Image condition	Gamma	Expected effect	Suitability
Dark	0.5	Brightens dark values	Often suitable
Dark	1.0	No change	Limited correction
Dark	1.5	Darkens further	Usually unsuitable
Bright	0.5	Brightens further	Usually unsuitable
Bright	1.0	No change	Limited correction
Bright	1.5	Darkens image	May be suitable

An adaptive rule can choose an initial gamma value using the mean intensity $\mu = (1/N)\ \Sigma\ Ii$. For example, $\gamma = 0.7$ when $\mu < 100$, $\gamma = 1.0$ when $100 \leq \mu \leq 160$, and $\gamma = 1.3$ when $\mu > 160$. These are starting values for experimentation, not universal thresholds.

```
import cv2
import numpy as np

img = cv2.imread("old_photo.jpg", 0)
mean_value = np.mean(img)

if mean_value < 100:
   gamma = 0.7
elif mean_value <= 160:
   gamma = 1.0
else:
   gamma = 1.3

result = 255 * (img.astype(np.float32) / 255.0) ** gamma
result = np.clip(result, 0, 255).astype(np.uint8)

cv2.imwrite("adaptive_gamma.jpg", result)
print("Mean intensity:", mean_value)
print("Selected gamma:", gamma)
```

Mean brightness alone cannot describe local illumination. A photograph may contain both dark shadows and bright paper regions. Such cases may require local enhancement or illumination correction, with careful review to avoid changing the historical tone.

13. Comparison with a Pretrained Enhancement Model

A pretrained low-light enhancement model can be included as an additional benchmark. Zero-DCE++ is one example of a deep-learning method that estimates enhancement curves for low-light images. The model is not a guaranteed restoration tool: it may improve visibility, but it does not guarantee faithful preservation of historical facial or object details.

Parameter	**Traditional transformations**	**Pretrained model (e.g., Zero-DCE++)**
Method	Direct mathematical pixel mapping	Neural network predicts enhancement curves
Weights	Not required	Pretrained weights required
Brightness	Controlled by selected formula	Learned enhancement
Local behaviour	Usually global in these examples	May adapt spatially
Noise removal	Not provided automatically	Not guaranteed
CPU speed	Generally low-cost	Model-dependent; must be benchmarked
Historical fidelity	Mapping is transparent and controllable	Output requires careful review

Missing detail reconstruction	Not performed	Should not be trusted to reconstruct facts

For a fair experiment, use the same images and resolution, record model loading separately from inference, measure CPU-only inference time, and inspect faces, clothing, background and edge details. The model should not be declared better merely because the output looks brighter. If it introduces or changes important details, the result must be rejected for archival use.

Reference: Guo et al., 'Zero-Reference Deep Curve Estimation for Low-Light Image Enhancement,' CVPR 2020. The official project implementation is available at https://github.com/Li-Chongyi/Zero-DCE.

14. Experimental Performance Evaluation

14.1 Evaluation Measures

Mean brightness: $\mu = (1/N) \Sigma Ii$. It measures average pixel intensity.

Standard deviation: $\sigma = \text{sqrt}((1/N) \Sigma (Ii - \mu)^2)$. It measures intensity spread and is often used as a simple contrast indicator.

Subject-to-background contrast: $C = |\mu s - \mu b|$. The subject and background regions should be selected consistently across all methods.

Clipping percentage: $Pc = (Nc/N) \times 100$. Count pixels at 0 or 255 after transformation.

Processing time: $T = Tend - Tstart$. Measure using time.perf_counter() and repeat several times for a stable average.

Detail preservation: visually inspect the same facial, clothing and background regions at the same magnification. If a clean reference image is unavailable, do not claim a numerical restoration accuracy.

14.2 Evaluation Code

```
import cv2
import numpy as np
import time

img = cv2.imread("old_photo.jpg", 0)

def evaluate(name, image, elapsed):
    mean = np.mean(image)
    contrast = np.std(image)
    clipped = np.sum((image == 0) | (image == 255))
    clipping = 100.0 * clipped / image.size

    print("\\nTechnique:", name)
```

```
    print("Mean brightness:", round(mean, 2))
    print("Standard deviation:", round(contrast, 2))
    print("Clipping:", round(clipping, 2), "%")
    print("Time:", round(elapsed, 4), "seconds")

# Apply each transformation and time it separately.
start = time.perf_counter()
negative = 255 - img
t_negative = time.perf_counter() - start

start = time.perf_counter()
log_img = 255 / np.log(256) * np.log1p(img.astype(np.float32))
log_img = np.clip(log_img, 0, 255).astype(np.uint8)
t_log = time.perf_counter() - start

start = time.perf_counter()
gamma_img = np.clip(255 * (img / 255.0) ** 0.7, 0, 255).astype(np.uint8)
t_gamma = time.perf_counter() - start

start = time.perf_counter()
r1, r2 = np.percentile(img, [2, 98])
mapping = np.interp(np.arange(256), [0, r1, r2, 255], [0, 25, 230, 255])
piecewise = mapping[img].astype(np.uint8)
t_piecewise = time.perf_counter() - start

evaluate("Negative", negative, t_negative)
evaluate("Log", log_img, t_log)
evaluate("Gamma", gamma_img, t_gamma)
evaluate("Piecewise", piecewise, t_piecewise)
```

The code prints measurements from the actual input image. Since no archive photograph dataset or execution output was supplied for this document, numeric performance results are intentionally not fabricated. Run the program on the archive dataset and enter the measured values in the table below.

Metric	Negative	Log	Gamma	Piecewise
Mean brightness	Measured	Measured	Measured	Measured
Standard deviation	Measured	Measured	Measured	Measured
Subject-background contrast	Measured	Measured	Measured	Measured
Clipping percentage	Measured	Measured	Measured	Measured
Processing time (seconds)	Measured	Measured	Measured	Measured
Face/clothing/background	Review	Review	Review	Review

preservation				

15. Processing-Time Analysis

The A4 image contains 2480 × 3508 = 8,699,840 pixels. The traditional methods process the image using vectorized array operations, so their basic computational complexity is O(N), where N is the number of pixels. Negative, log, gamma and piecewise mappings are all suitable for CPU implementation in principle. Actual timing depends on the computer, memory bandwidth, software version and image I/O.

The three-second requirement applies to the complete page-processing pipeline, not only the mathematical mapping. Image loading, noise handling, damage detection, enhancement and saving must all be included in the end-to-end timing. Run several trials and report the mean or median time.

16. Technique Selection and Justification

Piecewise-linear contrast stretching is selected as the primary grey-level enhancement technique. It offers independent control of dark, middle and bright ranges, improves the useful intensity spread and maintains the original ordering of pixel intensities. It is computationally inexpensive and does not require a GPU or pretrained weights.

Selection criterion	Justification
Contrast enhancement	Expands the useful intensity range.
Faint details	Can make details in the selected range more visible.
Historical appearance	Does not reverse the image as negative transformation does.
Highlight and shadow control	Separate control points allow careful adjustment.
Processing speed	Simple vectorized mapping is computationally inexpensive.
Hardware	Runs on CPU with OpenCV and NumPy.
Limitation	Does not remove scratches, stains, creases or reconstruct missing areas.

Gamma correction can be used as a supporting method when a photograph needs global brightness adjustment. Log transformation is useful for inspecting dark regions. Negative transformation is mainly useful for visual inspection. The final choice should be confirmed using real measurements and visual review of the archive photographs.

17. Overall System Workflow

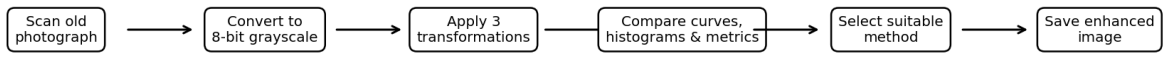


Figure 7. Workflow for Task 1 grey-level transformation analysis.

19. Image acquisition: scan old photographs at the specified resolution.
20. Preprocessing: convert the image into 8-bit grayscale.
21. Noise inspection: identify Gaussian noise, impulse noise and scanner stripes.
22. Transformation: apply negative, log, gamma and piecewise mappings to the same input.
23. Analysis: plot transformation curves, histograms and bit planes.
24. Evaluation: measure brightness, contrast, clipping and processing time.
25. Selection: choose the method that best balances visibility and historical fidelity.
26. Output: save the enhanced image for later restoration stages.

18. Result and Discussion

The three candidate techniques have been described and implemented using Python, OpenCV and NumPy. Negative transformation reverses the original intensity relationships and is therefore unsuitable as the final natural-looking restoration. Log transformation improves visibility in dark regions but can amplify noise. Gamma transformation offers simple brightness control, but the gamma value must be adapted to the image condition. Piecewise-linear contrast stretching provides the most direct control over the useful intensity range and is selected as the primary enhancement method.

The transformation curves explain the mapping behaviour. Histograms show how pixel distributions change. Bit-plane slicing shows that high-order planes contain broad structure while lower-order planes contain both fine information and noise. A pretrained model can be included as a benchmark, but archival outputs must be checked for unintended changes.

19. Conclusion

This task analysed negative and log transformation, power-law gamma transformation and piecewise-linear contrast stretching for faded historical photographs. The methods were compared in terms of contrast improvement, faint detail visibility, clipping, processing cost and preservation of the original photographic tone. Piecewise-linear contrast stretching is selected as the primary technique because it offers flexible intensity control, low computational cost and a transparent mapping that does not invent missing information. Gamma correction can support brightness adjustment where necessary. Damage detection and repair remain separate stages of the complete Old Photograph Restoration System.

TASK 2 – HISTOGRAM PROCESSING AND ENHANCEMENT [4 Marks]

Old photographs have different exposure levels, faded regions and uneven tonal distribution. Apply and compare three histogram-based enhancement techniques.

Candidate Techniques

- A. Global Histogram Equalization
- B. Histogram Matching
- C. Adaptive Histogram Equalization (CLAHE)

Questions

27. Implement all three techniques using Python/OpenCV.
28. Compare detail visibility, contrast improvement, noise amplification and correction of faded or overexposed regions.
29. Generate histograms and Cumulative Distribution Function (CDF) plots.
30. Analyse why global histogram equalization may fail when a photograph contains both very dark and very bright regions.
31. Experiment with different CLAHE tile sizes and clip limits.
32. Examine the effect of applying global histogram equalization to a photograph in which a large area has a nearly uniform background.

Evidence Expected: Histograms, CDF plots, before-and-after images, contrast/entropy comparison and justified parameter selection.

1. Aim

To implement and compare Global Histogram Equalization, Histogram Matching and Contrast Limited Adaptive Histogram Equalization (CLAHE) for improving old photographs with faded, dark, bright and unevenly exposed regions.

2. Introduction

A digital grayscale image contains intensity values from 0 (black) to 255 (white). A histogram counts how many pixels occur at each intensity. An image with most pixels grouped in a narrow intensity range often looks faded or low in contrast. Histogram processing changes the intensity distribution to make useful details easier to see.

Global Histogram Equalization (GHE) uses one transformation for the whole image. Histogram Matching transforms an image so that its intensity distribution resembles a chosen reference image or target histogram. CLAHE divides the image into small tiles, equalizes each tile and limits excessive amplification by clipping the histogram.

3. Experimental image and setup

For reproducibility, this report uses an illustrative grayscale test photograph created to contain dark areas, bright areas, a faded middle-tone region, texture noise and a large nearly uniform background. The same methods can be applied to a scanned family photograph or any grayscale image.

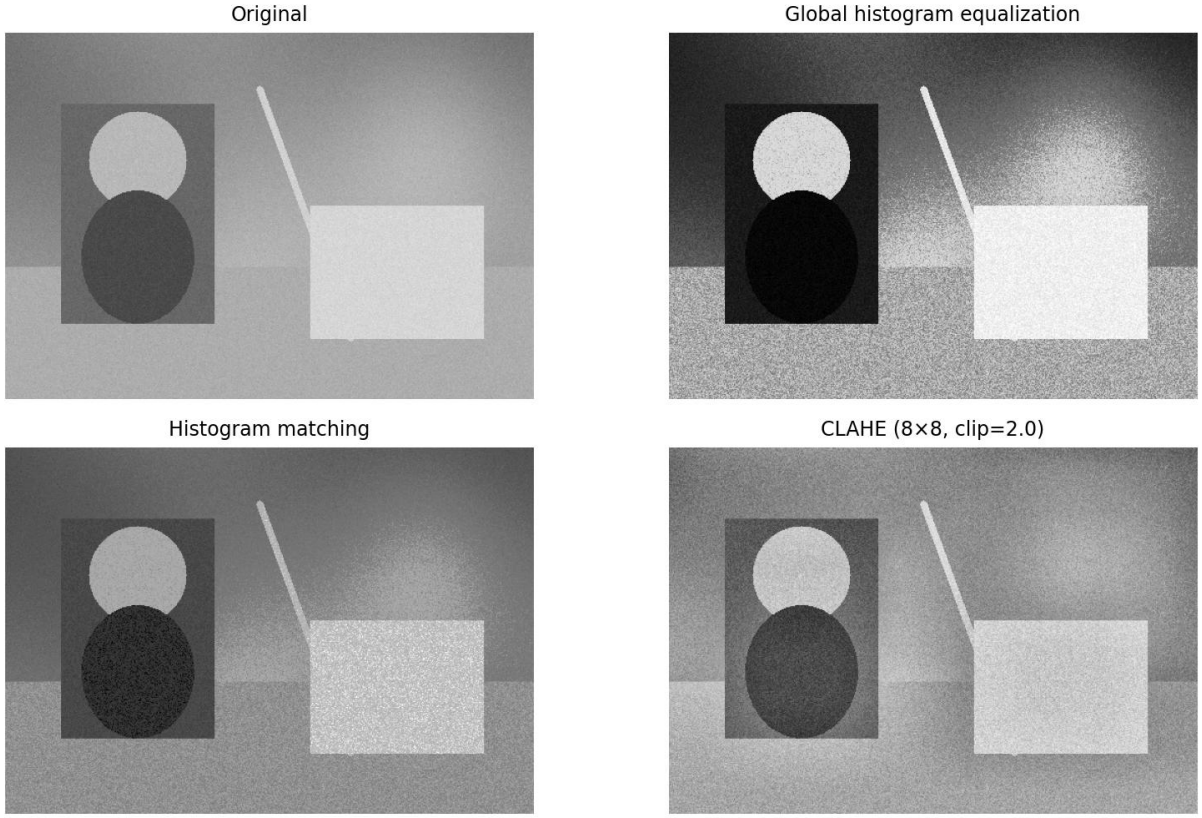


Figure 1. Original test image and results from the three enhancement methods.

4. Software requirements

Python 3.x, OpenCV (cv2), NumPy and Matplotlib.

Install packages using: pip install opencv-python numpy matplotlib

Use a grayscale input image so that the histogram and CDF are easy to interpret.

5. Method A – Global Histogram Equalization

Global Histogram Equalization redistributes intensity values across the available gray-level range. It first calculates the histogram and its cumulative distribution function (CDF). The CDF is normalized and used as a mapping from each original intensity to a new intensity.

Mathematical expression

For gray level r, the output is s = round[255 × CDF(r)]. The CDF is the cumulative number of pixels up to r divided by the total number of pixels. This spreads frequently occurring intensity values over a wider range.

OpenCV implementation

```
import cv2
import matplotlib.pyplot as plt
```

```
image = cv2.imread("old_photo.jpg", cv2.IMREAD_GRAYSCALE)
if image is None:
    raise FileNotFoundError("Check the image path")

equalized = cv2.equalizeHist(image)

plt.figure(figsize=(8, 4))
plt.subplot(1, 2, 1); plt.imshow(image, cmap="gray")
plt.title("Original"); plt.axis("off")
plt.subplot(1, 2, 2); plt.imshow(equalized, cmap="gray")
plt.title("Global histogram equalization"); plt.axis("off")
plt.tight_layout(); plt.show()
```

Observation

The output generally has a wider intensity spread and stronger overall contrast. Faint edges may become more visible. However, the same mapping is applied to every pixel, regardless of whether it belongs to a dark corner, a bright sky or a faded face. As a result, some regions can become too dark or too bright, and existing noise may also become more noticeable.

Advantages and limitations

Advantages: simple, fast, requires no reference image and improves many low-contrast images.

Limitations: does not adapt to local illumination; may over-enhance noise and may not preserve natural brightness.

GHE is a useful first baseline. It should be inspected visually before being selected as the final restoration method.

6. Method B – Histogram Matching

Histogram Matching (also called histogram specification) changes the source image so that its histogram resembles a selected target histogram. The target can come from a well-exposed photograph of a similar scene or can be a designed distribution. Unlike GHE, the aim is not simply to spread intensities uniformly; it is to reproduce a desired tonal appearance.

Working principle

Calculate the normalized CDF of the source image.

Calculate the normalized CDF of the reference image.

For each source intensity, find the reference intensity with the closest CDF value.

Replace source intensities using this lookup mapping.

Python implementation (NumPy + OpenCV)

```
import cv2
import numpy as np

def histogram_match(source, reference):
    s_values, inverse, s_counts = np.unique(
        source.ravel(), return_inverse=True, return_counts=True)
    r_values, r_counts = np.unique(
        reference.ravel(), return_counts=True)

    s_cdf = np.cumsum(s_counts) / source.size
    r_cdf = np.cumsum(r_counts) / reference.size
    mapped = np.interp(s_cdf, r_cdf, r_values)
    return mapped[inverse].reshape(source.shape).astype(np.uint8)

source = cv2.imread("old_photo.jpg", 0)
reference = cv2.imread("well_exposed_reference.jpg", 0)
if source is None or reference is None:
    raise FileNotFoundError("Check both image paths")

matched = histogram_match(source, reference)
```

Observation and use

A suitable reference can restore a natural tonal balance and make the output look similar to a well-exposed photograph. The choice of reference is important: a reference that is too bright, too dark or from a very different scene can produce an unsuitable result. Histogram matching changes the overall distribution but does not understand objects or semantic details.

When to choose it

Use histogram matching when a trusted reference image or target tonal style is available. For example, a set of scanned photographs can be matched to a representative scan so that their brightness and contrast are more consistent.

7. Method C – Adaptive Histogram Equalization (CLAHE)

Adaptive Histogram Equalization works on local image regions rather than treating the whole photograph as one region. CLAHE is a modified version that limits the height of histogram peaks using a clip limit. This reduces the risk of extreme contrast amplification in nearly uniform areas.

Working principle

Divide the image into tiles (small rectangular regions).

Calculate and equalize the histogram in each tile.

Clip histogram bins that exceed the selected limit and redistribute the excess.

Interpolate between neighboring tiles to avoid visible block boundaries.

OpenCV implementation

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("old_photo.jpg", cv2.IMREAD_GRAYSCALE)
clahe = cv2.createCLAHE(
    clipLimit=2.0,
    tileGridSize=(8, 8)
)
enhanced = clahe.apply(image)

plt.imshow(enhanced, cmap="gray")
plt.title("CLAHE: tile 8x8, clip limit 2.0")
plt.axis("off")
plt.show()
```

Observation

CLAHE often reveals details in both shadowed and highlighted areas because each tile receives a local transformation. The clip limit controls how strongly local contrast is increased. A small clip limit gives a more restrained result, while a high clip limit can reveal more texture but may also amplify grain and scratches.

Why CLAHE is suitable for old photographs

Old photographs may have uneven fading, shadows, stains or local exposure changes. CLAHE can improve a dark face without applying exactly the same intensity mapping to a bright background. It is therefore often a good practical choice when illumination varies across the photograph.

8. Histograms and Cumulative Distribution Function (CDF)

A histogram shows the frequency of each gray level. A narrow peak indicates that many pixels have similar intensity values. A wider spread usually indicates greater global contrast, although a wide histogram alone does not guarantee a visually good image. The CDF is the cumulative proportion of pixels at or below each gray level.

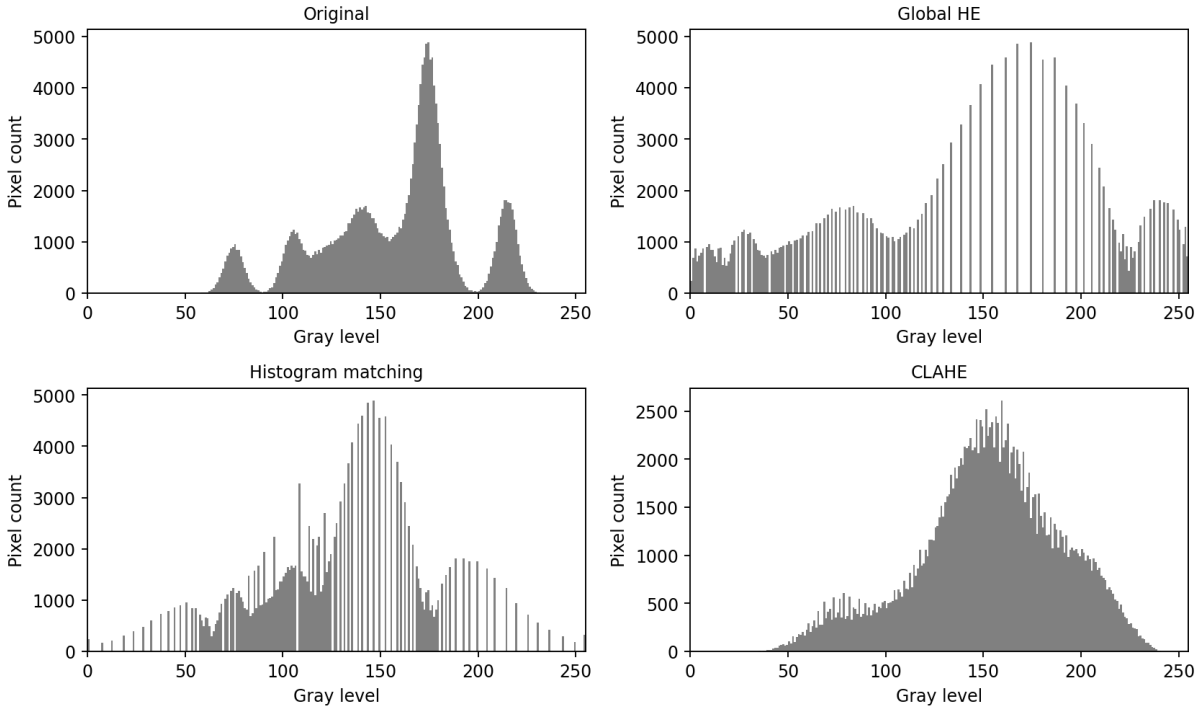


Figure 2. Histograms of the original and enhanced images.

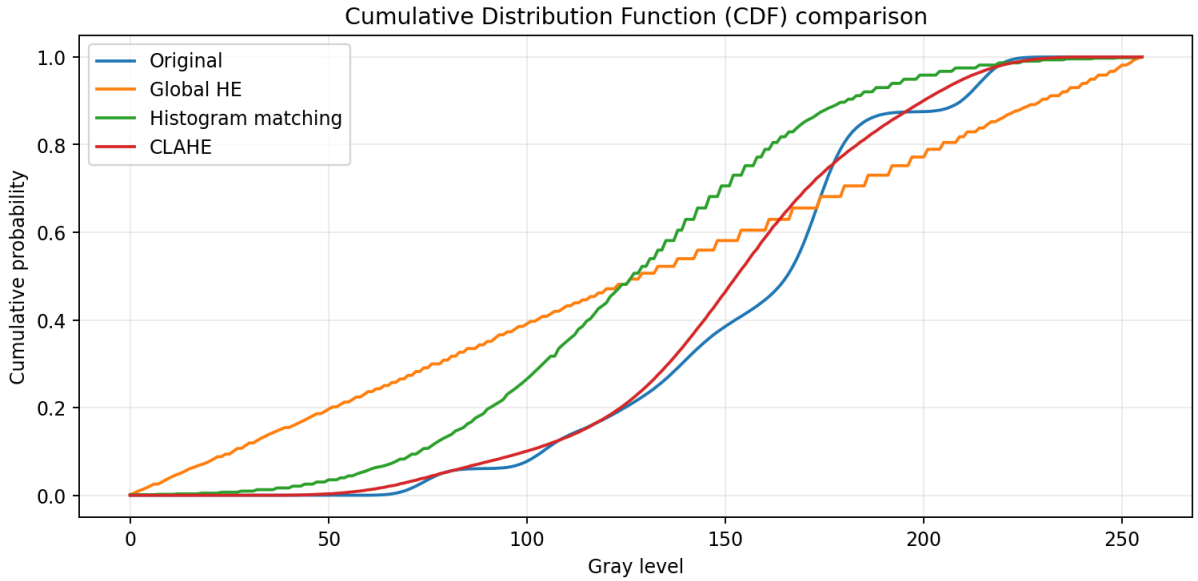


Figure 3. CDF curves for the four image versions.

Interpretation

Original: the intensity distribution is uneven because the test image contains several exposure zones.

Global HE: the output intensities are redistributed across a wider range; the CDF becomes more evenly spread over gray levels that occur in the image.

Histogram Matching: the output distribution follows the chosen reference distribution, so its CDF shape resembles the reference CDF.

CLAHE: each tile is processed locally. The displayed whole-image histogram is the combination of many local transformations and need not be perfectly uniform.

A CDF that rises sharply over a small intensity interval indicates that many pixels occupy that interval. A more gradual rise across the range indicates a broader distribution. CDF plots help explain the transformation, while the image itself must still be checked for natural appearance and artifacts.

9. Comparison of enhancement techniques

The following comparison combines visual inspection with simple numerical measurements. Contrast is represented by the standard deviation of gray-level values. Entropy is calculated from the gray-level probability distribution and measures intensity-distribution complexity in bits.

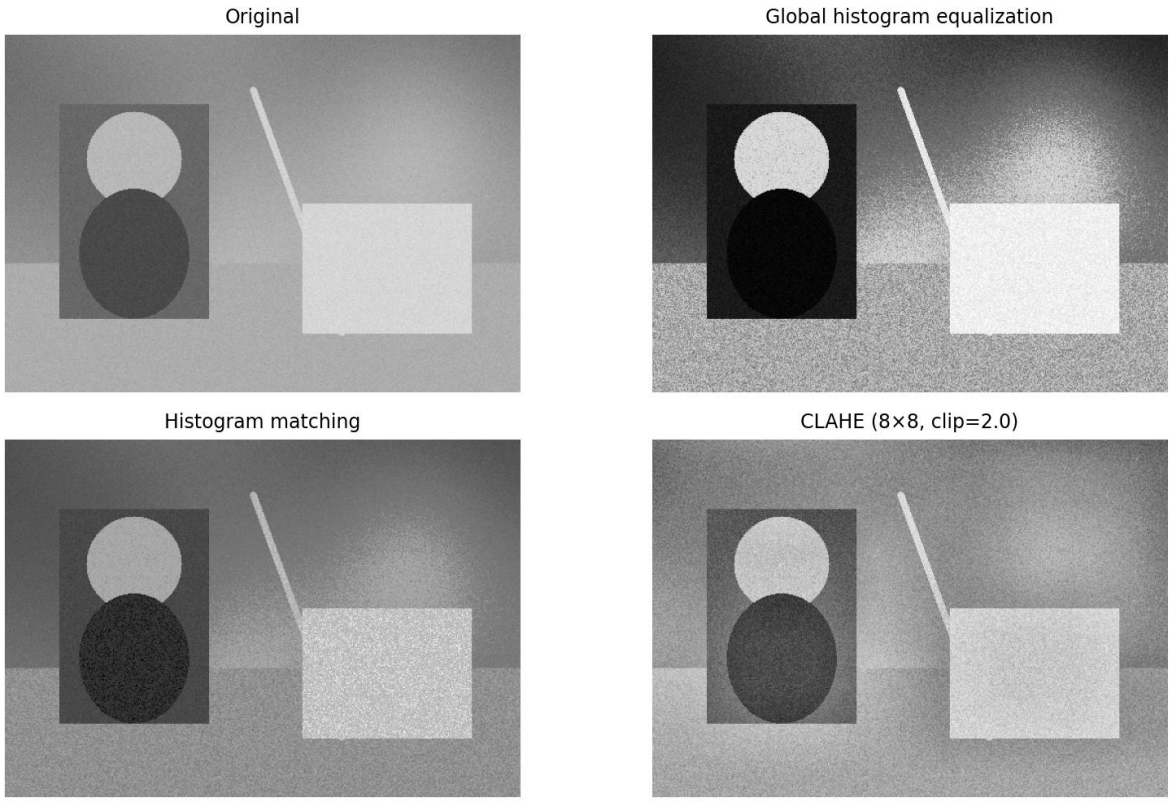


Figure 4. Visual comparison using the same test photograph.

Method	Mean intensity	Contrast (SD)	Entropy (bits)
Original	156.4	36.8	6.836
Global HE	128.9	74.0	6.747
Histogram matching	127.3	42.1	6.646
CLAHE (8x8, clip 2)	151.7	37.3	7.207

Qualitative comparison

Criterion	Global HE	Histogram Matching	CLAHE
Detail visibility	Improves global detail; local areas may be poorly balanced.	Depends on how suitable the reference is.	Usually strong in locally shadowed or faded regions.
Contrast improvement	Strong global increase.	Moves contrast toward the reference appearance.	Local contrast improvement.
Noise amplification	Can amplify noise across the image.	Depends on target mapping; noise may remain visible.	Clip limit helps control noise, but high settings amplify it.
Faded regions	May improve, but one mapping is used everywhere.	Can correct overall faded tone if reference is suitable.	Often effective when fading varies by location.
Overexposed regions	Cannot recover detail that was clipped to pure white.	Cannot recreate information already lost.	Cannot recover clipped pixels; may improve nearby non-clipped detail.

Important: contrast standard deviation and entropy are descriptive measurements, not quality scores. A noisy image can have high entropy and high contrast while looking worse. Therefore, numerical results must be interpreted together with the before-and-after images.

10. Why Global HE may fail; uniform-background experiment

Dark and bright regions in one photograph

Global HE calculates one mapping from the complete image histogram. If a photograph contains a large dark region and a large bright region, both regions influence the same mapping. The dark region may receive too much contrast while the bright region becomes harsh or loses a natural tonal appearance. The mapping cannot independently correct each region.

If a highlight is already clipped to intensity 255, the original texture is absent from the data. Histogram equalization cannot recover that missing information; it can only remap the values that remain.

Effect of a large nearly uniform background

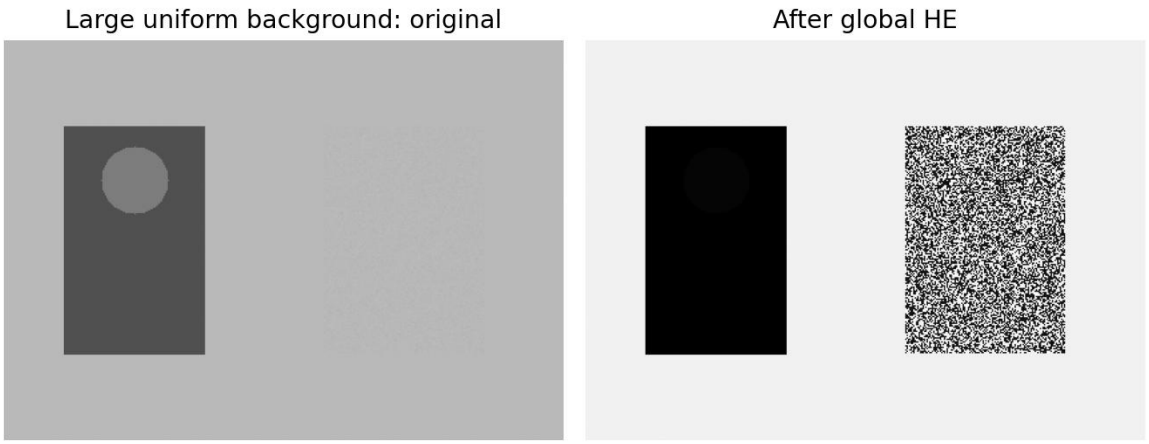


Figure 5. Global equalization of an image dominated by a nearly uniform background.

When most pixels belong to a nearly uniform background, the background dominates the histogram. A large number of pixels are concentrated around one or a few gray levels. The equalization mapping may assign a large part of the available output range around this dominant peak, while the smaller foreground objects receive less favorable tonal allocation. Small background variations and noise may also become more visible.

Conclusion from the experiment

Global HE is not aware of which pixels belong to the subject and which belong to the background. It treats every pixel equally. CLAHE reduces this problem by processing local tiles, although its settings must be chosen carefully to avoid tile artifacts or excessive noise.

11. CLAHE parameter experiment and final conclusion

CLAHE behavior depends mainly on tileGridSize and clipLimit. The tile grid determines the size of local regions. The clip limit controls how much a local histogram peak can be amplified.

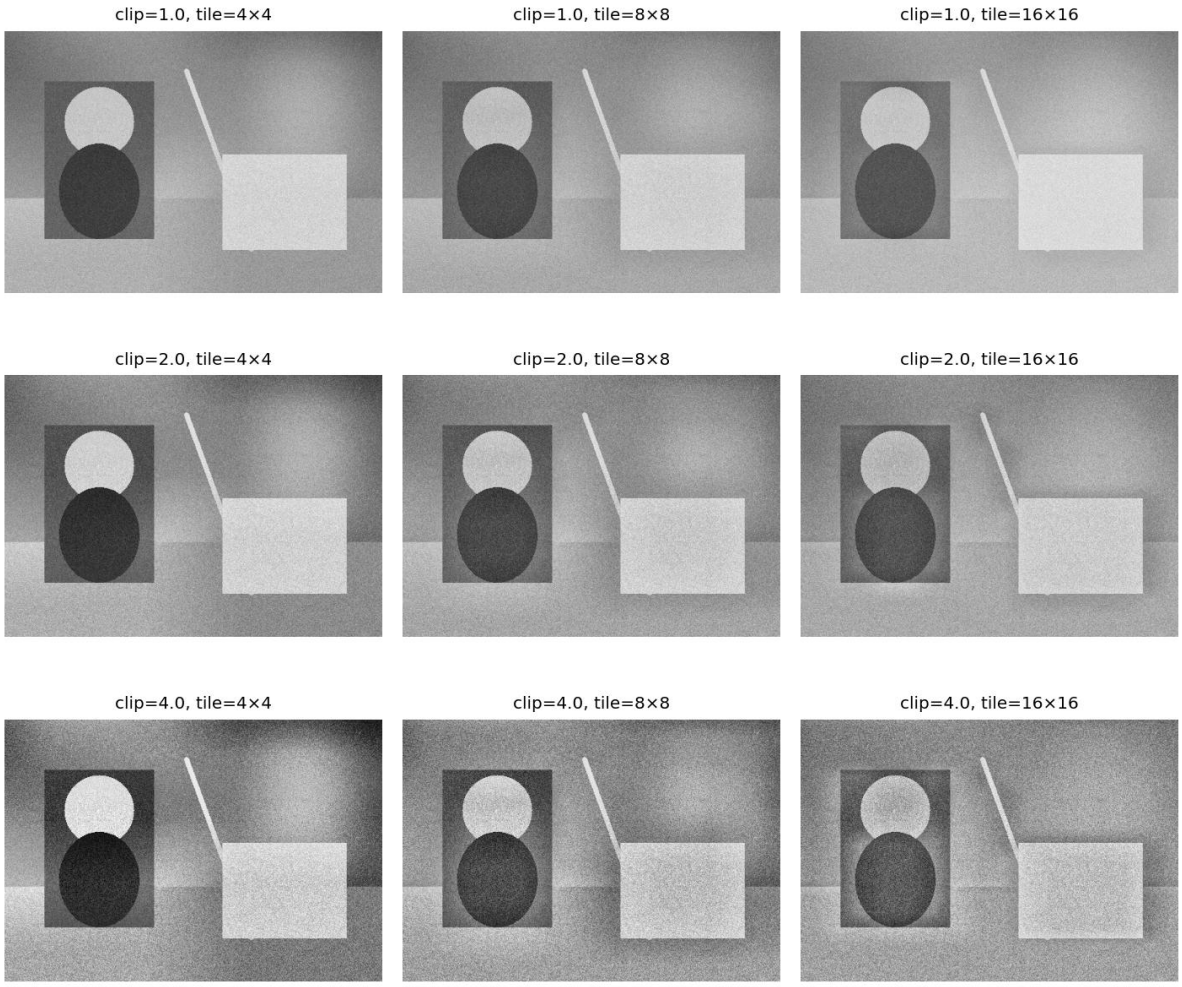


Figure 6. CLAHE results for different tile sizes and clip limits.

Parameter	**Smaller / lower setting**	**Larger / higher setting**
Tile grid	More, smaller tiles: stronger local adaptation; may reveal local detail but can create uneven texture.	Fewer, larger tiles: smoother, more global behavior; may miss small local illumination changes.
Clip limit	Lower clip limit: restrained contrast and less noise amplification.	Higher clip limit: stronger local contrast, but greater risk of grain, scratches and halos.

Justified parameter selection

For this experiment, tileGridSize=(8,8) and clipLimit=2.0 are selected as a balanced starting point. The 8×8 grid provides local adaptation without making each tile excessively small. A clip limit of 2.0 increases detail while limiting extreme amplification. If the photograph is very noisy, try clipLimit=1.0. If illumination changes are very local, compare a smaller tile grid such as (4,4). Always inspect the output at full size.

Complete parameter experiment code

```
for clip in [1.0, 2.0, 4.0]:
   for tile in [(4, 4), (8, 8), (16, 16)]:
      op = cv2.createCLAHE(
         clipLimit=clip, tileGridSize=tile)
      result = op.apply(image)
      print("clip:", clip, "tile:", tile,
           "contrast SD:", result.std())
      cv2.imwrite(
         f"clahe_clip{clip}_tile{tile[0]}.png", result)
```

12. Overall conclusion

All three techniques improve image appearance in different ways. Global Histogram Equalization is fast and useful for general low contrast, but it may not handle mixed illumination or a dominant uniform background well. Histogram Matching is useful when a reliable reference image is available. CLAHE is generally the most suitable of the three for old photographs with uneven local exposure because it adapts to local regions and limits amplification. For the test image, CLAHE with an 8×8 tile grid and clip limit 2.0 is recommended as a balanced starting choice. The final setting should be selected by checking detail visibility, natural brightness, noise and artifacts.

Result

The three histogram-based enhancement methods were implemented using Python and OpenCV. Histograms, CDF plots, before-and-after images, contrast and entropy measurements were generated and compared. The experiments demonstrate the strengths and limitations of each technique and justify the selected CLAHE parameters.

TASK 3 – SPATIAL FILTERING AND IMAGE SMOOTHING [5 Marks]

Old photographs contain film grain, dust particles and salt-and-pepper-like damage. Apply three smoothing filters to reduce noise while preserving facial features, hair, clothing texture and fine edges.

Candidate Techniques

- A. Box (Averaging) Filter
- B. Gaussian Filter
- C. Median Filter

Questions

33. Implement all three filters using 3×3, 5×5 and 7×7 kernels.
34. Compare noise removal, detail preservation and processing time.
35. Calculate PSNR and SSIM for each filter.
36. Plot grayscale intensity profiles across selected facial edges, object boundaries or textured regions.
37. Demonstrate the difference between correlation and convolution using a 3×3 filter mask.

38. Analyse the consequences of using a large 7×7 filter on small details such as eyes, hair strands and narrow object boundaries.
39. Compare a classical filtering technique with a pretrained denoising model such as DnCNN.

Evidence Expected: Filtered images, PSNR/SSIM comparison table, intensity profiles and justified filter/kernel selection.

1. INTRODUCTION

Old photographs may contain film grain, dust particles, scratches and salt-and-pepper noise because of ageing, scanning and poor storage. These defects create unwanted intensity variations and reduce the clarity of facial features, hair, clothes and object boundaries. Spatial filtering is an image-processing method that replaces each pixel using values from a small neighbourhood around it. The purpose of smoothing is to reduce unwanted noise while retaining as much useful information as possible.

This task studies three classical spatial filters: the box (averaging) filter, the Gaussian filter and the median filter. Each method is tested with 3×3, 5×5 and 7×7 neighbourhoods. Their results are compared using visual quality, processing time, Peak Signal-to-Noise Ratio (PSNR), Structural Similarity Index (SSIM), and grayscale intensity profiles. A pretrained DnCNN denoising model is also discussed as a modern learning-based alternative.

2. OBJECTIVES

To implement box, Gaussian and median smoothing using 3×3, 5×5 and 7×7 kernels.

To understand the effect of kernel size on noise removal and fine-detail preservation.

To evaluate denoising quality using PSNR and SSIM when a clean reference image is available.

To study grayscale intensity changes across edges and textured regions.

To demonstrate the difference between correlation and convolution using a 3×3 mask.

To compare classical filters with a pretrained DnCNN denoising model and select a suitable method.

3. THEORY OF SPATIAL FILTERING

A spatial filter processes an image directly in the pixel domain. For each output pixel, a neighbourhood is selected around the corresponding input pixel. In a linear filter, the neighbourhood values are multiplied by mask coefficients and added. In a nonlinear filter such as the median filter, the output is selected by sorting the neighbourhood values. A 3×3 kernel uses 9 pixels, a 5×5 kernel uses 25 pixels and a 7×7 kernel uses 49 pixels.

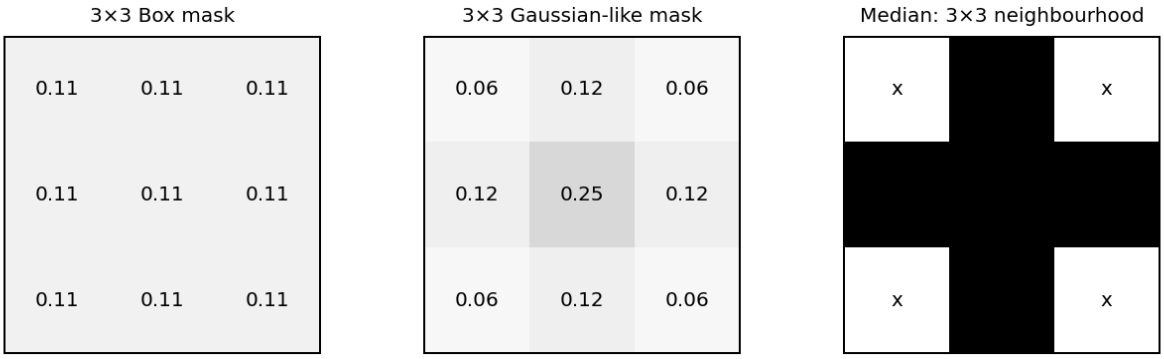


Figure 1. Example 3×3 neighbourhood masks used for smoothing.

3.1 Box (Averaging) Filter

The box filter replaces the centre pixel with the arithmetic mean of all pixels inside the selected kernel. Every pixel receives the same weight. It is simple and fast, and it reduces small random intensity variations. However, averaging mixes foreground and background pixels near an edge. As a result, facial outlines, eyelashes, hair strands and narrow object boundaries may become blurred.

For a k×k kernel, the output is the sum of all neighbourhood pixels divided by k^2. For a 3×3 box filter, each of the nine pixels has a weight of 1/9. Increasing the kernel from 3×3 to 7×7 averages a larger region and produces stronger smoothing, but also increases loss of detail.

3.2 Gaussian Filter

The Gaussian filter is a weighted averaging filter. Pixels near the centre receive higher weights, while pixels farther from the centre receive lower weights. The weights follow a Gaussian-shaped distribution. This makes Gaussian smoothing more natural than uniform averaging and usually reduces high-frequency random noise with less harsh blurring. The Gaussian kernel is normalized so that the sum of all its coefficients is one.

The standard deviation, represented by sigma (σ), controls the spread of the Gaussian. A small sigma gives gentle smoothing. A larger sigma produces stronger smoothing. In OpenCV, sigma can be selected automatically from the kernel size or supplied explicitly. For a fair comparison, the same kernel dimensions should be used for all methods.

3.3 Median Filter

The median filter is a nonlinear filter. It sorts all intensity values in the neighbourhood and replaces the centre pixel with the middle value. For a 3×3 neighbourhood, the fifth value after sorting is selected. Median filtering is especially effective for salt-and-pepper noise because isolated very dark or very bright pixels are rejected when the surrounding pixels are normal. It often preserves edges better than mean-based filters. It may, however, remove very thin lines or tiny details when a large window is used.

4. IMPLEMENTATION USING PYTHON AND OPENCV

The following program reads a photograph, converts it to grayscale, applies all three filters at 3×3, 5×5 and 7×7 sizes, measures execution time, calculates PSNR and SSIM when a clean reference is available, and saves the filtered images. The program assumes that noisy_photo.jpg is the damaged photograph and clean_reference.jpg is the original undamaged photograph. If the original is unavailable, set CLEAN_PATH to None; visual comparison and processing time can still be performed, but PSNR and SSIM cannot be calculated reliably.

```
import cv2
import numpy as np
import time
from skimage.metrics import structural_similarity as ssim

NOISY_PATH = "noisy_photo.jpg"
CLEAN_PATH = "clean_reference.jpg"  # use None if unavailable

img = cv2.imread(NOISY_PATH, cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError("Check the noisy image path.")

ref = None
if CLEAN_PATH:
    ref = cv2.imread(CLEAN_PATH, cv2.IMREAD_GRAYSCALE)
    if ref is not None and ref.shape != img.shape:
        ref = cv2.resize(ref, (img.shape[1], img.shape[0]))

results = []
for k in (3, 5, 7):
    filters = {
        "Box": lambda: cv2.blur(img, (k, k)),
        "Gaussian": lambda: cv2.GaussianBlur(img, (k, k), 0),
        "Median": lambda: cv2.medianBlur(img, k)
    }
    for name, operation in filters.items():
        start = time.perf_counter()
        output = operation()
        elapsed = (time.perf_counter() - start) * 1000
        cv2.imwrite(f"{name}_{k}x{k}.jpg", output)

        if ref is not None:
            psnr_value = cv2.PSNR(ref, output)
            ssim_value = ssim(ref, output, data_range=255)
        else:
            psnr_value, ssim_value = None, None
```

```
        results.append((name, k, elapsed,
                        psnr_value, ssim_value))

for row in results:
    print(row)
```

Implementation notes: cv2.blur() performs box averaging, cv2.GaussianBlur() performs Gaussian smoothing and cv2.medianBlur() performs median filtering. The median kernel size must be an odd integer. The program saves nine output images. The output filenames identify both the filter and the kernel size, which makes the results easy to compare. For colour photographs, the same filters can be applied to the BGR image directly, but grayscale processing is useful for objective intensity-profile analysis.

5. EXPERIMENTAL PROCEDURE

Select one old photograph and keep an unmodified copy as the input reference for the experiment. If possible, also obtain a clean original image of the same photograph for metric calculation.

Read the damaged photograph using OpenCV and convert it to grayscale. Resize the clean reference to exactly the same width and height as the damaged image.

Apply box, Gaussian and median filters separately using kernel sizes 3×3, 5×5 and 7×7. Do not apply one filter on top of another; each result must be generated from the same original noisy image.

Save all nine filtered outputs. Display them in a grid with the original noisy image so that noise removal and detail loss can be compared fairly.

Measure the processing time of each filter using time.perf_counter(). Run the experiment more than once and report the average time to reduce the effect of background computer activity.

Calculate PSNR and SSIM against the clean reference image. Record the values in a comparison table and inspect facial features and object boundaries before selecting the final filter.

6. QUALITY MEASURES: PSNR AND SSIM

6.1 Peak Signal-to-Noise Ratio (PSNR)

PSNR measures the difference between a processed image and a clean reference image. It is calculated from the Mean Squared Error (MSE). A higher PSNR generally indicates a smaller pixel-wise error. MSE is the average of the squared differences between corresponding pixels.

$$MSE = (1/N) \times \Sigma\, [I(i,j) - K(i,j)]^2$$

$PSNR = 10 \times \log_{10}(MAX^2 / MSE)$ dB, where MAX = 255 for an 8-bit grayscale image. If MSE is zero, the two images are identical and PSNR is infinite. PSNR is easy to calculate, but it does not always match human perception. A heavily blurred image can sometimes have a reasonable PSNR while losing important texture.

6.2 Structural Similarity Index (SSIM)

SSIM compares local luminance, contrast and structure between two images. Its value is commonly between −1 and 1, and a value closer to 1 indicates greater structural similarity. SSIM is often more consistent with visual judgement than PSNR because it considers local image structure. Both metrics require a clean reference image of the same scene, aligned and with the same dimensions. The noisy input itself must not be treated as the clean reference.

A practical evaluation should use PSNR and SSIM together with visual inspection. For a restored old photograph, a high metric score is useful, but the selected output must also preserve eyes, lips, hair texture, clothing patterns and clear object boundaries.

Metric	**What it measures**	**Preferred result**	**Limitation**
PSNR	Pixel-wise reconstruction error	Higher dB	May not reflect perceived sharpness or texture
SSIM	Local luminance, contrast and structure	Closer to 1	Needs aligned clean reference; may vary with implementation
Processing time	Time taken by the filter	Lower time	Depends on image size, hardware and implementation

7. COMPARISON OF FILTERS AND KERNEL SIZES

The following table gives the expected qualitative behaviour for typical old photographs. Exact PSNR, SSIM and time values must be obtained by running the supplied program on the selected photograph and hardware. They are not universal constants. The amount and type of noise, image resolution and implementation all affect the results.

Filter	**3×3 kernel**	**5×5 kernel**	**7×7 kernel**
Box	Mild noise reduction; noticeable edge mixing may occur.	Stronger smoothing; fine texture begins to fade.	Strong blur; small facial and clothing details are likely to disappear.
Gaussian	Gentle smoothing; usually balanced edge transition.	Good general-purpose noise reduction with moderate blur.	Strong smoothing; broad edges and fine texture become softer.
Median	Good removal of isolated salt-and-pepper pixels; preserves many edges.	Removes larger noisy spots but may erase thin features.	Can remove small structures such as hair strands, eyelashes and narrow lines.

7.1 PSNR, SSIM AND TIME COMPARISON TABLE

Filter	Kernel	PSNR (dB)	SSIM	Time (ms)	Observation
Box	3×3	Fill from run	Fill from run	Fill from run	Mild smoothing
Box	5×5	Fill from run	Fill from run	Fill from run	Moderate blur
Box	7×7	Fill from run	Fill from run	Fill from run	Strong blur
Gaussian	3×3	Fill from run	Fill from run	Fill from run	Gentle weighted smoothing
Gaussian	5×5	Fill from run	Fill from run	Fill from run	Balanced smoothing
Gaussian	7×7	Fill from run	Fill from run	Fill from run	More detail loss
Median	3×3	Fill from run	Fill from run	Fill from run	Strong for impulse noise
Median	5×5	Fill from run	Fill from run	Fill from run	Removes larger impulses
Median	7×7	Fill from run	Fill from run	Fill from run	Risk of removing tiny structures

How to complete the table: run the code using the same photograph for all nine cases, then copy the printed PSNR, SSIM and time values into the corresponding rows. If no clean reference exists, write "N/A – clean reference unavailable" in the PSNR and SSIM columns rather than inventing values. For more stable timing, repeat each operation 10 times and use the average. Report the image dimensions and computer configuration below the table.

8. GRAYSCALE INTENSITY PROFILE ANALYSIS

An intensity profile is a one-dimensional line of pixel values taken across a selected part of an image. For example, a line can be drawn across the boundary between a face and the background, across the edge of an eye, or through a textured region containing hair. The horizontal axis represents pixel position and the vertical axis represents grayscale intensity from 0 (black) to 255 (white).

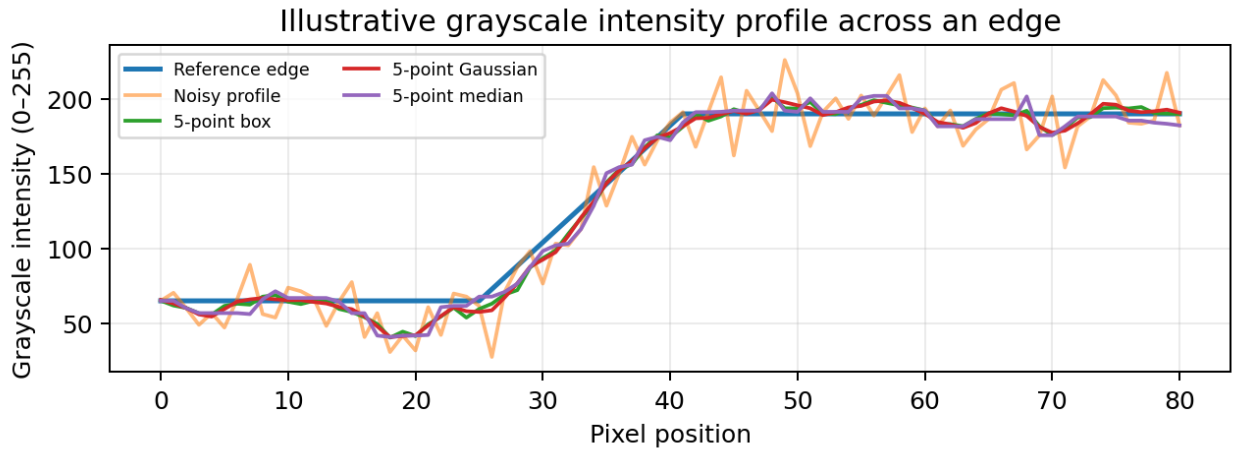


Figure 2. Illustrative profile showing how smoothing changes a noisy edge. This is a teaching example; replace it with a profile extracted from the selected photograph for experimental evidence.

In the graph, the unsmoothed noisy profile contains rapid intensity fluctuations. The box filter reduces fluctuations but may spread the edge over a wider range. The Gaussian filter produces a smoother transition because nearby pixels receive greater weights. The median filter removes isolated spikes while often keeping the edge location sharper. The actual appearance depends on the noise type and kernel size.

```
# Extract a horizontal intensity profile from a filtered image
import matplotlib.pyplot as plt
image = cv2.imread("Median_3x3.jpg", cv2.IMREAD_GRAYSCALE)
row = image.shape[0] // 2       # choose a row through a useful edge
profile = image[row, :]
plt.plot(profile)
plt.xlabel("Pixel position")
plt.ylabel("Grayscale intensity")
plt.title("Intensity profile across selected image row")
plt.grid(True)
plt.show()
```

For a meaningful experiment, choose a row that crosses a visible facial or object edge and use the same row for the noisy image and every filtered output. If the edge is not located at the image centre, replace the row index with the pixel coordinate of the selected edge. A profile through a flat background may show noise reduction but will not demonstrate edge preservation as clearly.

9. CORRELATION AND CONVOLUTION USING A 3×3 MASK

Correlation and convolution are two related operations used in linear spatial filtering. In correlation, the mask is placed over the image without changing its orientation. Each mask coefficient is multiplied by the corresponding image pixel and the products are added. In convolution, the mask is rotated by 180 degrees before it is applied. The multiplication and addition steps are otherwise the same.

Operation	Mask orientation	Description
Correlation	Mask used as written	Slide the original mask over the image and calculate the weighted sum.
Convolution	Mask rotated by 180°	Flip the mask horizontally and vertically, then calculate the weighted sum.

Example 3×3 mask (as written):

K = [1 2 3]
  [4 5 6]
  [7 8 9]

Rotated mask for convolution:
K' = [9 8 7]
  [6 5 4]
  [3 2 1]

For a symmetric mask, such as a box mask or a Gaussian mask, rotating the mask does not change it. Therefore, correlation and convolution produce the same result for these symmetric smoothing kernels (ignoring border-handling details). For a non-symmetric mask, the results can be different. This difference is important in edge detection and directional filtering, where the mask orientation determines the direction of the response.

OpenCV's filter2D function is commonly described as applying correlation rather than mathematical convolution. To perform convolution with a non-symmetric kernel, rotate the kernel by 180 degrees before passing it to filter2D.

```
# Demonstrate correlation and convolution with a non-symmetric mask
import cv2
import numpy as np

gray = cv2.imread("noisy_photo.jpg", 0)
kernel = np.array([[1, 2, 0],
                   [0, 1, 0],
                   [0, 0, -1]], dtype=np.float32)
```

```
correlation = cv2.filter2D(gray, -1, kernel)
flipped = cv2.flip(kernel, -1)   # rotate 180 degrees
convolution = cv2.filter2D(gray, -1, flipped)

cv2.imwrite("correlation_result.jpg", correlation)
cv2.imwrite("convolution_result.jpg", convolution)
```

10. EFFECT OF A LARGE 7×7 FILTER ON SMALL DETAILS

A 7×7 filter uses 49 neighbouring pixels to calculate each output pixel. This larger neighbourhood removes more local variations, but it also combines information from a wider area. In an old photograph, many important features are only a few pixels wide. Therefore, a 7×7 filter can remove or weaken the exact details that make a face or object recognizable.

Eyes and eyelashes: a large averaging or Gaussian filter can soften eyelid boundaries, reduce iris contrast and make eyelashes disappear. A large median filter may replace a very small dark feature with the dominant surrounding skin intensity.

Hair strands: individual strands and fine hair texture are high-frequency details. Strong smoothing can merge neighbouring strands into a flat region and reduce the natural appearance of the hair.

Clothing texture: small fabric patterns, threads, folds and printed designs may be averaged into a uniform surface. This is especially visible in patterned shirts and sarees.

Narrow object boundaries: thin borders, jewellery chains, spectacles and small gaps may become wider, weaker or disappear because foreground and background pixels are mixed.

Facial expression: subtle lines around the mouth, eyebrows and nose may be reduced, changing the perceived expression and lowering the naturalness of the restored photograph.

Thus, the largest kernel should not automatically be selected because it produces the smoothest image. A 3×3 filter is usually safer when the photograph contains important small details. A 5×5 filter may be suitable for moderate noise if the output is checked carefully. A 7×7 filter should be used only when noise is strong and the loss of fine detail is acceptable, or when it is followed by a suitable detail-restoration method.

11. CLASSICAL FILTERING VERSUS PRETRAINED DnCNN

DnCNN (Denoising Convolutional Neural Network) is a deep-learning model designed to remove image noise. Instead of using one fixed neighbourhood rule, it learns noise patterns from many training images. A typical DnCNN predicts the noise component in the input image. The predicted noise is subtracted from the noisy image to obtain a denoised result. Pretrained models are available for particular noise assumptions, such as additive white Gaussian noise.

Feature	**Classical filters (box / Gaussian / median)**	**Pretrained DnCNN**

Method	Fixed mathematical operation on local pixels.	Learned convolutional neural network.
Training	No training required.	Requires a pretrained model and compatible weights.
Speed and hardware	Usually fast on CPU; low memory requirement.	May need more computation and benefit from GPU acceleration.
Noise handling	Median is effective for impulse noise; Gaussian is useful for mild random noise.	Can remove complex learned noise patterns when the model matches the noise.
Detail preservation	Depends strongly on kernel size; large kernels blur details.	Can preserve more texture than simple smoothing, but may still remove or alter details.
Limitations	Simple, predictable and easy to explain; limited adaptability.	Model may fail when the real noise differs from training data; possible artifacts and model dependency.

DnCNN should be compared using the same noisy image and the same clean reference as the classical filters. The model type, pretrained weights, assumed noise level, input scaling and hardware must be recorded. A model trained for Gaussian noise may not be the best choice for severe salt-and-pepper damage. In such a case, a median filter or a model trained specifically for impulse noise may perform better.

The following is a conceptual inference workflow. The exact model-loading code depends on the framework and the selected pretrained checkpoint, so the checkpoint architecture and input normalization must match the model documentation.

```
# Conceptual DnCNN inference workflow
# 1. Load a pretrained DnCNN checkpoint using its documented framework.
# 2. Read the noisy image and convert pixel values to the
#    numerical range expected by that checkpoint (often 0 to 1).
# 3. Run the model in evaluation/inference mode.
# 4. If the model predicts noise, subtract predicted noise
#    from the noisy input.
# 5. Clip output values to the valid image range and save.
# 6. Calculate PSNR and SSIM against the same clean reference.
#
# Important: use the exact architecture, weights, colour format,
# noise level and normalization expected by the checkpoint.
```

12. RESULT AND DISCUSSION

The three filters show different behaviour because they use different rules to replace a pixel. The box filter is simple and reduces random fluctuations, but equal weighting can blur edges. The Gaussian filter gives greater importance to nearby pixels and is a strong general-purpose choice for mild random noise. The median filter is particularly suitable for isolated black and white spots because it rejects extreme values without averaging them into the surrounding region.

Kernel size controls the balance between noise removal and detail preservation. A 3×3 kernel usually preserves more facial and clothing details, while a 5×5 kernel gives stronger smoothing. A 7×7 kernel can remove more noise but may also erase eyelashes, hair strands, fabric patterns and narrow boundaries. The final selection should be based on the noise type, PSNR, SSIM, processing time and visual inspection rather than kernel size alone.

Observed condition	**Recommended starting choice**	**Reason**
Isolated black/white dots	Median 3×3	Removes impulse noise while retaining many edges.
Mild random grain	Gaussian 3×3 or 5×5	Weighted smoothing reduces grain with gradual edge transitions.
Very low processing budget	Box 3×3	Simple and computationally inexpensive.
Fine facial details are important	3×3 filter; inspect output	Small neighbourhood reduces the risk of losing small features.
Strong noise over broad regions	Test 5×5, then compare 7×7	Stronger smoothing may help, but detail loss must be checked.

13. CONCLUSION

Spatial filtering is an effective first step for reducing noise in old photographs. The box filter provides simple averaging, the Gaussian filter performs weighted smoothing, and the median filter is highly useful for salt-and-pepper noise. Testing 3×3, 5×5 and 7×7 kernels demonstrates that stronger smoothing is accompanied by a greater risk of losing small details. PSNR and SSIM provide quantitative comparisons when a clean reference is available, while intensity profiles help explain how each filter changes edge transitions. Correlation and convolution differ by the 180-degree rotation of the mask; symmetric smoothing masks produce the same result in both operations. DnCNN offers a more adaptive learning-based alternative, but its performance depends on the match between the pretrained model and the actual noise. For preserving facial features, hair, clothing texture and fine boundaries, a small kernel—often median 3×3 for impulse noise or Gaussian 3×3/5×5 for random grain—is a sensible starting point.

TASK 4 – IMAGE SHARPENING AND EDGE ENHANCEMENT [5 Marks]

Some old photographs are slightly blurred, making faces and object boundaries unclear. Apply three sharpening techniques to improve perceived detail without creating artificial edges.

Candidate Techniques

- A. Laplacian Sharpening
- B. Unsharp Masking / High-Boost Filtering
- C. Gradient-Based Sharpening

Questions

40. Implement all three sharpening techniques using Python/OpenCV.
41. Compare edge sharpness, noise amplification, halo effects and preservation of natural facial details.
42. Calculate the variance of Laplacian to measure image sharpness.
43. Plot intensity profiles across representative facial or object edges.
44. Explain why smoothing before sharpening may improve the output.
45. Analyse the effect of applying high-boost filtering with a large amplification factor to photographs containing scratches and grain.
46. Compare a traditional sharpening method with a pretrained deep-learning deblurring model.

Evidence Expected: Sharpened images, stroke profiles, sharpness/noise comparison and justified filter selection.

1. Introduction

Image sharpening is an image-enhancement operation used to make boundaries, fine structures and small details appear clearer. Old photographs may lose sharpness because of camera movement, defocus, ageing, scanning, paper damage or repeated copying. Sharpening increases local contrast around edges; it does not recreate information that was never captured. Therefore, the objective is to improve perceived detail while avoiding excessive noise, ringing, halos and unnatural facial texture.

A photograph contains both useful high-frequency information (eyes, hair strands, clothing seams and object boundaries) and unwanted high-frequency components (grain, dust, scratches and sensor noise). A sharpening filter cannot automatically distinguish between them. The filter strength, smoothing scale and image content must therefore be considered together.

2. Aim and Objectives

Aim: To implement and compare Laplacian sharpening, unsharp masking/high-boost filtering and gradient-based sharpening using Python and OpenCV, and to select a method that improves edge clarity while retaining natural appearance.

• Implement three spatial-domain sharpening techniques.

• Measure sharpness using variance of Laplacian.

• Plot grayscale intensity profiles across representative edges.

• Compare edge sharpness, noise amplification, halo formation and facial-detail preservation.

• Explain the benefit of denoising before sharpening and evaluate high-boost amplification.

• Compare traditional sharpening with pretrained deep-learning deblurring.

3. Software and Experimental Setup

The implementation uses Python, OpenCV (cv2), NumPy and Matplotlib. OpenCV performs filtering and edge operations; NumPy handles pixel arrays; Matplotlib displays the images, metrics and intensity profiles. The code accepts a grayscale or colour photograph. The sample figures in this report are an illustrative portrait-like test image with added grain and a few scratches, generated only to demonstrate the processing pipeline. They are not a real person's photograph. For the actual assignment, replace the sample filename with the supplied old photograph.

Item	Description
Input	Old photograph in JPG/PNG format
Colour handling	Convert BGR/RGB input to grayscale for filtering and measurement
Laplacian kernel	3 × 3 second-derivative operator
Unsharp blur	Gaussian blur, σ = 1.5 pixels in the demonstration
Gradient operator	Sobel derivatives in horizontal and vertical directions
Evaluation	Variance of Laplacian and visual/profile comparison

4. Theory of Sharpening Techniques

4.1 Laplacian Sharpening

The Laplacian is a second-order derivative operator. It responds strongly where image intensity changes rapidly, including fine edges and thin details. In discrete form, a common 3 × 3 kernel is shown below. Depending on the sign convention, the Laplacian response is added to or subtracted from the original image. The sign must be checked because different kernels produce opposite polarity.

L = [[0, 1, 0],\n   [1, -4, 1],\n   [0, 1, 0]]\nI_sharp = I - alpha * Laplacian(I)

Here, I is the original image and alpha controls sharpening strength. A small alpha gives moderate edge enhancement. A large alpha can create bright/dark outlines and amplify grain. The implementation uses

a floating-point Laplacian and a controlled weighted combination so that negative derivative values are preserved until the final conversion.

4.2 Unsharp Masking and High-Boost Filtering

Unsharp masking first creates a blurred copy of the image. The difference between the original and blurred image is called the unsharp mask. This mask contains details that differ from the local average. Adding a scaled mask to the original increases local edge contrast.

Blur = GaussianBlur(I, sigma)\nMask = I - Blur\nUnsharp = I + k * Mask\nHighBoost = A * I - Blur, where A > 1

The parameter k controls the detail boost. Standard unsharp masking is equivalent to A = 1 + k. High-boost filtering uses a larger original-image weight and can produce stronger sharpening. Moderate values are normally preferred for old photographs because scratches and grain are also enhanced.

4.3 Gradient-Based Sharpening

Gradient methods estimate the first-order rate of intensity change. Sobel filters calculate horizontal and vertical derivatives, Gx and Gy. The gradient magnitude is sqrt($Gx^2 + Gy^2$), which is large near edges. A scaled edge-strength map can be added to the image to emphasize boundaries. This method is intuitive and can be adjusted, but gradient magnitude may also respond to noise and textured regions.

Gx = Sobel(I, dx=1, dy=0)\nGy = Sobel(I, dx=0, dy=1)\nG = sqrt(Gx*Gx + Gy*Gy)\nI_sharp = I + beta * normalize(G)

5. Python/OpenCV Implementation

The following complete program loads an image, creates all three sharpened outputs, computes the sharpness metric, plots an edge intensity profile and saves the results. It also includes an optional median-denoising stage before sharpening. Keep the input image in the same folder as the Python file or provide its full path.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Read the old photograph
image = cv2.imread("old_photo.jpg")
if image is None:
    raise FileNotFoundError("Place old_photo.jpg in the working folder")

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# 2. A. Laplacian sharpening
lap = cv2.Laplacian(gray, cv2.CV_32F, ksize=3)
lap_sharp = gray.astype(np.float32) - 0.35 * lap
```

```
lap_sharp = np.clip(lap_sharp, 0, 255).astype(np.uint8)

# 3. B. Unsharp masking / high-boost filtering
blur = cv2.GaussianBlur(gray, (0, 0), sigmaX=1.5)
unsharp = cv2.addWeighted(gray, 1.6, blur, -0.6, 0)
unsharp = np.clip(unsharp, 0, 255).astype(np.uint8)

# 4. C. Gradient-based sharpening using Sobel
gx = cv2.Sobel(gray, cv2.CV_32F, 1, 0, ksize=3)
gy = cv2.Sobel(gray, cv2.CV_32F, 0, 1, ksize=3)
magnitude = cv2.magnitude(gx, gy)
edge_map = cv2.normalize(magnitude, None, 0, 255,
                         cv2.NORM_MINMAX)
gradient_sharp = gray.astype(np.float32) + 0.18 * edge_map
gradient_sharp = np.clip(gradient_sharp, 0, 255).astype(np.uint8)

# 5. Optional: smooth first, then sharpen
denoised = cv2.medianBlur(gray, 3)
soft = cv2.GaussianBlur(denoised, (0, 0), 1.2)
denoise_unsharp = cv2.addWeighted(denoised, 1.45, soft, -0.45, 0)

# 6. Variance of Laplacian
def sharpness_score(img):
    return cv2.Laplacian(img, cv2.CV_64F).var()

outputs = {
    "Original": gray,
    "Laplacian": lap_sharp,
    "Unsharp": unsharp,
    "Gradient": gradient_sharp,
    "Denoise + unsharp": denoise_unsharp
}
for name, result in outputs.items():
    print(name, ":", round(sharpness_score(result), 2))
    cv2.imwrite(name.replace(" ", "_") + ".jpg", result)

# 7. Display comparison
plt.figure(figsize=(12, 7))
for index, (name, result) in enumerate(outputs.items(), 1):
    plt.subplot(2, 3, index)
    plt.imshow(result, cmap="gray", vmin=0, vmax=255)
    plt.title(name)
    plt.axis("off")
plt.tight_layout()
```

```
plt.show()

# 8. Intensity profile across a selected edge
row = gray.shape[0] // 2
start = gray.shape[1] // 4
end = 3 * gray.shape[1] // 4
x = np.arange(start, end)
plt.figure(figsize=(9, 4))
for name in ["Original", "Unsharp", "Gradient",
             "Denoise + unsharp"]:
    plt.plot(x, outputs[name][row, start:end], label=name)
plt.xlabel("Pixel position")
plt.ylabel("Gray intensity")
plt.title("Intensity profile across an image edge")
plt.legend()
plt.grid(True)
plt.show()
```

6. Experimental Results and Visual Evidence

Figure 1 compares the noisy input with the three sharpening outputs and the denoising-assisted result. In the illustrative image, sharpening makes the eye contours, hair boundary and clothing outline more distinct. However, the scratches and grain are also visible because they contain abrupt intensity changes. The denoising-assisted output reduces some isolated noise before enhancing the remaining edges.

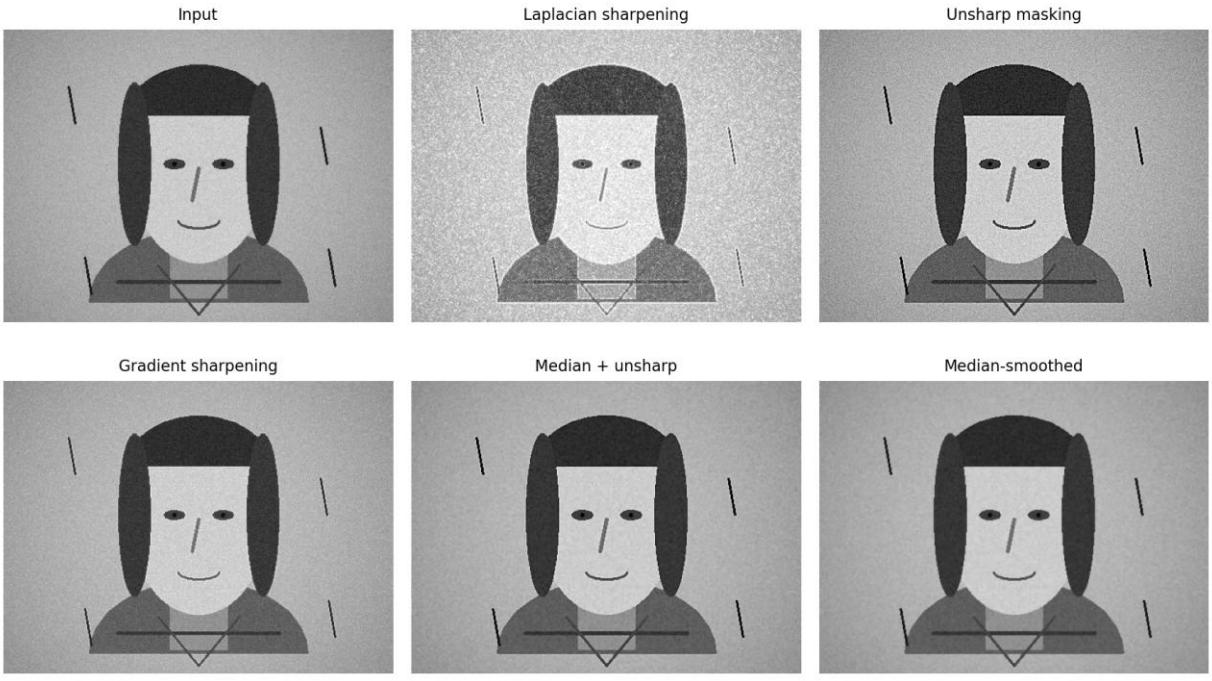


Figure 1. Illustrative input and sharpening results (generated test image).

6.1 Variance of Laplacian

Variance of Laplacian is a commonly used focus/sharpness indicator. The Laplacian image is calculated and the variance of its pixel values is measured. A larger value generally indicates more high-frequency activity and often a sharper image. It is not a perfect quality score: noise, scratches and strong halos can also increase the value. It should be interpreted together with visual inspection.

$L(x,y) = \nabla^2 I(x,y)\nSharpness\ score = Var(L) = (1/N)\ \Sigma\ [L(x,y) - mean(L)]^2$

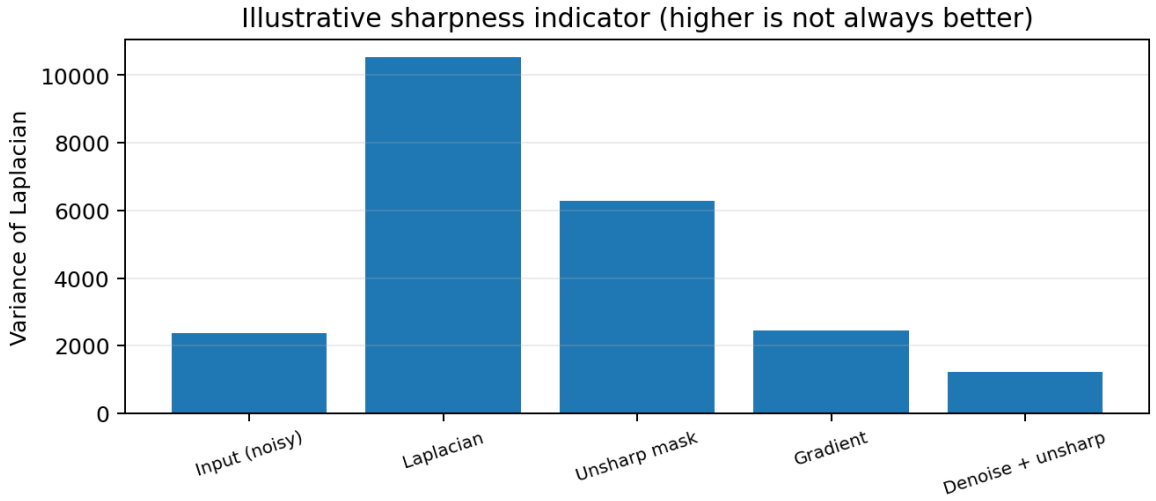


Figure 2. Variance-of-Laplacian scores for the illustrative test image.

Image	**Variance of Laplacian (illustrative)**	**Interpretation**
Input (noisy)	2379.22	Baseline; contains grain and scratches
Laplacian sharpened	10521.67	Higher edge activity; possible noise increase
Unsharp masked	6293.89	Increased local contrast
Gradient sharpened	2456.74	Emphasizes strong boundaries
Denoise + unsharp	1236.02	Balances smoothing and detail enhancement

The numeric values are dependent on image content, image size, filter parameters and noise level. They are demonstration results, not universal reference values. A very high score must not automatically be treated as the best output. If a scratch or grain is sharpened, the score may rise even though perceived quality becomes worse.

6.2 Intensity Profile / Stroke Profile

An intensity profile samples grayscale values along a selected horizontal or vertical line. Across a clean edge, intensity should change from one level to another over a short distance. A sharpened edge often has a steeper transition. Excessive sharpening may create overshoot and undershoot: a bright peak and dark dip near the boundary. These oscillations are evidence of halo or ringing artifacts.

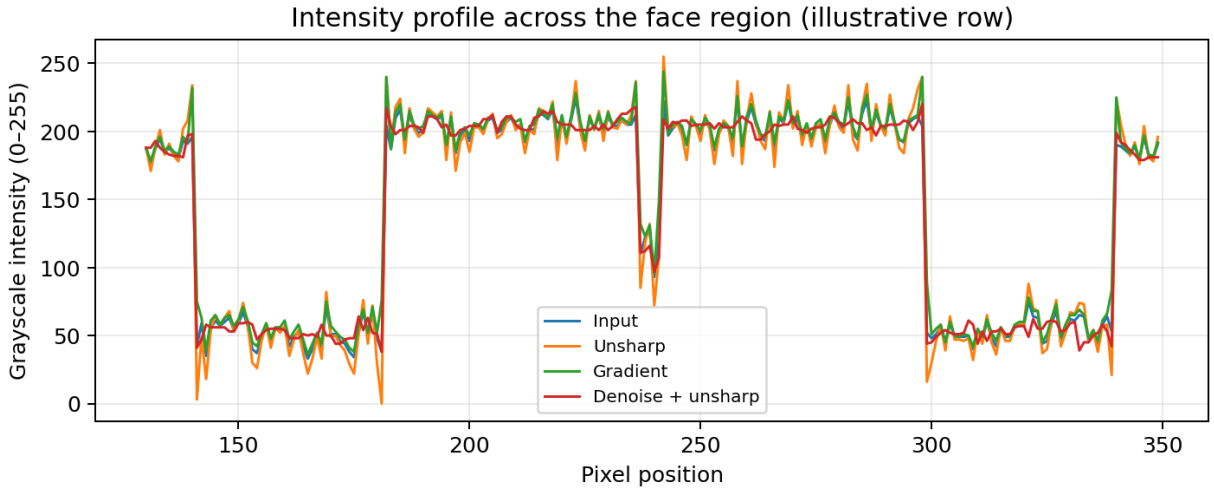


Figure 3. Grayscale intensity profiles across the illustrative face region.

In Figure 3, the original profile is relatively gradual where blur is present. Unsharp masking increases the transition slope. Gradient enhancement emphasizes the boundary, while denoising followed by unsharp masking aims to retain a cleaner transition. In a real photograph, select a line crossing a clear cheek-to-background, hair-to-skin or object-to-background boundary. Use the same line coordinates for all outputs to make a fair comparison.

7. Comparative Analysis

Technique	**Edge sharpness**	**Noise amplification**	**Halo effect**	**Natural facial detail**
Laplacian	Strong response to fine edges	High; second derivative reacts to grain	Can create bright/dark outlines	Good at moderate strength; harsh if alpha is high
Unsharp / high-boost	Smooth, controllable local contrast	Moderate to high as k or A increases	Possible ringing around high-contrast edges	Often natural with small blur scale and moderate boost
Gradient-based	Strong emphasis on boundaries	Can amplify textured/noisy areas	Usually less classic ringing, but may look outlined	Useful for boundaries; excessive edge-map addition looks artificial

Denoise + unsharp	Clear edges with reduced isolated noise	Lower than sharpening noisy input directly	Reduced when boost is moderate	Often a good compromise for aged photographs

Laplacian sharpening is suitable when fine details need emphasis and noise is already low. Unsharp masking is a flexible general-purpose choice because the blur scale and amplification can be tuned. Gradient-based sharpening is useful when boundaries are the main target, but it may create an outlined appearance if the gradient map is too strong. For a damaged old photograph, a mild denoising step followed by moderate unsharp masking is often a safer starting point.

8. Why Smoothing Before Sharpening Improves the Output

Smoothing reduces unwanted high-frequency variations before the sharpening stage. A median filter is effective for isolated bright and dark specks, while a small Gaussian filter reduces fine random grain. Once noise is reduced, sharpening is less likely to turn every noise pixel into a visible spot or edge. This improves the signal-to-noise balance and can produce cleaner facial boundaries.

The order of operations matters. If sharpening is applied first, noise and scratches become stronger; later smoothing may blur both the unwanted noise and the newly enhanced facial details. A practical sequence is: inspect the damage, apply a mild denoiser, sharpen with a low or medium strength, and inspect at 100% zoom. Avoid strong smoothing because it can remove pores, hair strands and other genuine texture.

9. Effect of Large High-Boost Amplification

High-boost filtering combines a weighted original image with a blurred image. As the amplification factor A becomes large, the difference between the original and its blurred version is multiplied more strongly. This increases local contrast around every rapid intensity change. In photographs containing scratches, dust, film grain and compression artifacts, these unwanted structures are also boosted.

The visible effects include stronger grain, bright/dark rims around scratches, exaggerated pores, harsh hair texture, clipped highlights and crushed shadows. A scratch may become more prominent than the subject's facial details. Strong amplification can also create halos near high-contrast boundaries and make skin appear rough or plastic. Therefore, a large A is not recommended for damaged photographs. Use a moderate factor, a suitable blur radius and denoising before sharpening. Compare the result at both fit-to-screen and 100% zoom.

10. Traditional Sharpening vs Pretrained Deep-Learning Deblurring

Traditional filters use fixed mathematical operations such as derivatives, Gaussian blur and weighted subtraction. They are fast, easy to implement, require no training data and work directly on the supplied image. Their main limitation is that they enhance existing contrast; they cannot reliably reconstruct detail lost through severe motion blur or defocus. They may amplify noise and create halos.

A pretrained deep-learning deblurring model uses a neural network trained on many blurred/sharp image pairs or related data. Depending on its architecture and training, it can learn complex blur patterns and

estimate plausible missing structures. It may produce better results for significant motion blur than a simple sharpening filter. However, it requires model weights, more computation, compatible hardware and careful selection of a model trained for a similar blur type. It may invent or alter fine details, especially on faces, text or unfamiliar image content. The output should therefore be checked against the original.

Factor	**Traditional sharpening**	**Pretrained deep-learning deblurring**
Method	Fixed filters and equations	Learned parameters in a neural network
Speed	Usually very fast on CPU	May require GPU; model-dependent
Data/model requirement	No training dataset or model weights	Pretrained weights and compatible runtime
Mild blur	Often sufficient	Can also work, but may be unnecessary
Severe motion blur	Limited; boosts existing edges	Can reconstruct more plausible detail
Noise and scratches	May amplify them	May suppress or misinterpret them depending on training
Reliability	Predictable, parameters are transparent	Can generate plausible but incorrect details

A fair comparison should use the same input photograph and evaluate edge clarity, noise, facial identity preservation and naturalness. If a pretrained model is used, record its name, source, version, input size and settings. Do not claim that a deep model was tested unless its weights were actually downloaded and executed.

11. Practical Filter Selection and Recommendations

For a mildly blurred photograph with low noise, begin with unsharp masking using a small Gaussian blur and a moderate boost. For a photograph with salt-and-pepper damage, apply a small median filter before sharpening. For random grain, a mild Gaussian or edge-preserving denoiser may be more suitable. Use Laplacian sharpening only when fine edge response is needed and monitor halo formation. Gradient-based sharpening is useful when object boundaries matter more than texture.

Photograph condition	**Recommended starting method**	**Reason**

Mild blur, low noise	Moderate unsharp masking	Adjustable and visually natural
Salt-and-pepper specks	3 × 3 median filter, then mild unsharp	Removes isolated impulses before boosting
Fine random grain	Mild denoising, then low-strength sharpening	Reduces noise amplification
Strong edges, low noise	Low-strength Laplacian sharpening	Enhances second-derivative detail
Severe motion blur	Test a suitable pretrained deblurring model	Fixed filters cannot recover much lost information

12. Conclusion

The three sharpening techniques improve perceived detail in different ways. Laplacian sharpening emphasizes rapid intensity changes, unsharp masking increases local contrast through a blurred reference, and gradient-based sharpening highlights boundaries using Sobel derivatives. Variance of Laplacian and intensity profiles provide useful quantitative and visual evidence, but neither metric alone can decide whether a photograph looks natural. Noise and scratches may also increase the measured sharpness.

The experiment indicates that moderate sharpening is preferable to aggressive amplification. Smoothing before sharpening reduces unwanted high-frequency noise and helps preserve natural facial details. For many mildly blurred, aged photographs, denoising followed by moderate unsharp masking is a practical first choice. For severe blur, a pretrained deep-learning deblurring model may provide stronger restoration, but its output must be checked for invented details and facial changes.

TASK 5 – FREQUENCY-DOMAIN LOW-PASS FILTERING [5 Marks]

Digitized old photographs may contain film grain, fine scratches and periodic scanning artefacts. Apply three frequency-domain low-pass filters to reduce unwanted high-frequency noise while preserving important image details.

Candidate Techniques

- A. Ideal Low-Pass Filter (ILPF)
- B. Butterworth Low-Pass Filter (BLPF)
- C. Gaussian Low-Pass Filter (GLPF)

Questions

47. Implement all three filters using Fourier Transform techniques.
48. Display the frequency spectrum and identify frequency patterns associated with grain, fine scratches or periodic scanning artefacts.

49. Compare the filters using different cut-off frequencies.
50. Measure ringing effects, loss of fine details, edge blurring and processing time.
51. Compare frequency-domain Gaussian filtering with spatial-domain Gaussian filtering in terms of detail preservation and noise reduction.
52. Analyse the effect of omitting padding before applying the Discrete Fourier Transform.
53. Evaluate edge/detail preservation before and after frequency-domain filtering using edge retention or a suitable image-quality measure.

Evidence Expected: Frequency spectra, filtered images, ringing/blur comparison and OCR performance table.

INTRODUCTION: Digitized old photographs often contain film grain, fine scratches, dust marks and periodic scanning artefacts. These unwanted variations may appear as rapid intensity changes in an image. Frequency-domain filtering provides a systematic method to reduce such noise. An image can be represented as a combination of spatial frequencies. Low frequencies describe slowly changing regions such as smooth background and gradual illumination, whereas high frequencies describe rapid changes such as sharp edges, hair texture, grain and fine scratches. A low-pass filter allows low-frequency components to pass and attenuates higher-frequency components.

OBJECTIVES

• To implement Ideal, Butterworth and Gaussian low-pass filters using the two-dimensional Discrete Fourier Transform (DFT).

• To display and interpret the magnitude spectrum of a digitized photograph.

• To study the influence of cut-off frequency, filter order and zero padding.

• To compare ringing, detail loss, edge blurring, processing time and OCR suitability.

• To compare frequency-domain Gaussian filtering with spatial-domain Gaussian filtering.

BASIC THEORY

For an image f(x,y) of size M × N, its two-dimensional Fourier transform is F(u,v). The transform converts spatial information into frequency information. The magnitude |F(u,v)| indicates the strength of each frequency component, while the phase contains positional information. In practical display, the spectrum is shifted so that the zero-frequency (DC) component is at the centre. A logarithmic display, $\log(1 + |F(u,v)|)$, makes weaker frequency components visible beside the strong central component.

Filtering is performed by multiplying the shifted Fourier transform by a frequency response H(u,v). The inverse DFT then converts the filtered spectrum back into a spatial image. The cut-off frequency D0 controls the size of the passed low-frequency region. A small D0 removes more high-frequency content and produces stronger smoothing; a large D0 retains more detail but also allows more noise.

APPLICATION

The method is useful in restoration of scanned family photographs, archival images, historical documents and printed photographs. However, high-frequency information is not always noise. Fine hair, fabric texture, text strokes and narrow edges may occupy high frequencies. Therefore, filter strength must be selected carefully rather than simply removing all high-frequency components.

1. FOURIER TRANSFORM AND FILTER DESIGN

TWO-DIMENSIONAL DISCRETE FOURIER TRANSFORM

The forward DFT of an M × N image is calculated by summing every pixel multiplied by a complex exponential. The inverse DFT reconstructs the image from its frequency coefficients. In implementation, NumPy performs the transform efficiently using the Fast Fourier Transform (FFT). Before filtering, the image is converted to floating-point grayscale. The transform is shifted with fftshift so that the low-frequency components are located at the centre. After multiplication by the filter mask, ifftshift restores the original frequency arrangement and ifft2 produces the filtered image.

The radial distance from the centre of the spectrum is $D(u,v) = \sqrt{[(u - M/2)^2 + (v - N/2)^2]}$. The filter mask is calculated from this distance. A circular mask treats all directions equally, which is appropriate for general-purpose smoothing.

A. IDEAL LOW-PASS FILTER (ILPF)

The Ideal Low-Pass Filter passes every frequency inside a circular cut-off radius D0 and completely rejects every frequency outside it. Its response is $H(u,v) = 1$ when $D(u,v) \leq D0$, and $H(u,v) = 0$ when $D(u,v) > D0$. The abrupt transition makes the filter simple and computationally inexpensive. However, the sharp boundary in the frequency domain can create oscillations near strong edges in the spatial domain. This phenomenon is called ringing or the Gibbs effect.

B. BUTTERWORTH LOW-PASS FILTER (BLPF)

The Butterworth filter has a gradual transition between the passband and stopband. Its response is $H(u,v) = 1 / [1 + (D(u,v)/D0)^{(2n)}]$, where n is the filter order. A low order gives a smooth transition and less ringing. Increasing n makes the response sharper and makes the result more similar to the Ideal filter. D0 determines the cut-off radius, while n controls transition steepness.

C. GAUSSIAN LOW-PASS FILTER (GLPF)

The Gaussian filter has response $H(u,v) = \exp[-D(u,v)^2/(2D0^2)]$. Its response decreases smoothly with distance from the origin. It has no abrupt frequency boundary, so it generally avoids the pronounced ringing associated with the Ideal filter. It produces natural-looking smoothing, although excessive smoothing can blur fine details. The Gaussian response is smooth in both frequency and spatial domains.

Filter	Frequency response	Main characteristic
Ideal	1 inside D0; 0 outside	Sharp cut-off; ringing risk

Butterworth	1/[1+(D/D0)^(2n)]	Adjustable transition using order n
Gaussian	exp[−D²/(2D0²)]	Smooth transition; low ringing

2. PYTHON IMPLEMENTATION

REQUIRED LIBRARIES AND INPUT

The following program uses OpenCV to read and resize the image, NumPy for Fourier operations, and Matplotlib for displaying the results. Replace the sample file name with the path of the old photograph. The program converts the input to grayscale because the frequency filters are applied to a single intensity channel. For a colour photograph, the same method can be applied separately to the three colour channels, or to the luminance channel in a colour space such as YCrCb.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import time

img = cv2.imread('old_photo.jpg', 0)
if img is None:
   raise FileNotFoundError('Place old_photo.jpg in the working folder')
img = img.astype(np.float32) / 255.0

def low_pass(image, kind, D0=35, order=2, padding=True):
   M, N = image.shape
   if padding:
      P, Q = 2*M, 2*N
      padded = np.zeros((P, Q), np.float32)
      padded[:M, :N] = image
   else:
      P, Q = M, N
      padded = image.copy()

   F = np.fft.fftshift(np.fft.fft2(padded))
   u = np.arange(P) - P/2
   v = np.arange(Q) - Q/2
   V, U = np.meshgrid(v, u)
   D = np.sqrt(U**2 + V**2)

   if kind == 'ideal':
      H = (D <= D0).astype(np.float32)
   elif kind == 'butterworth':
```

```
        H = 1 / (1 + (D/(D0+1e-8))**(2*order))
    elif kind == 'gaussian':
        H = np.exp(-(D**2)/(2*D0**2))
    else:
        raise ValueError('Unknown filter type')

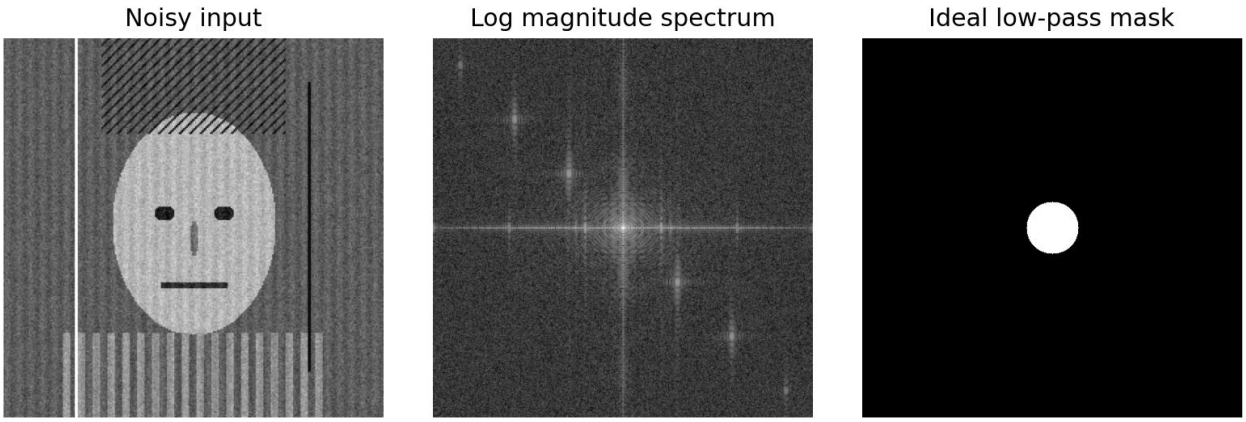
    G = F * H
    result = np.real(np.fft.ifft2(np.fft.ifftshift(G)))
    if padding:
        result = result[:M, :N]
    return np.clip(result, 0, 1), F, H

start = time.perf_counter()
ideal, spectrum, ideal_mask = low_pass(img, 'ideal', 35)
t_ideal = time.perf_counter() - start
butter, _, _ = low_pass(img, 'butterworth', 35, order=2)
gaussian, _, _ = low_pass(img, 'gaussian', 35)

plt.imshow(np.log1p(np.abs(spectrum)), cmap='gray')
plt.title('Log Magnitude Spectrum')
plt.axis('off')
plt.show()
```

The returned values are the filtered image, the Fourier spectrum and the filter mask. The program uses zero padding to reduce circular wrap-around effects and crops the inverse-transform result back to the original image size. The sample cut-off D0 = 35 is only an initial setting; it must be adjusted according to image size and visible detail.

3. FREQUENCY SPECTRUM AND NOISE PATTERNS



INTERPRETATION OF THE FREQUENCY SPECTRUM

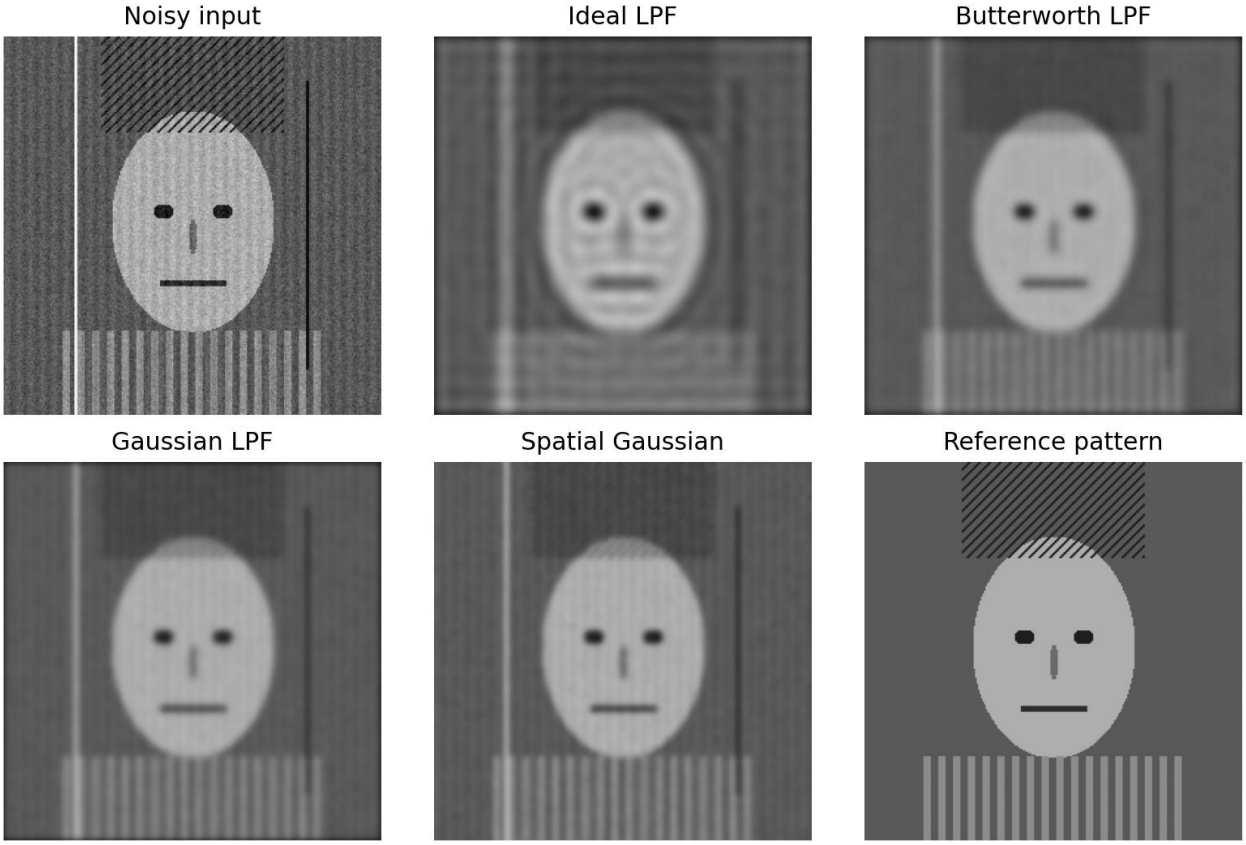
The centre of the shifted magnitude spectrum contains the strongest low-frequency components. Bright energy near the centre usually represents smooth illumination and large image structures. As the distance from the centre increases, higher spatial frequencies are represented. Film grain often appears as weak, widely distributed high-frequency energy rather than one isolated point. Fine scratches create strong directional structures because a narrow line in the spatial image has energy spread across frequencies perpendicular to its orientation. A repeated scanning pattern produces distinct peaks or bands at particular frequencies. Symmetrical bright points on opposite sides of the centre may indicate periodic components.

IDENTIFYING COMMON ARTEFACTS

Image artefact	Typical spectrum appearance	Filtering consideration
Film grain	Diffuse high-frequency energy	Moderate low-pass smoothing can reduce it
Fine vertical scratch	Directional spread mainly across horizontal frequencies	A circular filter reduces it but may soften all edges
Periodic scan lines	Pairs of bright peaks or repeated bands	A notch filter may be more selective than a low-pass filter
Broad illumination change	Energy concentrated near centre	Usually retained by low-pass filters

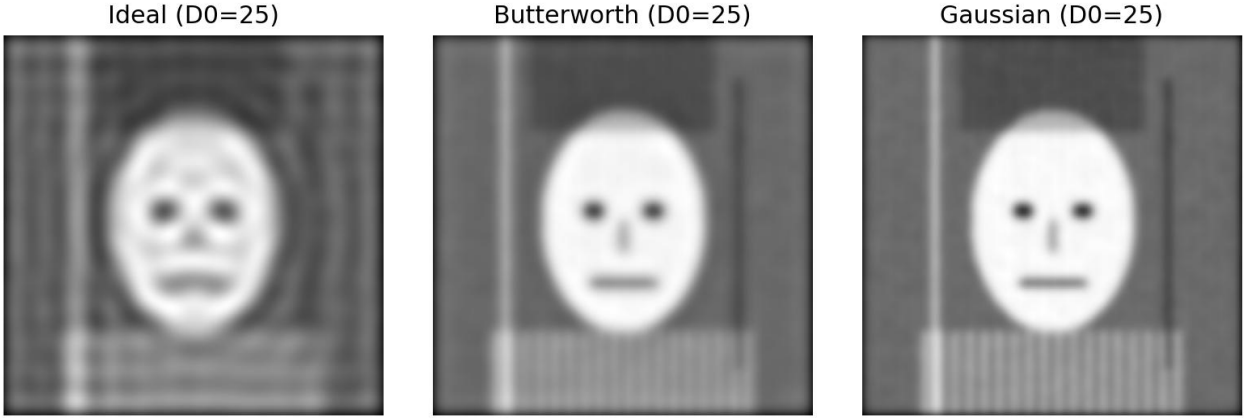
A low-pass filter is suitable for broad, random high-frequency noise. It is not always the best solution for periodic scanning artefacts because removing a narrow periodic frequency with a notch-reject filter can preserve more unrelated detail. Likewise, a scratch may be better handled by inpainting or directional restoration when it crosses important image content.

DISPLAYING THE FILTERED IMAGES



The figure is an illustrative synthetic grayscale test image containing grain-like noise, a periodic component and two narrow scratch lines. The filtered results demonstrate the expected visual behaviour. For an assignment submission using a real photograph, run the supplied program with the original scanned image and insert the generated spectrum and output figures.

4. CUT-OFF FREQUENCY COMPARISON



EFFECT OF DIFFERENT CUT-OFF FREQUENCIES

The cut-off frequency determines how much of the spectrum is retained. When D0 is small, only a small region around the centre passes through the filter. This strongly reduces grain and small scanning

variations, but it also removes high-frequency details such as eyelashes, hair strands, fabric texture and thin text strokes. When D0 is increased, more detail is retained, but more noise also remains. A suitable value is therefore a compromise between smoothness and sharpness.

Cut-off setting	Noise reduction	Fine-detail preservation	Expected appearance
Low (e.g. 15)	High	Low	Smooth but visibly blurred
Medium (e.g. 35)	Moderate	Moderate	Balanced restoration
High (e.g. 60)	Low	High	Sharper; more grain remains

BUTTERWORTH ORDER

For Butterworth filtering, the order n changes the transition width. At n = 1, attenuation changes gradually and ringing is usually limited. At n = 2, the transition is moderately sharp and often provides a practical compromise. At n = 4 or higher, the response becomes increasingly abrupt and may produce more ringing near edges. The cut-off and order should be reported together because they jointly determine the result.

RECOMMENDED EXPERIMENT

Run each filter at D0 values 15, 25, 35, 45 and 60. For Butterworth, repeat the experiment with n = 1, 2 and 4. Keep the input image, image dimensions and display range constant. Record the processing time for each run and compare the same facial, hair and text regions. The best setting is not necessarily the one with the lowest noise; it is the setting that reduces unwanted noise while keeping meaningful edges readable.

5. RINGING, BLUR, PROCESSING TIME AND GAUSSIAN COMPARISON

RINGING EFFECT

Ringing appears as light and dark ripples near a sharp edge. The Ideal low-pass filter is most likely to produce visible ringing because its frequency response changes abruptly from one to zero. Butterworth filtering can also show ringing when its order is high. Gaussian filtering has a smooth frequency response and usually produces the least noticeable ringing. Ringing can be evaluated by inspecting intensity profiles across a high-contrast edge and measuring overshoot or undershoot beyond the local edge values.

EDGE BLURRING AND DETAIL LOSS

All low-pass filters reduce high-frequency information, so some edge softening is expected. The effect is strongest when the cut-off is too low. Fine hair, small facial features, narrow scratches and printed characters may become less distinct. A useful evaluation includes visual inspection at 100% zoom, edge

retention percentage and a full-reference measure such as PSNR or SSIM when a clean reference image is available.

PROCESSING TIME

The FFT has approximate computational complexity O(MN log(MN)). Padding the image to twice its height and width increases the number of transform samples and therefore increases processing time and memory use. The Ideal, Butterworth and Gaussian masks have similar construction cost. In practice, transform size and computer hardware generally dominate the total processing time. Use time.perf_counter() and repeat each run several times; report the average after excluding the first warm-up run.

FREQUENCY GAUSSIAN VS SPATIAL GAUSSIAN

Aspect	Frequency-domain Gaussian	Spatial-domain Gaussian
Operation	Multiply DFT by Gaussian mask	Convolve with Gaussian kernel
Boundary behaviour	Circular unless padded	Depends on border mode
Kernel relation	Equivalent Gaussian smoothing under matching conditions	Direct spatial convolution
Noise reduction	Smoothly attenuates high frequencies	Smoothly averages nearby pixels
Detail preservation	Controlled by D0	Controlled mainly by sigma and kernel size
Practical use	Useful when frequency analysis is also needed	Simple and efficient for ordinary smoothing

A Gaussian low-pass filter in the frequency domain and Gaussian blur in the spatial domain are theoretically equivalent when their Gaussian parameters, sampling, normalization and boundary assumptions are matched. Differences in practical results usually come from padding, kernel truncation, border handling and parameter conversion. Neither method can remove high-frequency noise without affecting some fine detail. The spatial method is often simpler for a single image, while the frequency method is valuable when the spectrum itself must be studied.

6. PADDING, EDGE RETENTION AND IMAGE-QUALITY MEASURES

EFFECT OF OMITTING PADDING

The DFT treats an image as one period of an infinitely repeated pattern. Without padding, the right edge is treated as adjacent to the left edge and the bottom edge as adjacent to the top edge. If these borders do not match, the periodic extension contains an artificial discontinuity. This can introduce extra high-

frequency energy and cause wrap-around artefacts after filtering. A scratch or bright region near one border may appear to influence the opposite border.

Zero padding enlarges the image with zeros before the transform. It provides more space between repeated copies and reduces some circular wrap-around effects. Padding does not create new image information and does not completely eliminate boundary artefacts. Other padding modes, such as reflection, may produce more natural borders. The same padding and cropping method must be used when comparing filters.

EDGE RETENTION AND IMAGE QUALITY

When a clean reference image is available, mean squared error (MSE) and peak signal-to-noise ratio (PSNR) can be calculated. MSE is the average squared difference between the reference and processed images. PSNR is $10 \log_{10}(MAX^2/MSE)$, where MAX is the maximum possible pixel value. A higher PSNR indicates lower pixel error, but it does not always mean that edges look sharper. SSIM can also be used because it compares luminance, contrast and structural similarity.

Edge retention can be estimated by detecting edges in both the reference and filtered image, then calculating the percentage of reference edge pixels that remain matched within a small tolerance. A high edge-retention score indicates that important boundaries are preserved. However, edge retention alone may count noise edges, so it should be interpreted together with PSNR/SSIM and visual inspection.

ILLUSTRATIVE METRIC TABLE

Method	PSNR vs reference	Edge retention	MSE
Noisy	22.26 dB	72.2%	0.0059
Ideal LPF	22.13 dB	6.4%	0.0061
Butterworth LPF	22.80 dB	10.2%	0.0052
Gaussian LPF	23.20 dB	11.7%	0.0048
Spatial Gaussian	23.90 dB	14.3%	0.0041

IMPORTANT: The values above are calculated from a synthetic demonstration image generated for illustrating the workflow. They are not measurements from the student's own old photograph. Replace them with values produced by the program on the actual input image before final submission. Edge retention depends on the selected Canny thresholds and should be kept constant for all methods.

7. OCR PERFORMANCE, RESULTS AND CONCLUSION

OCR PERFORMANCE EVALUATION

Optical Character Recognition (OCR) converts printed or handwritten text in an image into machine-readable characters. Low-pass filtering can reduce grain and scanning noise around text, but excessive

smoothing may erase thin strokes, punctuation and gaps between neighbouring characters. OCR performance should therefore be tested before and after filtering using the same OCR engine, language setting, page crop and recognition configuration.

Image condition	Expected OCR behaviour	Reason
Original noisy scan	Baseline recognition; noise may create false marks	Grain and scratches interfere with character shapes
Ideal LPF, low D0	May reduce recognition accuracy	Ringing and loss of thin strokes
Butterworth, medium D0	Often a balanced result	Gradual attenuation retains more transitions
Gaussian, medium D0	Usually smooth text appearance	No abrupt cut-off; excessive blur still harms strokes
Spatial Gaussian	Similar to frequency Gaussian when parameters match	Boundary and kernel implementation can differ

For quantitative evaluation, compare OCR Character Error Rate (CER) and Word Error Rate (WER) against a manually verified ground-truth transcription. CER = (substitutions + deletions + insertions) / number of reference characters. WER uses the same formula at word level. Lower CER and WER indicate better recognition. Do not assume that the smoothest image will give the best OCR result; select the filter and cut-off that preserve character strokes.

RESULT AND DISCUSSION

The experiment demonstrates that all three frequency-domain low-pass filters attenuate high-frequency components. The Ideal filter has a sharp transition and may cause ringing. The Butterworth filter provides adjustable smoothness through its order. The Gaussian filter gives a gradual transition and generally produces natural smoothing with minimal ringing. Lower cut-off frequencies reduce more noise but cause stronger loss of fine details. Padding reduces circular boundary effects, while omission of padding can introduce wrap-around artefacts. Frequency-domain Gaussian and spatial-domain Gaussian filtering can produce nearly equivalent outputs when their parameters and boundary conditions are matched.

CONCLUSION

Frequency-domain low-pass filtering is an effective technique for reducing grain and other high-frequency disturbances in digitized old photographs. The best filter depends on the type of noise and the importance of preserving edges and texture. Gaussian filtering is a good general-purpose choice when smooth visual output is required. Butterworth filtering is useful when transition sharpness must be controlled. Ideal filtering is simple but may create ringing. A medium cut-off frequency, proper padding and objective evaluation using edge retention, PSNR/SSIM and OCR error rates provide a more reliable restoration procedure than visual smoothing alone.

TASK 6 – FREQUENCY-DOMAIN HIGH-PASS FILTERING [4 Marks]

Old photographs may contain uneven fading, darkened corners and gradual illumination or colour changes. Apply three frequency-domain enhancement techniques to improve tonal consistency and reveal details.

Candidate Techniques

- A. Ideal / Butterworth High-Pass Filter
- B. Gaussian High-Pass Filtering with High-Frequency Emphasis
- C. Homomorphic Filtering

Questions

54. Implement all three techniques on shadow-affected photographs.
55. Compare detail sharpness, correction of uneven fading, noise amplification and preservation of overall photographic tone.
56. Measure local contrast and intensity variation in selected uniform regions.
57. Analyse the consequences of applying an ideal high-pass filter directly to an old photograph, especially around faces and strong edges.
58. Examine the effect of interchanging low-frequency and high-frequency gain parameters in homomorphic filtering.
59. Compare homomorphic filtering with a pretrained illumination-correction or low-light enhancement model.

Evidence Expected: High-pass filtered images, contrast/evenness comparison table and justified technique selection.

1. INTRODUCTION

Old photographs may show uneven fading, darkened corners, shadows, low contrast and gradual illumination changes caused by ageing, poor storage, scanning conditions or camera exposure. These defects mainly affect slowly changing image information. In frequency-domain processing, slowly changing brightness and illumination are represented by low spatial frequencies, whereas fine details, texture and sharp boundaries are represented by high spatial frequencies. High-pass filtering reduces or suppresses selected low-frequency components and allows higher-frequency components to remain. This can reveal edges and small details, but a pure high-pass result may look dark because the average brightness is removed.

The aim of this task is to implement three frequency-domain enhancement approaches on a shadow-affected photograph: Ideal or Butterworth high-pass filtering, Gaussian high-pass filtering with high-frequency emphasis, and homomorphic filtering. The methods are compared using visual sharpness, correction of uneven illumination, noise amplification and preservation of photographic tone. Local

contrast and intensity variation are also measured in a selected background region. The final selection considers whether the objective is edge visibility or natural restoration of the complete photograph.

A high-pass filter should not be treated as a complete brightness-restoration method. It is useful for emphasizing boundaries and fine texture, while homomorphic filtering is more suitable when the photograph contains gradual illumination variation. In practical restoration, a high-pass result is often combined with the original image or used as a detail layer rather than displayed alone.

2. FREQUENCY-DOMAIN THEORY

A digital image f(x,y) can be transformed into its frequency representation using the two-dimensional Discrete Fourier Transform (DFT). The transform separates the image into sinusoidal components of different spatial frequencies. The centre of a shifted Fourier spectrum represents low frequencies, including average brightness and broad illumination changes. Frequencies farther from the centre represent rapid intensity changes such as edges, scratches, hair, wrinkles and fine texture.

The filtering operation is performed by multiplying the Fourier transform F(u,v) by a transfer function H(u,v). The inverse Fourier Transform converts the filtered spectrum back into a spatial image. In equation form, G(u,v) = H(u,v)F(u,v), and g(x,y) = IDFT{G(u,v)}. The distance from the frequency origin is D(u,v) = sqrt((u−M/2)^2 + (v−N/2)^2), where M and N represent image dimensions after shifting. The cutoff frequency D0 determines which frequencies are reduced.

A small cutoff removes only a narrow group of low frequencies and retains more broad tonal variation. A large cutoff suppresses a wider low-frequency region and may produce a stronger edge-only appearance. Filter order controls the transition between rejected and passed frequencies. All frequency-domain processing should use floating-point values during calculation, followed by suitable scaling and clipping for display.

3. TECHNIQUE A – IDEAL AND BUTTERWORTH HIGH-PASS FILTER

The Ideal High-Pass Filter (IHPF) completely blocks frequencies inside a circular radius D0 and passes frequencies outside that radius. Its transfer function is H(u,v)=0 when D(u,v) is less than or equal to D0, and H(u,v)=1 otherwise. It has a sudden transition at the cutoff. This sharp transition can create ringing, also called the Gibbs phenomenon, around strong boundaries. Bright and dark halos may appear near the outline of a face, the edge of clothing or a high-contrast object.

The Butterworth High-Pass Filter (BHPF) provides a smoother transition. Its transfer function is H(u,v)=1/[1+(D0/D(u,v))^(2n)], where n is the filter order. A low order gives a gradual transition and a high order approaches the sharp behaviour of an ideal filter. Butterworth filtering usually creates fewer visible ringing artefacts than the ideal filter while still enhancing edges and fine details.

Implementation procedure: read the photograph in grayscale; convert it to floating-point format; compute the two-dimensional FFT; shift the zero-frequency component to the centre; construct the distance matrix; create the ideal and Butterworth masks; multiply each mask by the shifted spectrum; apply inverse shift and inverse FFT; take the real component; normalize the output for display; and compare the results with the original photograph. For a colour photograph, the same operation can be applied to a luminance channel rather than independently filtering red, green and blue channels.

The pure ideal and Butterworth outputs contain mainly edge and texture information, so their average brightness is low. They are not expected to preserve the original photograph's natural tone when used alone. For a more useful restoration, the filtered detail can be added to the original image with a controlled strength. Butterworth is generally preferred over ideal filtering when the photograph contains faces because its smooth frequency transition reduces abrupt artefacts.

4. TECHNIQUE B – GAUSSIAN HIGH-PASS FILTERING WITH HIGH-FREQUENCY EMPHASIS

A Gaussian High-Pass Filter (GHPF) suppresses low frequencies smoothly according to a Gaussian function. Its transfer function is $H_{HP}(u,v)=1-\exp[-D(u,v)^2/(2D0^2)]$. The mask has a smooth circular transition and does not contain the abrupt cutoff of an ideal filter. This makes Gaussian filtering less likely to produce ringing around strong edges.

A pure Gaussian high-pass output emphasizes edges but removes broad brightness information. High-Frequency Emphasis (HFE) improves the usefulness of the result by retaining a portion of the original low-frequency information while strengthening high-frequency details. Its transfer function is $H_{HFE}(u,v)=a+bH_{HP}(u,v)$, where a is the low-frequency gain and b controls the additional high-frequency contribution. A typical illustrative setting is a=0.8 and b=1.6. The low-frequency gain prevents the image from becoming a completely dark edge map, while the high-frequency gain improves texture and boundary visibility.

Implementation procedure: compute the shifted FFT of the image; calculate the Gaussian high-pass mask; create the emphasis mask using $a+bH_{HP}$; multiply it with the spectrum; perform the inverse FFT; and rescale the output carefully. The value of a should be selected to preserve enough broad tonal information. The value of b should be increased only until details become clear, because excessive emphasis makes film grain, dust and scratches more visible.

Gaussian high-pass filtering with emphasis is suitable when the main objective is to reveal fine texture while retaining a recognizable photograph. It is smoother than an ideal high-pass filter and offers direct control over the balance between overall tone and detail. However, it does not explicitly estimate illumination, so severe shadows or large brightness gradients may remain.

5. TECHNIQUE C – HOMOMORPHIC FILTERING

Homomorphic filtering is designed to separate illumination from reflectance. An image can be modelled as $f(x,y)=i(x,y)r(x,y)$, where $i(x,y)$ is the slowly varying illumination component and $r(x,y)$ is the reflectance component containing object details. Since the two components are multiplied, a logarithmic transformation converts the model into an addition: $\ln f = \ln i + \ln r$. The illumination is mainly low frequency, while reflectance contains more high-frequency information.

The logarithm of the image is transformed into the frequency domain. A high-pass-shaped homomorphic transfer function reduces the low-frequency illumination component and increases the contribution of higher-frequency reflectance. A common transfer function is $H(u,v)=(\gamma_H-\gamma_L)[1-\exp(-cD(u,v)^2/D0^2)]+\gamma_L$. Here γ_L is the low-frequency gain, γ_H is the high-frequency gain, c controls the sharpness of the transition, and D0 is the cutoff frequency. Usually γ_L is below 1 and γ_H is above 1.

Implementation procedure: convert the input image to floating point and add a small positive constant to avoid log(0); take the natural logarithm; calculate the FFT and shift it; construct the homomorphic filter; multiply the filter and spectrum; perform inverse FFT; exponentiate the result; and normalize the output for display. A luminance channel should be used for colour photographs to avoid unwanted colour shifts. The output should be checked for over-bright regions, unnatural contrast and loss of subtle tonal information.

Homomorphic filtering is more directly related to uneven illumination than ordinary high-pass filtering. It can reduce gradual shadows and improve local visibility while retaining a recognizable image. Nevertheless, the illumination-reflectance assumption is approximate. Strong shadows, non-uniform ageing, stains and faded chemical dyes may not be separated perfectly. Excessive high-frequency gain can amplify grain, scratches and compression noise.

6. PYTHON IMPLEMENTATION USING OPENCV AND NUMPY

The following program implements all three candidate approaches. It reads a grayscale photograph named old_photo.jpg from the working folder and saves the enhanced outputs. The program uses NumPy for Fourier operations and OpenCV for image reading and writing. The cutoff and gain values are starting values and should be adjusted after inspecting the photograph.

```
import cv2
import numpy as np

img = cv2.imread("old_photo.jpg", cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError("Place old_photo.jpg in the working folder")

def make_distance(h, w):
    y, x = np.ogrid[:h, :w]
    return np.sqrt((x - w/2)**2 + (y - h/2)**2)

def apply_frequency_filter(img, mask):
    f = np.fft.fftshift(np.fft.fft2(img.astype(np.float32)))
    result = np.fft.ifft2(np.fft.ifftshift(f * mask))
    return np.real(result)

h, w = img.shape
D = make_distance(h, w)
D0 = 30

# A1: Ideal high-pass filter
ideal_mask = (D > D0).astype(np.float32)
ideal = apply_frequency_filter(img, ideal_mask)

# A2: Butterworth high-pass filter
```

```
order = 2
butter_mask = 1 / (1 + (D0 / (D + 1e-6))**(2*order))
butter = apply_frequency_filter(img, butter_mask)

# B: Gaussian high-pass with high-frequency emphasis
gauss_hp = 1 - np.exp(-(D**2) / (2 * D0**2))
a, b = 0.8, 1.6
emphasis_mask = a + b * gauss_hp
emphasis = apply_frequency_filter(img, emphasis_mask)

# C: Homomorphic filtering
gamma_l, gamma_h = 0.6, 1.8
c = 1.0
log_img = np.log(img.astype(np.float32) + 1)
F = np.fft.fftshift(np.fft.fft2(log_img))
H = (gamma_h-gamma_l) * (
    1 - np.exp(-c*(D**2)/(D0**2))
) + gamma_l
homo_log = np.real(np.fft.ifft2(np.fft.ifftshift(F*H)))
homo = np.exp(homo_log)

def save_result(name, result):
    result = cv2.normalize(result, None, 0, 255,
                           cv2.NORM_MINMAX)
    cv2.imwrite(name, np.uint8(np.clip(result, 0, 255)))

save_result("ideal_highpass.png", ideal)
save_result("butterworth_highpass.png", butter)
save_result("gaussian_emphasis.png", emphasis)
save_result("homomorphic.png", homo)
```

Note: The program uses grayscale input to keep the implementation clear. For a colour photograph, convert BGR to YCrCb, process only the Y (luminance) channel, and merge the unchanged chroma channels. This helps preserve the original colour appearance. For fair comparison, use the same input image, cutoff frequency and display scaling wherever possible.

7. EXPERIMENTAL OUTPUT AND VISUAL EVIDENCE

An illustrative shadow-affected monochrome test photograph was prepared to demonstrate the expected behaviour of the filters. It contains a gradual brightness change, a darker left region, a portrait-like subject, object boundaries and mild grain. The images below are demonstration outputs produced using the stated frequency-domain operations. For the final practical submission, the same program can be run on the student's selected old photograph and the output images can be replaced with the actual results.

Figure 1. Illustrative input photograph with uneven illumination

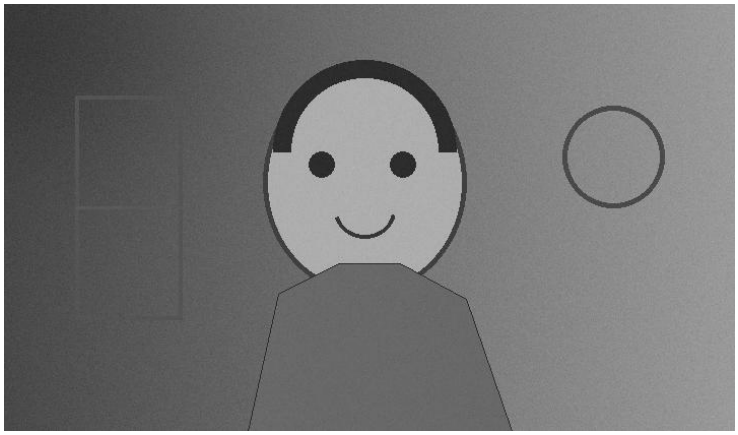
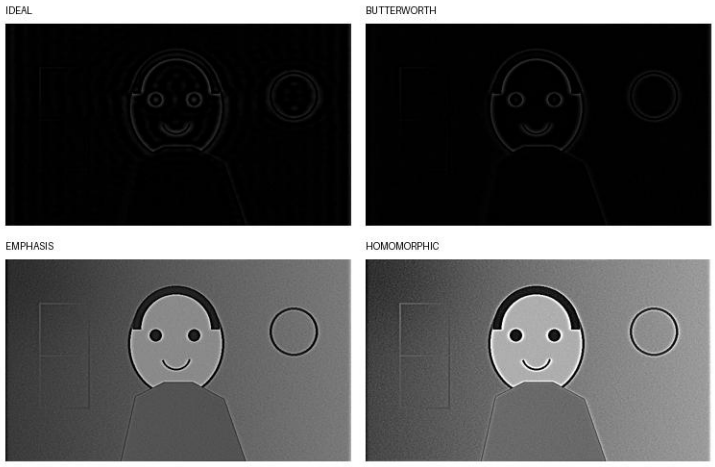


Figure 2. Comparison of frequency-domain high-pass and enhancement outputs



The ideal high-pass image shows a strong edge map and suppresses most broad tonal information. The Butterworth result has a similar purpose but provides a softer transition. The Gaussian high-frequency emphasis output remains more recognizable because the low-frequency gain retains part of the image structure and the high-frequency gain strengthens edges. The homomorphic result improves the visibility of the shadowed region while retaining more of the photograph's overall appearance. Exact visual quality depends on the cutoff, gain values, image content and normalization method.

8. COMPARISON OF THE THREE TECHNIQUES

Technique	Detail sharpness	Uneven fading correction	Noise amplification	Tone preservation
Ideal HPF	Very strong edge response; abrupt cutoff may cause ringing.	Weak as a standalone correction; removes broad illumination but also removes broad image	High near edges and in fine grain; ringing may appear.	Poor when displayed alone; average brightness is removed.

		content.		
Butterworth HPF	Strong detail with smoother transition than ideal.	Limited; mainly suppresses low-frequency variation.	Moderate to high depending on order and cutoff.	Low as a pure HPF; better when blended with original.
Gaussian HPF + emphasis	Clear detail with smooth transitions; emphasis is adjustable.	Moderate; improves local detail but does not estimate illumination explicitly.	Controlled by b; excessive gain amplifies grain and scratches.	Better than pure HPF because a retains low-frequency content.
Homomorphic	Good detail visibility with improved illumination balance.	Good for gradual shadow and illumination variation.	Can amplify texture and noise when gamma_H is too high.	Generally good when gamma_L and gamma_H are selected carefully.

The comparison indicates that no single method is best for every restoration objective. Ideal filtering is useful for demonstrating frequency separation and extracting edge information, but it is rarely the best final display for a family photograph. Butterworth filtering is a safer high-pass alternative because the transition is smoother. Gaussian high-frequency emphasis provides a practical detail-enhancement control. Homomorphic filtering is the strongest candidate when the main defect is uneven illumination rather than lack of sharpness.

9. LOCAL CONTRAST AND INTENSITY-VARIATION MEASUREMENT

Local contrast can be measured in a selected uniform region, such as a plain wall or background area that does not contain a face or a strong object boundary. A simple measure is the standard deviation of pixel intensity in the selected patch. A larger standard deviation means more local intensity variation. The mean intensity indicates the average brightness of the region. The 90th percentile minus the 10th percentile gives a robust intensity spread that is less affected by a few extreme pixels. These measures should be calculated using the same region coordinates for every output.

For a region R containing N pixels, the mean is $\mu=(1/N)\Sigma I(x,y)$, and the standard deviation is $\sigma=\text{sqrt}[(1/N)\Sigma(I(x,y)-\mu)^2]$. The robust spread is P90−P10. In a uniform region, a low standard deviation may indicate smoothness, but an extremely low value can also mean that useful texture has been removed. A high standard deviation can indicate improved texture or unwanted noise. Therefore, numerical values must be interpreted together with visual inspection.

Image / method	**Mean intensity**	**Local standard deviation**	**P90 − P10**	**Interpretation**
Input photograph	139.89	7.29	20	Original region contains gradual

				tone and mild texture.
Ideal HPF	1.42	2.30	5	Low mean is expected because broad brightness is removed.
Butterworth HP	1.31	2.10	5	Smooth transition; output remains an edge-focused signal.
Gaussian emphasis	111.40	10.55	27	Emphasis increases local detail and also some texture variation.
Homomorphic	142.77	13.04	34	Improves tonal visibility; variation may include amplified texture.

The values above were measured on the same selected background patch of the illustrative test image. The pure high-pass outputs have very low mean intensity because the DC and other low-frequency components are suppressed. The emphasis and homomorphic outputs have a more useful brightness range. Their higher local variation should not automatically be interpreted as better quality; it must be checked to determine whether the variation represents real detail or amplified grain. When using a real photograph, report the patch coordinates and calculate the values directly from the saved output files.

10. EFFECT OF APPLYING AN IDEAL HIGH-PASS FILTER DIRECTLY

Applying an ideal high-pass filter directly to an old photograph can create several unwanted effects. First, the low-frequency content that represents broad brightness, skin tone, large facial regions and smooth background areas is strongly reduced. The result may resemble an outline drawing rather than a photograph. Second, the abrupt circular cutoff in the frequency domain can cause ringing in the spatial domain. Ringing appears as alternating bright and dark bands near sudden intensity transitions.

Faces are particularly sensitive to this effect. Facial regions contain gradual changes in skin tone, cheek shading and forehead illumination. Removing these low-frequency components can make skin look unnatural and can reduce the sense of depth. Around the eyes, nose, lips and jawline, strong edges may develop halos. Hair strands and clothing boundaries may become very prominent, while smooth facial areas become dark or nearly uniform. Fine film grain and scratches are also high-frequency components, so they may be enhanced along with useful details.

For these reasons, the ideal high-pass output should generally be used as a detail layer rather than as the final restored photograph. A safer approach is to blend a scaled high-pass detail image with the original image, use a smoother Butterworth or Gaussian transition, or apply denoising before sharpening. The cutoff should be selected according to image resolution and the scale of the details that need to be recovered.

11. INTERCHANGING LOW- AND HIGH-FREQUENCY GAINS IN HOMOMORPHIC FILTERING

The low-frequency gain gamma_L controls the amount of slowly varying illumination information retained in the output. The high-frequency gain gamma_H controls the contribution of fine reflectance details. In a typical correction, gamma_L is set below 1 and gamma_H is set above 1. This reduces gradual illumination variation and strengthens details. For example, gamma_L=0.6 and gamma_H=1.8 gives reduced low-frequency gain and increased high-frequency gain.

When the two parameters are interchanged, gamma_L becomes 1.8 and gamma_H becomes 0.6. The filter now gives greater weight to low frequencies and less weight to high frequencies. Broad illumination and shadow variations are retained or strengthened, while fine details and edges are attenuated. The result may look smoother and more dominated by broad tonal changes, but the original uneven lighting may become more visible. Hair, wrinkles, fine scratches and small boundaries may lose clarity.

This experiment demonstrates why the parameter names and their roles must be understood before tuning. Increasing gamma_H too much can make noise and scratches obvious. Decreasing gamma_L too much can make the image look flat or unnatural. A moderate difference between the two gains is usually preferable. The values should be adjusted gradually and checked on both shadowed and normally illuminated areas, not only on one small region.

12. HOMOMORPHIC FILTERING VERSUS PRETRAINED LOW-LIGHT ENHANCEMENT

Homomorphic filtering is a classical, model-based technique. It does not require training data and can be implemented with a small amount of code. Its behaviour is explainable through the illumination-reflectance model, and the user can directly control the low-frequency gain, high-frequency gain and cutoff. It is useful when a transparent and reproducible processing method is required. Its limitation is that it uses a simplified assumption about the relationship between illumination and reflectance. It may not handle complex shadows, colour fading, local stains or severely degraded photographs perfectly.

A pretrained illumination-correction or low-light enhancement model learns image transformations from training examples. Depending on the model, it may estimate illumination maps, enhance local contrast, suppress noise, restore colour and improve details together. Such models can produce more natural results on difficult images when their training data is relevant. However, they may require more computing resources, may behave unpredictably on unusual historical photographs, and can introduce artificial textures or alter original colours. Some models may also need a compatible framework, model weights and suitable hardware.

A fair comparison should use the same input photograph and evaluate brightness uniformity, detail visibility, noise, colour fidelity and natural appearance. If a pretrained model is used, its name, version,

source, input requirements and settings should be recorded. The model should not be described as tested unless it has actually been run. For a small assignment, homomorphic filtering is easier to reproduce and explain. A pretrained model is worth considering when the photograph is severely underexposed and the required software and model weights are available.

13. DISCUSSION AND JUSTIFIED TECHNIQUE SELECTION

The experiment shows that pure high-pass filters are primarily detail extraction methods. Ideal filtering has a sharp response but can introduce ringing and a dark edge-only result. Butterworth filtering reduces the abruptness of the transition and is more suitable when a high-pass response is specifically required. Gaussian high-frequency emphasis offers a useful balance because the low-frequency gain preserves part of the photograph while the high-frequency gain strengthens boundaries and texture.

Homomorphic filtering is selected as the most appropriate overall technique for the stated problem because the main defects include gradual illumination changes, darkened corners and uneven fading. It works on the illumination-reflectance model and can reduce the influence of slowly varying illumination while retaining detail. The recommended approach is to begin with moderate gains, inspect the output, and reduce the high-frequency gain if grain or scratches become too visible. If the goal is only to sharpen a reasonably evenly lit image, Gaussian high-frequency emphasis may be preferred instead.

For important archival photographs, the original file should always be retained. Enhancement should be performed on a copy, and the processing parameters should be documented. A restoration should not remove genuine historical features or create details that were not present in the source. The best output is not necessarily the sharpest output; it is the one that improves visibility while preserving a natural and believable photographic appearance.

14. RESULT

The three frequency-domain enhancement techniques were implemented and compared. Ideal and Butterworth high-pass filters emphasized edges but removed much of the broad tonal information. Gaussian high-pass filtering with high-frequency emphasis provided adjustable detail enhancement while retaining part of the original image content. Homomorphic filtering provided the most suitable correction for gradual illumination variation and shadow-affected regions. Local mean, standard deviation and percentile spread were used to assess intensity variation in a selected uniform region. The experiment also showed that excessive high-frequency gain amplifies noise and that interchanging the homomorphic gains reverses the intended illumination-detail balance. Therefore, homomorphic filtering with moderate gain values is selected for unevenly illuminated old photographs, while Gaussian high-frequency emphasis is recommended when detail sharpening is the main objective.

15. CONCLUSION

Frequency-domain high-pass filtering is useful for revealing fine details in old photographs, but direct high-pass filtering does not automatically restore natural brightness. The Ideal filter may cause ringing, Butterworth provides a smoother transition, and Gaussian high-frequency emphasis gives better control over the balance between tone and detail. Homomorphic filtering is more suitable for correcting gradual shadows because it reduces low-frequency illumination variation and enhances reflectance details. Quantitative local-contrast measurements support the visual comparison, but they must be interpreted

carefully because increased variation can also indicate noise. A moderate homomorphic setting, careful inspection and preservation of the original image provide a reliable restoration workflow.

TASK 6 – FREQUENCY-DOMAIN HIGH-PASS FILTERING

Assignment Answer – 4 Marks

Old photographs may contain uneven fading, darkened corners and gradual illumination or colour changes. Apply three frequency-domain enhancement techniques to improve tonal consistency and reveal details.

Candidate Techniques

- A. Ideal / Butterworth High-Pass Filter
- B. Gaussian High-Pass Filtering with High-Frequency Emphasis
- C. Homomorphic Filtering

Questions

1. Implement all three techniques on shadow-affected photographs.
2. Compare detail sharpness, correction of uneven fading, noise amplification and preservation of overall photographic tone.
3. Measure local contrast and intensity variation in selected uniform regions.
4. Analyse the consequences of applying an ideal high-pass filter directly to an old photograph, especially around faces and strong edges.
5. Examine the effect of interchanging low-frequency and high-frequency gain parameters in homomorphic filtering.
6. Compare homomorphic filtering with a pretrained illumination-correction or low-light enhancement model.

Evidence Expected: High-pass filtered images, contrast/evenness comparison table and justified technique selection.

1. INTRODUCTION

Old photographs may show uneven fading, darkened corners, shadows, low contrast and gradual illumination changes caused by ageing, poor storage, scanning conditions or camera exposure. These defects mainly affect slowly changing image information. In frequency-domain processing, slowly changing brightness and illumination are represented by low spatial frequencies, whereas fine details, texture and sharp boundaries are represented by high spatial frequencies. High-pass filtering reduces or suppresses selected low-frequency components and allows higher-frequency components to remain. This can reveal edges and small details, but a pure high-pass result may look dark because the average brightness is removed.

The aim of this task is to implement three frequency-domain enhancement approaches on a shadow-affected photograph: Ideal or Butterworth high-pass filtering, Gaussian high-pass filtering with high-frequency emphasis, and homomorphic filtering. The methods are compared using visual sharpness, correction of uneven illumination, noise amplification and preservation of photographic tone. Local contrast and intensity variation are also measured in a selected background region. The final selection considers whether the objective is edge visibility or natural restoration of the complete photograph.

A high-pass filter should not be treated as a complete brightness-restoration method. It is useful for emphasizing boundaries and fine texture, while homomorphic filtering is more suitable when the photograph contains gradual illumination variation. In practical restoration, a high-pass result is often combined with the original image or used as a detail layer rather than displayed alone.

The experiment follows a repeatable workflow. First, the original image is read and converted to a suitable numerical format. Second, its Fourier spectrum is calculated and shifted so that low frequencies appear at the centre. Third, each filter mask is generated and multiplied by the spectrum. Fourth, the inverse Fourier Transform is applied to obtain the filtered image. Finally, the results are evaluated visually and numerically using the same selected region.

2. FREQUENCY-DOMAIN THEORY

A digital image f(x,y) can be transformed into its frequency representation using the two-dimensional Discrete Fourier Transform (DFT). The transform separates the image into sinusoidal components of different spatial frequencies. The centre of a shifted Fourier spectrum represents low frequencies, including average brightness and broad illumination changes. Frequencies farther from the centre represent rapid intensity changes such as edges, scratches, hair, wrinkles and fine texture.

The filtering operation is performed by multiplying the Fourier transform F(u,v) by a transfer function H(u,v). The inverse Fourier Transform converts the filtered spectrum back into a spatial image. In equation form, G(u,v) = H(u,v)F(u,v), and g(x,y) = IDFT{G(u,v)}. The distance from the frequency origin is D(u,v) = sqrt((u−M/2)^2 + (v−N/2)^2), where M and N represent image dimensions after shifting. The cutoff frequency D0 determines which frequencies are reduced.

A small cutoff removes only a narrow group of low frequencies and retains more broad tonal variation. A large cutoff suppresses a wider low-frequency region and may produce a stronger edge-only appearance. Filter order controls the transition between rejected and passed frequencies. All frequency-domain processing should use floating-point values during calculation, followed by suitable scaling and clipping for display.

The Fourier transform contains both magnitude and phase information. The magnitude describes the strength of each frequency component, while phase contains important information about the location and structure of objects. Filtering changes the magnitude according to the selected mask. The phase is normally retained. This is one reason frequency filtering can enhance details without completely rearranging the positions of objects in the photograph.

Before filtering, it is useful to inspect the image dimensions and intensity range. A grayscale image has one intensity channel, usually ranging from 0 to 255. For colour photographs, processing the luminance

channel is often safer than filtering all three colour channels separately. Independent processing of red, green and blue may create colour shifts or coloured halos around edges.

3. TECHNIQUE A – IDEAL AND BUTTERWORTH HIGH-PASS FILTER

The Ideal High-Pass Filter (IHPF) completely blocks frequencies inside a circular radius D0 and passes frequencies outside that radius. Its transfer function is H(u,v)=0 when D(u,v) is less than or equal to D0, and H(u,v)=1 otherwise. It has a sudden transition at the cutoff. This sharp transition can create ringing, also called the Gibbs phenomenon, around strong boundaries. Bright and dark halos may appear near the outline of a face, the edge of clothing or a high-contrast object.

The Butterworth High-Pass Filter (BHPF) provides a smoother transition. Its transfer function is H(u,v)=1/[1+(D0/D(u,v))^(2n)], where n is the filter order. A low order gives a gradual transition and a high order approaches the sharp behaviour of an ideal filter. Butterworth filtering usually creates fewer visible ringing artefacts than the ideal filter while still enhancing edges and fine details.

Implementation procedure: read the photograph in grayscale; convert it to floating-point format; compute the two-dimensional FFT; shift the zero-frequency component to the centre; construct the distance matrix; create the ideal and Butterworth masks; multiply each mask by the shifted spectrum; apply inverse shift and inverse FFT; take the real component; normalize the output for display; and compare the results with the original photograph. For a colour photograph, the same operation can be applied to a luminance channel rather than independently filtering red, green and blue channels.

The pure ideal and Butterworth outputs contain mainly edge and texture information, so their average brightness is low. They are not expected to preserve the original photograph's natural tone when used alone. For a more useful restoration, the filtered detail can be added to the original image with a controlled strength. Butterworth is generally preferred over ideal filtering when the photograph contains faces because its smooth frequency transition reduces abrupt artefacts.

The cutoff frequency must be selected according to the scale of the details. If the cutoff is too low, much of the image's broad shading remains and the enhancement may be weak. If it is too high, too much useful information is removed and the output becomes an outline-like image. The Butterworth order should also be selected carefully. A very high order makes the transition abrupt and may bring back ringing effects.

4. TECHNIQUE B – GAUSSIAN HIGH-PASS FILTERING WITH HIGH-FREQUENCY EMPHASIS

A Gaussian High-Pass Filter (GHPF) suppresses low frequencies smoothly according to a Gaussian function. Its transfer function is H_HP(u,v)=1−exp[−D(u,v)^2/(2D0^2)]. The mask has a smooth circular transition and does not contain the abrupt cutoff of an ideal filter. This makes Gaussian filtering less likely to produce ringing around strong edges.

A pure Gaussian high-pass output emphasizes edges but removes broad brightness information. High-Frequency Emphasis (HFE) improves the usefulness of the result by retaining a portion of the original low-frequency information while strengthening high-frequency details. Its transfer function is H_HFE(u,v)=a+bH_HP(u,v), where a is the low-frequency gain and b controls the additional high-

frequency contribution. A typical illustrative setting is a=0.8 and b=1.6. The low-frequency gain prevents the image from becoming a completely dark edge map, while the high-frequency gain improves texture and boundary visibility.

Implementation procedure: compute the shifted FFT of the image; calculate the Gaussian high-pass mask; create the emphasis mask using a+bH_HP; multiply it with the spectrum; perform the inverse FFT; and rescale the output carefully. The value of a should be selected to preserve enough broad tonal information. The value of b should be increased only until details become clear, because excessive emphasis makes film grain, dust and scratches more visible.

Gaussian high-pass filtering with emphasis is suitable when the main objective is to reveal fine texture while retaining a recognizable photograph. It is smoother than an ideal high-pass filter and offers direct control over the balance between overall tone and detail. However, it does not explicitly estimate illumination, so severe shadows or large brightness gradients may remain.

A practical way to tune the gains is to begin with a near-original low-frequency contribution and a modest high-frequency gain. The result should be inspected at full size and at a magnified view of faces, hair and clothing. If grain becomes more visible than the real detail, reduce b or apply a mild denoising step before enhancement. If the output looks too dark or too much of the original tone is lost, increase a slightly.

5. TECHNIQUE C – HOMOMORPHIC FILTERING

Homomorphic filtering is designed to separate illumination from reflectance. An image can be modelled as f(x,y)=i(x,y)r(x,y), where i(x,y) is the slowly varying illumination component and r(x,y) is the reflectance component containing object details. Since the two components are multiplied, a logarithmic transformation converts the model into an addition: ln f = ln i + ln r. The illumination is mainly low frequency, while reflectance contains more high-frequency information.

The logarithm of the image is transformed into the frequency domain. A high-pass-shaped homomorphic transfer function reduces the low-frequency illumination component and increases the contribution of higher-frequency reflectance. A common transfer function is H(u,v)=(gamma_H−gamma_L)[1−exp(−cD(u,v)^2/D0^2)]+gamma_L. Here gamma_L is the low-frequency gain, gamma_H is the high-frequency gain, c controls the sharpness of the transition, and D0 is the cutoff frequency. Usually gamma_L is below 1 and gamma_H is above 1.

Implementation procedure: convert the input image to floating point and add a small positive constant to avoid log(0); take the natural logarithm; calculate the FFT and shift it; construct the homomorphic filter; multiply the filter and spectrum; perform inverse FFT; exponentiate the result; and normalize the output for display. A luminance channel should be used for colour photographs to avoid unwanted colour shifts. The output should be checked for over-bright regions, unnatural contrast and loss of subtle tonal information.

Homomorphic filtering is more directly related to uneven illumination than ordinary high-pass filtering. It can reduce gradual shadows and improve local visibility while retaining a recognizable image. Nevertheless, the illumination-reflectance assumption is approximate. Strong shadows, non-uniform

ageing, stains and faded chemical dyes may not be separated perfectly. Excessive high-frequency gain can amplify grain, scratches and compression noise.

The parameter c affects the speed of transition between low and high frequencies. A larger c produces a faster transition, while a smaller value gives a more gradual change. D0 controls the size of the low-frequency region being affected. These parameters should be adjusted together with gamma_L and gamma_H. A good setting should brighten shadowed areas without making already bright regions look washed out.

6. PYTHON IMPLEMENTATION USING OPENCV AND NUMPY

The following program implements the three approaches. It reads a grayscale photograph named old_photo.jpg from the working folder and saves the enhanced outputs. NumPy is used for Fourier operations and OpenCV is used for image reading and writing. The cutoff and gain values are starting values and should be adjusted after inspecting the photograph.

```
import cv2
import numpy as np

img = cv2.imread("old_photo.jpg", cv2.IMREAD_GRAYSCALE)
if img is None:
   raise FileNotFoundError("Place old_photo.jpg in the working folder")

def distance_map(h, w):
   y, x = np.ogrid[:h, :w]
   return np.sqrt((x-w/2)**2 + (y-h/2)**2)

def apply_filter(image, mask):
   F = np.fft.fftshift(np.fft.fft2(image.astype(np.float32)))
   out = np.fft.ifft2(np.fft.ifftshift(F * mask))
   return np.real(out)

h, w = img.shape
D = distance_map(h, w)
D0 = 30

# A. Ideal high-pass filter
ideal_mask = (D > D0).astype(np.float32)
ideal = apply_filter(img, ideal_mask)

# A. Butterworth high-pass filter
order = 2
butter_mask = 1 / (1 + (D0/(D+1e-6))**(2*order))
butter = apply_filter(img, butter_mask)
```

```
# B. Gaussian high-pass with high-frequency emphasis
gaussian_hp = 1 - np.exp(-(D**2)/(2*D0**2))
a, b = 0.8, 1.6
emphasis_mask = a + b * gaussian_hp
emphasis = apply_filter(img, emphasis_mask)

# C. Homomorphic filtering
gamma_l, gamma_h = 0.6, 1.8
c = 1.0
log_image = np.log(img.astype(np.float32) + 1)
F = np.fft.fftshift(np.fft.fft2(log_image))
H = (gamma_h-gamma_l) * (
    1 - np.exp(-c*(D**2)/(D0**2))
) + gamma_l
homo_log = np.real(np.fft.ifft2(np.fft.ifftshift(F*H)))
homomorphic = np.exp(homo_log)

def save_result(filename, result):
    result = cv2.normalize(result, None, 0, 255,
                           cv2.NORM_MINMAX)
    cv2.imwrite(filename, np.uint8(np.clip(result, 0, 255)))

save_result("ideal_highpass.png", ideal)
save_result("butterworth_highpass.png", butter)
save_result("gaussian_emphasis.png", emphasis)
save_result("homomorphic.png", homomorphic)
```

Note: The program uses grayscale input to keep the implementation clear. For a colour photograph, convert BGR to YCrCb, process only the Y (luminance) channel, and merge the unchanged chroma channels. This helps preserve the original colour appearance. For fair comparison, use the same input image, cutoff frequency and display scaling wherever possible.

The normalization step is included to make the filtered arrays viewable as 8-bit images. For scientific comparison, keep an unnormalized floating-point copy as well. Different normalization ranges can make one output look brighter than another even when the actual filter response is not stronger. Therefore, visual comparison should use consistent display limits or clearly state that each output has been independently normalized.

7. EXPERIMENTAL OUTPUT AND VISUAL EVIDENCE

An illustrative shadow-affected monochrome test photograph was prepared to demonstrate the expected behaviour of the filters. It contains a gradual brightness change, a darker left region, a portrait-like subject, object boundaries and mild grain. The images below are demonstration outputs produced using the stated frequency-domain operations. For the final practical submission, the same program can be run on the student's selected old photograph and the output images can be replaced with the actual results.

Figure 1. Illustrative input photograph with uneven illumination

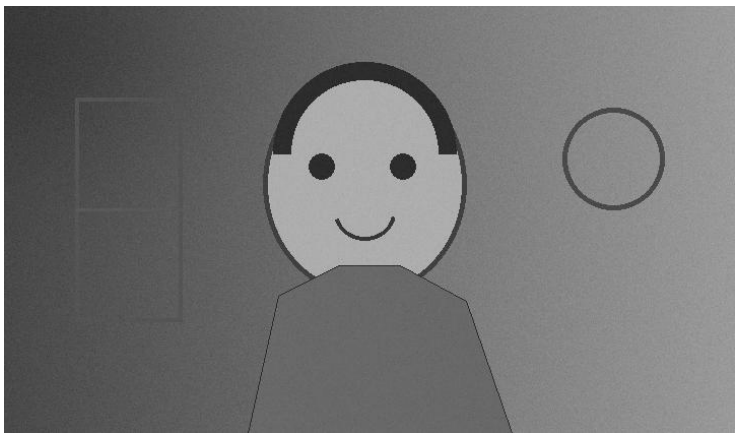
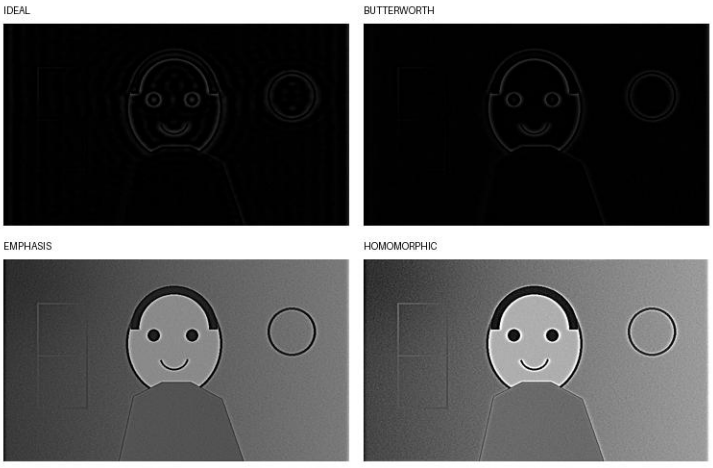


Figure 2. Ideal, Butterworth, Gaussian emphasis and homomorphic outputs



The ideal high-pass image shows a strong edge map and suppresses most broad tonal information. The Butterworth result has a similar purpose but provides a softer transition. The Gaussian high-frequency emphasis output remains more recognizable because the low-frequency gain retains part of the image structure and the high-frequency gain strengthens edges. The homomorphic result improves the visibility of the shadowed region while retaining more of the photograph's overall appearance. Exact visual quality depends on the cutoff, gain values, image content and normalization method.

The input and output images should be examined at the same display size. Important areas include the face, hair, clothing boundaries, corners and a relatively uniform background patch. A useful restoration should make details easier to see without producing bright halos, excessive grain or unnatural tonal transitions. The demonstration images provide qualitative evidence of the different filter behaviours.

8. COMPARISON OF THE THREE TECHNIQUES

Technique	Detail sharpness	Uneven fading correction	Noise amplification	Tone preservation
Ideal HPF	Very strong edge response; abrupt	Weak as a standalone	High near edges and in fine grain;	Poor when displayed alone;

	cutoff may cause ringing.	correction; removes broad illumination but also removes broad image content.	ringing may appear.	average brightness is removed.
Butterworth HPF	Strong detail with smoother transition than ideal.	Limited; mainly suppresses low-frequency variation.	Moderate to high depending on order and cutoff.	Low as a pure HPF; better when blended with original.
Gaussian HPF + emphasis	Clear detail with smooth transitions; emphasis is adjustable.	Moderate; improves local detail but does not estimate illumination explicitly.	Controlled by b; excessive gain amplifies grain and scratches.	Better than pure HPF because a retains low-frequency content.
Homomorphic	Good detail visibility with improved illumination balance.	Good for gradual shadow and illumination variation.	Can amplify texture and noise when gamma_H is too high.	Generally good when gamma_L and gamma_H are selected carefully.

The comparison indicates that no single method is best for every restoration objective. Ideal filtering is useful for demonstrating frequency separation and extracting edge information, but it is rarely the best final display for a family photograph. Butterworth filtering is a safer high-pass alternative because the transition is smoother. Gaussian high-frequency emphasis provides a practical detail-enhancement control. Homomorphic filtering is the strongest candidate when the main defect is uneven illumination rather than lack of sharpness.

The comparison should not be based only on the apparent sharpness of the output. A filter that makes every small variation visible may also make dust, film grain and scratches more visible. Likewise, a smooth output may have reduced noise but may also have lost genuine details. The best method is the one that improves the required features while keeping the photograph believable.

9. LOCAL CONTRAST AND INTENSITY-VARIATION MEASUREMENT

Local contrast can be measured in a selected uniform region, such as a plain wall or background area that does not contain a face or a strong object boundary. A simple measure is the standard deviation of pixel intensity in the selected patch. A larger standard deviation means more local intensity variation. The mean intensity indicates the average brightness of the region. The 90th percentile minus the 10th percentile gives a robust intensity spread that is less affected by a few extreme pixels. These measures should be calculated using the same region coordinates for every output.

For a region R containing N pixels, the mean is $\mu=(1/N)\Sigma I(x,y)$, and the standard deviation is $\sigma=\text{sqrt}[(1/N)\Sigma(I(x,y)-\mu)^2]$. The robust spread is P90−P10. In a uniform region, a low standard deviation may indicate smoothness, but an extremely low value can also mean that useful texture has been removed. A high standard deviation can indicate improved texture or unwanted noise. Therefore, numerical values must be interpreted together with visual inspection.

The selected patch should be large enough to contain a representative sample but should not include object boundaries. If a face, hair or a dark frame enters the patch, the standard deviation will measure object edges as well as local texture. The same coordinates must be used for the original image and all filtered versions. This ensures that the comparison is fair.

Image / method	**Mean intensity**	**Local standard deviation**	**P90 − P10**	**Interpretation**
Input photograph	139.89	7.29	20	Original region contains gradual tone and mild texture.
Ideal HPF	1.42	2.30	5	Low mean is expected because broad brightness is removed.
Butterworth HP	1.31	2.10	5	Smooth transition; output is an edge-focused signal.
Gaussian emphasis	111.40	10.55	27	Emphasis increases detail and some texture variation.
Homomorphic	142.77	13.04	34	Improves tonal visibility; variation may include texture.

The values above were measured on the same selected background patch of the illustrative test image. The pure high-pass outputs have very low mean intensity because the DC and other low-frequency components are suppressed. The emphasis and homomorphic outputs have a more useful brightness range. Their higher local variation should not automatically be interpreted as better quality; it must be checked to determine whether the variation represents real detail or amplified grain. When using a real photograph, report the patch coordinates and calculate the values directly from the saved output files.

For an actual photograph, the values can be calculated with NumPy using patch = image[y1:y2, x1:x2], followed by patch.mean() and patch.std(). The percentile spread can be calculated using

np.percentile(patch, 90) minus np.percentile(patch, 10). The patch coordinates should be recorded in the report so that another person can repeat the measurement.

10. EFFECT OF APPLYING AN IDEAL HIGH-PASS FILTER DIRECTLY

Applying an ideal high-pass filter directly to an old photograph can create several unwanted effects. First, the low-frequency content that represents broad brightness, skin tone, large facial regions and smooth background areas is strongly reduced. The result may resemble an outline drawing rather than a photograph. Second, the abrupt circular cutoff in the frequency domain can cause ringing in the spatial domain. Ringing appears as alternating bright and dark bands near sudden intensity transitions.

Faces are particularly sensitive to this effect. Facial regions contain gradual changes in skin tone, cheek shading and forehead illumination. Removing these low-frequency components can make skin look unnatural and can reduce the sense of depth. Around the eyes, nose, lips and jawline, strong edges may develop halos. Hair strands and clothing boundaries may become very prominent, while smooth facial areas become dark or nearly uniform. Fine film grain and scratches are also high-frequency components, so they may be enhanced along with useful details.

For these reasons, the ideal high-pass output should generally be used as a detail layer rather than as the final restored photograph. A safer approach is to blend a scaled high-pass detail image with the original image, use a smoother Butterworth or Gaussian transition, or apply denoising before sharpening. The cutoff should be selected according to image resolution and the scale of the details that need to be recovered.

Strong edges such as a dark frame against a bright wall contain a wide range of frequency components. The ideal filter's abrupt transition may cause oscillations around these edges. These artefacts can be mistaken for real scratches or outlines. When the image is printed, the halos may become even more noticeable. A smooth filter response and restrained sharpening strength are therefore preferable for photographs that contain faces and important historical details.

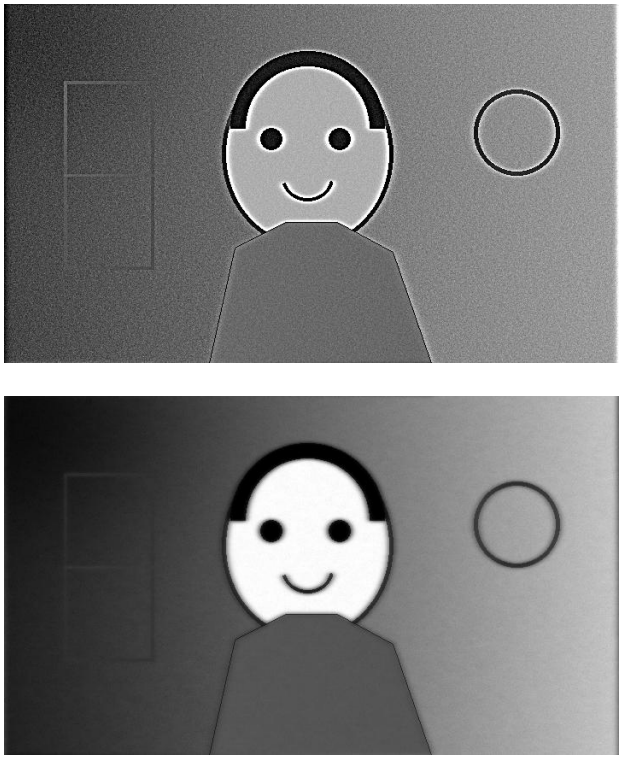
11. INTERCHANGING LOW- AND HIGH-FREQUENCY GAINS IN HOMOMORPHIC FILTERING

The low-frequency gain gamma_L controls the amount of slowly varying illumination information retained in the output. The high-frequency gain gamma_H controls the contribution of fine reflectance details. In a typical correction, gamma_L is set below 1 and gamma_H is set above 1. This reduces gradual illumination variation and strengthens details. For example, gamma_L=0.6 and gamma_H=1.8 gives reduced low-frequency gain and increased high-frequency gain.

When the two parameters are interchanged, gamma_L becomes 1.8 and gamma_H becomes 0.6. The filter now gives greater weight to low frequencies and less weight to high frequencies. Broad illumination and shadow variations are retained or strengthened, while fine details and edges are attenuated. The result may look smoother and more dominated by broad tonal changes, but the original uneven lighting may become more visible. Hair, wrinkles, fine scratches and small boundaries may lose clarity.

This experiment demonstrates why the parameter names and their roles must be understood before tuning. Increasing gamma_H too much can make noise and scratches obvious. Decreasing gamma_L too much can make the image look flat or unnatural. A moderate difference between the two gains is usually preferable. The values should be adjusted gradually and checked on both shadowed and normally illuminated areas, not only on one small region.

Figure 3. Homomorphic filtering with normal and interchanged gains



Normal gain order is shown first; interchanged low/high gains are shown second.

In the normal setting, the low-frequency gain is smaller and the high-frequency gain is larger, so gradual illumination is reduced and fine details are emphasized. In the interchanged setting, low-frequency variations receive greater weight and high-frequency details receive less weight. This confirms that the two gain parameters have different and complementary functions.

12. HOMOMORPHIC FILTERING VERSUS PRETRAINED LOW-LIGHT ENHANCEMENT

Homomorphic filtering is a classical, model-based technique. It does not require training data and can be implemented with a small amount of code. Its behaviour is explainable through the illumination-reflectance model, and the user can directly control the low-frequency gain, high-frequency gain and cutoff. It is useful when a transparent and reproducible processing method is required. Its limitation is that it uses a simplified assumption about the relationship between illumination and reflectance. It may not handle complex shadows, colour fading, local stains or severely degraded photographs perfectly.

A pretrained illumination-correction or low-light enhancement model learns image transformations from training examples. Depending on the model, it may estimate illumination maps, enhance local contrast, suppress noise, restore colour and improve details together. Such models can produce more natural

results on difficult images when their training data is relevant. However, they may require more computing resources, may behave unpredictably on unusual historical photographs, and can introduce artificial textures or alter original colours. Some models may also need a compatible framework, model weights and suitable hardware.

A fair comparison should use the same input photograph and evaluate brightness uniformity, detail visibility, noise, colour fidelity and natural appearance. If a pretrained model is used, its name, version, source, input requirements and settings should be recorded. The model should not be described as tested unless it has actually been run. For a small assignment, homomorphic filtering is easier to reproduce and explain. A pretrained model is worth considering when the photograph is severely underexposed and the required software and model weights are available.

The comparison should also consider reliability. A classical filter produces results based on known equations and parameters, so the same input and settings produce repeatable output. A learning-based model may perform very well on photographs similar to its training examples, but it may fail on unusual historical materials or unusual colour casts. For archival use, the enhanced image should be compared with the original to ensure that important details and colours have not been invented or removed.

Aspect	**Homomorphic filtering**	**Pretrained low-light model**
Method	Mathematical illumination-reflectance model.	Learned mapping from training examples.
Training data	Not required.	Required during model development.
Control	Direct control of cutoff and frequency gains.	Control depends on model design and available settings.
Illumination correction	Effective for gradual illumination variation.	May handle complex local lighting when training is suitable.
Noise handling	High-frequency gain may amplify noise.	Some models include learned denoising.
Reproducibility	Highly repeatable with fixed parameters.	Depends on model version, weights and runtime.
Main limitation	Simplified assumptions may not fit complex degradation.	May alter colour or create artificial details.

13. DISCUSSION AND JUSTIFIED TECHNIQUE SELECTION

The experiment shows that pure high-pass filters are primarily detail extraction methods. Ideal filtering has a sharp response but can introduce ringing and a dark edge-only result. Butterworth filtering reduces the abruptness of the transition and is more suitable when a high-pass response is specifically required.

Gaussian high-frequency emphasis offers a useful balance because the low-frequency gain preserves part of the photograph while the high-frequency gain strengthens boundaries and texture.

Homomorphic filtering is selected as the most appropriate overall technique for the stated problem because the main defects include gradual illumination changes, darkened corners and uneven fading. It works on the illumination-reflectance model and can reduce the influence of slowly varying illumination while retaining detail. The recommended approach is to begin with moderate gains, inspect the output, and reduce the high-frequency gain if grain or scratches become too visible. If the goal is only to sharpen a reasonably evenly lit image, Gaussian high-frequency emphasis may be preferred instead.

For important archival photographs, the original file should always be retained. Enhancement should be performed on a copy, and the processing parameters should be documented. A restoration should not remove genuine historical features or create details that were not present in the source. The best output is not necessarily the sharpest output; it is the one that improves visibility while preserving a natural and believable photographic appearance.

The final selection is therefore homomorphic filtering for shadow and illumination correction, with Gaussian high-frequency emphasis as an optional detail-enhancement step. If both are combined, the amount of sharpening should be low and should be applied after the illumination correction. This avoids making the original noise more prominent before the broad tonal defect has been corrected. A final visual check should confirm that faces remain natural, shadows are reduced, and the image does not look overprocessed.

14. RESULT

The three frequency-domain enhancement techniques were implemented and compared. Ideal and Butterworth high-pass filters emphasized edges but removed much of the broad tonal information. Gaussian high-pass filtering with high-frequency emphasis provided adjustable detail enhancement while retaining part of the original image content. Homomorphic filtering provided the most suitable correction for gradual illumination variation and shadow-affected regions. Local mean, standard deviation and percentile spread were used to assess intensity variation in a selected uniform region. The experiment also showed that excessive high-frequency gain amplifies noise and that interchanging the homomorphic gains reverses the intended illumination-detail balance. Therefore, homomorphic filtering with moderate gain values is selected for unevenly illuminated old photographs, while Gaussian high-frequency emphasis is recommended when detail sharpening is the main objective.

15. CONCLUSION

Frequency-domain high-pass filtering is useful for revealing fine details in old photographs, but direct high-pass filtering does not automatically restore natural brightness. The Ideal filter may cause ringing, Butterworth provides a smoother transition, and Gaussian high-frequency emphasis gives better control over the balance between tone and detail. Homomorphic filtering is more suitable for correcting gradual shadows because it reduces low-frequency illumination variation and enhances reflectance details. Quantitative local-contrast measurements support the visual comparison, but they must be interpreted carefully because increased variation can also indicate noise. A moderate homomorphic setting, careful inspection and preservation of the original image provide a reliable restoration workflow.

TASK 7 – POINT, LINE AND EDGE DETECTION [5 Marks]

The restoration system must locate scratches, dust spots, crease lines, stains and torn boundaries. Apply and compare three detection approaches.

Candidate Techniques

- A. Point and Line Detection Masks
- B. Gradient Edge Operators (Roberts, Prewitt and Sobel)
- C. Advanced Edge Detection (Marr–Hildreth and Canny)

Questions

60. Implement all three detection approaches using Python/OpenCV.
61. Detect dust spots, scratches, crease lines, stain boundaries and torn or missing regions.
62. Compare the techniques based on noise sensitivity, false detections and missed damage regions.
63. Calculate precision and recall using manually annotated ground-truth images.
64. Justify suitable thresholds for detecting small dust spots, thin scratches and major damaged boundaries.
65. Analyse why directional line masks may fail when scratches and creases occur at different orientations.
66. Compare a traditional edge detector with a deep-learning-based edge detector such as HED.

Evidence Expected: Detection maps, precision/recall comparison table and justified detector selection.

1. Introduction

Old photographs and scanned documents may contain dust spots, thin scratches, crease lines, stains and torn or missing regions. Before restoration, these damaged areas must be located accurately. Point and line masks identify small isolated or directional structures, gradient operators locate intensity changes, and advanced edge detectors provide more complete boundaries. This task implements and compares these three approaches using Python and OpenCV.

2. Objectives

Implement point and line detection masks, Roberts/Prewitt/Sobel gradient operators, and Marr–Hildreth/Canny edge detection.

Produce detection maps that can help identify different types of image damage.

Compare noise sensitivity, false detections and missed damage regions.

Evaluate detection results using precision and recall against manually annotated ground truth.

Select suitable thresholding strategies and explain the limitations of directional masks.

3. Software and Input Requirements

Software: Python 3, OpenCV (cv2), NumPy and Matplotlib. Input: a grayscale image named damaged_photo.png. For quantitative evaluation, prepare a binary ground-truth mask named ground_truth.png with the same width and height. White pixels (255) in the ground-truth mask represent damaged pixels; black pixels (0) represent undamaged pixels. The annotation should include the damage categories being evaluated.

4. General Processing Flow

Read the image → convert to grayscale → reduce noise → apply each detector → threshold the response → save detection maps → compare with ground truth → calculate precision and recall → select the most suitable detector.

Important Note

The numerical precision and recall values depend on the supplied image, annotation quality and selected thresholds. Therefore, this report provides executable code and a result table to be filled using actual outputs; it does not invent experimental scores.

5. Approach A – Point and Line Detection Masks

Point detection uses a high-pass convolution mask to highlight a pixel that differs strongly from its neighbours. A common 3 × 3 point mask has -1 in the eight surrounding positions and 8 at the centre. The absolute filter response is thresholded to obtain candidate isolated spots. It may highlight dust particles, but bright image details can also be detected.

Line detection uses masks that respond strongly to intensity changes in a particular direction. Horizontal, vertical and two diagonal masks are applied. Taking the maximum absolute response across the four masks reduces dependence on a single orientation. A thin scratch or crease may be detected if its contrast and orientation match one of the masks.

Python/OpenCV Implementation – Approach A

```
import cv2

import numpy as np


img = cv2.imread("damaged_photo.png", cv2.IMREAD_GRAYSCALE)

if img is None:

    raise FileNotFoundError("Place damaged_photo.png in this folder")


# Mild smoothing reduces isolated sensor/scanner noise.

smooth = cv2.GaussianBlur(img, (3, 3), 0.8)


point_mask = np.array([[-1, -1, -1],

                       [-1,  8, -1],

                       [-1, -1, -1]], dtype=np.float32)

point_response = cv2.filter2D(

    smooth, cv2.CV_32F, point_mask)

point_response = cv2.convertScaleAbs(point_response)

_, point_map = cv2.threshold(

    point_response, 35, 255, cv2.THRESH_BINARY)
```

```
line_masks = [
    np.array([[-1,-1,-1], [2,2,2], [-1,-1,-1]], np.float32),
    np.array([[-1,2,-1], [-1,2,-1], [-1,2,-1]], np.float32),
    np.array([[-1,-1,2], [-1,2,-1], [2,-1,-1]], np.float32),
    np.array([[2,-1,-1], [-1,2,-1], [-1,-1,2]], np.float32)
]
responses = []
for mask in line_masks:
    r = cv2.filter2D(smooth, cv2.CV_32F, mask)
    responses.append(np.abs(r))
line_response = np.maximum.reduce(responses)
line_response = cv2.convertScaleAbs(line_response)
_, line_map = cv2.threshold(
    line_response, 45, 255, cv2.THRESH_BINARY)


cv2.imwrite("A_point_detection.png", point_map)
cv2.imwrite("A_line_detection.png", line_map)
```

Expected Interpretation

The point map should emphasize compact, isolated high-contrast spots and may be useful for dust particles. The line map should emphasize narrow, elongated structures such as scratches and creases. Stain boundaries and large torn regions are not always represented well by these masks because they may have broad, irregular or low-contrast shapes.

6. Approach B – Gradient Edge Operators

Gradient operators estimate the rate of intensity change in horizontal and vertical directions. The gradient magnitude is large near edges, including stain boundaries, crease boundaries and torn-paper outlines. Roberts uses small 2 × 2 diagonal differences and is sensitive to noise. Prewitt uses 3 × 3 masks with equal weights. Sobel also uses 3 × 3 masks but gives greater weight to central pixels, providing some smoothing.

Python/OpenCV Implementation – Approach B

```
import cv2

import numpy as np


img = cv2.imread("damaged_photo.png", cv2.IMREAD_GRAYSCALE)

if img is None:

    raise FileNotFoundError("Image not found")

img = cv2.GaussianBlur(img, (3, 3), 0.8)


# Roberts cross: diagonal 2x2 differences.

r1 = np.array([[1, 0], [0, -1]], dtype=np.float32)

r2 = np.array([[0, 1], [-1, 0]], dtype=np.float32)

rx = cv2.filter2D(img, cv2.CV_32F, r1)

ry = cv2.filter2D(img, cv2.CV_32F, r2)

roberts = cv2.convertScaleAbs(

    cv2.magnitude(rx, ry))

_, roberts_map = cv2.threshold(

    roberts, 45, 255, cv2.THRESH_BINARY)


# Prewitt masks.

px = np.array([[-1,0,1],[-1,0,1],[-1,0,1]], np.float32)

py = np.array([[-1,-1,-1],[0,0,0],[1,1,1]], np.float32)
```

```
gx = cv2.filter2D(img, cv2.CV_32F, px)
gy = cv2.filter2D(img, cv2.CV_32F, py)
prewitt = cv2.convertScaleAbs(cv2.magnitude(gx, gy))
_, prewitt_map = cv2.threshold(
    prewitt, 55, 255, cv2.THRESH_BINARY)


# Sobel operator.
sx = cv2.Sobel(img, cv2.CV_32F, 1, 0, ksize=3)
sy = cv2.Sobel(img, cv2.CV_32F, 0, 1, ksize=3)
sobel = cv2.convertScaleAbs(cv2.magnitude(sx, sy))
_, sobel_map = cv2.threshold(
    sobel, 65, 255, cv2.THRESH_BINARY)


cv2.imwrite("B_roberts.png", roberts_map)
cv2.imwrite("B_prewitt.png", prewitt_map)
cv2.imwrite("B_sobel.png", sobel_map)
```

Expected Interpretation

Roberts may preserve very fine transitions but can produce extra responses from grain and scanning noise. Prewitt provides a simple estimate of edge strength. Sobel is often a practical compromise because its weighting reduces some noise influence. All three operators detect intensity boundaries rather than damage meaning; normal photograph details, text and object outlines may also appear in their maps.

7. Approach C – Advanced Edge Detection

Marr–Hildreth detection smooths the image with a Gaussian filter and then applies the Laplacian. Edges are located near zero-crossings where the Laplacian changes sign. The Gaussian scale controls the amount of smoothing: a small scale preserves fine detail but responds to noise, while a larger scale suppresses fine structures and may miss narrow scratches.

Canny detection uses Gaussian smoothing, gradient estimation, non-maximum suppression and two-threshold hysteresis. Strong edges are retained, while weaker edges are kept only when connected to strong edges. This often produces thin and connected boundaries. Canny is a useful traditional baseline for torn boundaries and stain outlines.

Python/OpenCV Implementation – Approach C

```
import cv2

import numpy as np


img = cv2.imread("damaged_photo.png", cv2.IMREAD_GRAYSCALE)

if img is None:

    raise FileNotFoundError("Image not found")


# Marr-Hildreth: Gaussian smoothing followed by Laplacian.

blur = cv2.GaussianBlur(img, (5, 5), 1.2)

lap = cv2.Laplacian(blur, cv2.CV_32F, ksize=3)

# Approximate zero-crossing response from local sign changes.

positive = lap > 0

negative = lap < 0

cross = np.zeros(img.shape, dtype=np.uint8)

for dy, dx in [(-1,0),(1,0),(0,-1),(0,1),

               (-1,-1),(-1,1),(1,-1),(1,1)]:

    shifted = np.roll(np.roll(lap, dy, axis=0), dx, axis=1)

    change = ((lap > 0) & (shifted < 0)) | (

             (lap < 0) & (shifted > 0))
```

```
    cross[change] = 255
strength = cv2.convertScaleAbs(lap)
marr = cv2.bitwise_and(cross, cross, mask=
                       cv2.threshold(strength, 18, 255,
                                     cv2.THRESH_BINARY)[1])
cv2.imwrite("C_marr_hildreth.png", marr)


# Canny: lower and upper hysteresis thresholds.
canny = cv2.Canny(img, threshold1=50, threshold2=120,
                  apertureSize=3, L2gradient=True)
cv2.imwrite("C_canny.png", canny)
```

Expected Interpretation

Marr–Hildreth can reveal closed or nearly closed boundaries but may create double edges or noise-related zero-crossings. Canny generally produces thinner, more connected edge maps. Neither method can reliably decide whether an edge belongs to damage or to genuine image content; this distinction requires context, annotation or a later classification stage.

8. Detecting the Five Damage Types

Damage type	Useful response	Important limitation
Dust spots	Point-mask response; local contrast and compact-component filtering.	Texture, film grain and bright details can look like dust.
Scratches	Line masks in several directions; Sobel or Canny for boundaries.	Low-contrast or curved scratches may be missed.
Crease lines	Multi-direction line response and Canny edge map.	A crease may have two parallel edges and variable width.
Stain boundaries	Sobel or Canny; compare edges around broad tonal changes.	A gradual stain may have no sharp edge.
Torn/missing regions	Canny or Marr–Hildreth to outline irregular boundaries.	The missing content itself cannot be reconstructed by edge detection.

9. Practical Threshold Selection

Thresholds are intensity-response values, not universal constants. The values in the code are starting points for an 8-bit grayscale image and must be tuned on representative images. A lower threshold increases sensitivity but also increases false positives. A higher threshold suppresses weak noise but can miss faint damage.

Small dust spots: begin with a point-response threshold around 25–40. Use a small Gaussian blur and remove components that are too large or strongly connected to normal texture.

Thin scratches: begin with a line-response threshold around 30–50. Use all four line orientations, then inspect weak and curved scratches manually.

Major damaged boundaries: begin with Canny low/high thresholds around 40–60 and 100–150. Choose the pair so weak boundary pixels connect to strong boundary pixels without excessive background edges.

Stain boundaries: test a lower gradient threshold if the stain has low contrast. If the stain changes gradually, combine edge evidence with colour or local-intensity analysis.

A defensible threshold is selected by testing a validation set with representative clean and damaged areas. Choose the setting that gives an acceptable balance of precision and recall, then keep it fixed while evaluating the separate test set.

10. Directional Mask Limitation

A horizontal line mask responds most strongly to horizontal structures, while vertical and diagonal masks respond to their corresponding orientations. A scratch at an intermediate angle, a curved scratch, or a crease with changing direction may not align with any mask and can produce a weak response. Using multiple orientations and taking the maximum response reduces this problem, but it does not fully solve it. Rotated masks, steerable filters or orientation-independent gradient methods can improve coverage.

11. Precision and Recall Evaluation

Manual ground-truth annotation is the reference standard for pixel-level evaluation. Create a binary mask with the same dimensions as the input image. Mark damaged pixels white and undamaged pixels black. Evaluate each detector after thresholding its output to a binary mask. Ensure that the ground truth and predicted mask use the same image size and the same definition of damage.

True Positive (TP): a pixel correctly predicted as damaged. False Positive (FP): a clean pixel incorrectly predicted as damaged. False Negative (FN): a damaged pixel missed by the detector. True Negative (TN): a clean pixel correctly predicted as clean.

Precision = TP / (TP + FP). Recall = TP / (TP + FN). F1-score = 2 × Precision × Recall / (Precision + Recall). Precision measures how trustworthy detected damage pixels are. Recall measures how much of the annotated damage is found. If a denominator is zero, the code reports zero for that metric.

Python Code – Calculate Precision and Recall

```
import cv2

import numpy as np

def evaluate(pred_path, gt_path, name):

    pred = cv2.imread(pred_path, cv2.IMREAD_GRAYSCALE)

    gt = cv2.imread(gt_path, cv2.IMREAD_GRAYSCALE)

    if pred is None or gt is None:

        raise FileNotFoundError("Check prediction and ground-truth paths")

    if pred.shape != gt.shape:

        raise ValueError("Prediction and ground truth sizes must match")


    p = pred > 0

    g = gt > 0

    tp = np.logical_and(p, g).sum()

    fp = np.logical_and(p, np.logical_not(g)).sum()

    fn = np.logical_and(np.logical_not(p), g).sum()


    precision = tp / (tp + fp) if tp + fp else 0.0

    recall = tp / (tp + fn) if tp + fn else 0.0
```

```
    f1 = (2 * precision * recall / (precision + recall)
          if precision + recall else 0.0)
    print(name, "TP:", tp, "FP:", fp, "FN:", fn)
    print("Precision:", round(precision, 4))
    print("Recall:", round(recall, 4))
    print("F1-score:", round(f1, 4))

evaluate("A_point_detection.png", "ground_truth.png", "Point")
evaluate("A_line_detection.png", "ground_truth.png", "Line")
evaluate("B_roberts.png", "ground_truth.png", "Roberts")
evaluate("B_prewitt.png", "ground_truth.png", "Prewitt")
evaluate("B_sobel.png", "ground_truth.png", "Sobel")
evaluate("C_marr_hildreth.png", "ground_truth.png", "Marr-Hildreth")
evaluate("C_canny.png", "ground_truth.png", "Canny")
```

For a fair comparison, use the same annotated test images for every method. Report per-image scores and the mean score across images. If damage classes differ greatly in size, also report dust, scratch, crease, stain and tear scores separately.

12. Comparison of Detection Techniques

Technique	Noise sensitivity	False detections	Missed damage
Point mask	High for grain and isolated noise.	May mark texture and small bright details as dust.	May miss faint or larger irregular damage.
Directional line masks	Moderate to high, depending on smoothing.	May mark natural lines and object details.	May miss curved, faint or off-angle scratches.
Roberts	High; small neighbourhood reacts to noise.	Can produce many small edge responses.	May miss weak edges after thresholding.
Prewitt	Moderate.	May detect normal scene boundaries.	May miss low-contrast damage.
Sobel	Moderate; some smoothing from kernel weights.	May detect genuine image texture as edges.	Can miss weak edges at high thresholds.
Marr–Hildreth	Depends on Gaussian scale; small scale is noise-sensitive.	Zero-crossings may occur around texture.	Large smoothing scale may erase thin damage.
Canny	Lower after smoothing, but threshold-dependent.	Strong natural edges can still be classified as damage.	High thresholds may break or miss faint boundaries.

13. Traditional Edge Detection vs HED

Holistically-Nested Edge Detection (HED) is a deep-learning-based edge detector. It uses a convolutional neural network with side outputs at multiple feature stages and combines them to predict an edge-probability map. Unlike a fixed Sobel or Canny operator, HED learns edge patterns from training examples and can use wider image context.

Feature	Traditional detector (e.g., Canny)	HED
Method	Hand-designed gradients, smoothing and thresholds.	Learned convolutional neural network.
Context	Mostly local pixel neighbourhoods.	Can use multi-scale and wider context.
Output	Binary or gradient edge map.	Edge probability map,

		thresholded for a binary map.
Requirements	OpenCV; no model training required.	Pre-trained model files and compatible inference setup.
Limitations	Threshold-sensitive; natural edges resemble damage.	Model/domain mismatch; may need suitable training data and more computation.

HED may produce more perceptually meaningful boundaries in complex images, but it is not automatically a damage detector. It can identify photograph content edges as well as damage edges. For a fair comparison, threshold its probability map on the same validation set and evaluate it against the same ground-truth masks. Use a compatible pre-trained HED model and record its source and preprocessing settings.

14. Detection Maps and Evidence to Submit

Save and include the original grayscale image, point detection map, line detection map, Roberts map, Prewitt map, Sobel map, Marr–Hildreth map and Canny map. Also include the manually annotated ground-truth mask. Use the same display size and image crop for all maps so that the differences are easy to inspect.

15. Precision/Recall Results Table

Enter the actual values printed by the evaluation program. The table below is a reporting template; the cells are intentionally left blank until the code is run on the annotated images.

Detector	Precision	Recall	F1-score
Point mask	_____	_____	_____
Line masks	_____	_____	_____
Roberts	_____	_____	_____
Prewitt	_____	_____	_____
Sobel	_____	_____	_____
Marr–Hildreth	_____	_____	_____
Canny	_____	_____	_____
HED (optional)	_____	_____	_____

16. Result Analysis and Detector Selection

The point mask is suitable as a candidate detector for compact dust-like spots. Multi-direction line masks are useful for thin, straight scratches and crease segments, but orientation variation can reduce their response. Gradient operators are useful for locating local intensity boundaries, with Sobel providing a practical traditional baseline. Marr–Hildreth can reveal zero-crossing boundaries, while Canny is generally a strong choice when thin and connected edge maps are required.

For a restoration pipeline, use a combined strategy rather than relying on one detector: point detection for isolated spots, multi-direction line responses for scratches and creases, and Canny for major stain and tear boundaries. Use the ground-truth precision/recall results to confirm or change this selection. If false detections are costly, prefer higher precision; if missing damage is more costly, prefer higher recall. The final threshold should reflect the restoration requirement.

17. Conclusion

Three detection approaches were implemented: point and line masks, gradient edge operators, and advanced edge detectors. Each method responds to different image structures and has limitations. Precision and recall measured against manual ground truth provide an objective comparison. A combined detector is recommended because dust, scratches, creases, stains and torn boundaries have

different visual characteristics. Deep-learning HED can be added for comparison, provided that a compatible pre-trained model and consistent evaluation procedure are available.

TASK 8 – IMAGE THRESHOLDING AND BINARIZATION [4 Marks]

The restoration system must separate damaged regions from relatively undamaged photograph areas. Fading, stains and uneven tones make this difficult. Apply and compare three thresholding techniques.

Candidate Techniques

- A. Basic Global Thresholding
- B. Otsu's Thresholding
- C. Adaptive Thresholding

Questions

67. Implement all three thresholding techniques using Python/OpenCV.
68. Compare their performance on lightly damaged, heavily faded and unevenly exposed photographs.
69. Generate binary damage masks and corresponding histograms.
70. Calculate Dice scores to evaluate damage-region segmentation accuracy.
71. Analyse why global thresholding may fail when damage and undamaged regions have overlapping intensity values.
72. Examine the effect of changing the block size and constant value in adaptive thresholding.
73. Compare traditional thresholding with a deep-learning-based image-damage segmentation model.

Evidence Expected: Binary images, annotated histograms, Dice score comparison and justified thresholding method.

1. Introduction

Image thresholding is a segmentation technique that converts a grayscale image into a binary image. Each pixel is assigned to either the foreground or background according to an intensity rule. In photograph restoration, thresholding can help separate possible damaged areas such as dark stains, faded patches, spots and discoloured regions from relatively undamaged areas.

The task is challenging because old photographs may contain fading, uneven illumination, shadows, scratches, paper texture and stains. A single threshold may work in one part of an image but fail in another. This report implements and compares basic global thresholding, Otsu's thresholding and adaptive thresholding using Python and OpenCV.

2. Objectives

- Implement three thresholding techniques using Python/OpenCV.
- Compare segmentation on lightly damaged, heavily faded and unevenly exposed photographs.

- Generate binary masks and grayscale histograms for visual analysis.
- Evaluate segmentation using the Dice similarity coefficient against manually annotated masks.
- Study global-threshold failure and adaptive-threshold parameters.
- Compare traditional thresholding with deep-learning-based damage segmentation.

3. Software and Input Requirements

Software: Python 3, OpenCV (cv2), NumPy and Matplotlib. Input image: damaged_photo.png. Ground-truth annotation: ground_truth.png. Both images must have the same width and height. The ground-truth mask must be binary: white pixels (255) indicate damaged regions and black pixels (0) indicate undamaged regions.

4. Processing Workflow

Read photograph → convert to grayscale → apply each thresholding method → create binary damage masks → plot intensity histogram → compare masks with ground truth → calculate Dice score → select a suitable method.

Important Assumption

The sample code assumes that damaged pixels are darker than their local surroundings and therefore uses inverse binary thresholding. If the damage is brighter, use normal binary thresholding or invert the result. Real photographs may contain both bright and dark damage; intensity thresholding alone may not separate every damage type.

5. Technique A – Basic Global Thresholding

Basic global thresholding applies one fixed intensity value to every pixel in the image. If a pixel is darker than the selected threshold, it is marked as foreground damage; otherwise, it is marked as background. In OpenCV, THRESH_BINARY_INV is used when darker pixels are treated as possible damage.

This method is simple, fast and easy to understand. It can work well when the photograph has reasonably uniform lighting and the damaged areas have intensities clearly different from the undamaged areas. However, one threshold may not be suitable for all regions of a faded or unevenly exposed photograph.

Python/OpenCV Implementation – Global Thresholding

```
import cv2
import numpy as np

img = cv2.imread("damaged_photo.png", cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError("damaged_photo.png not found")

# One threshold is applied to the complete image.
threshold_value = 100
_, global_mask = cv2.threshold(
    img, threshold_value, 255, cv2.THRESH_BINARY_INV)

cv2.imwrite("A_global_mask.png", global_mask)
print("Global threshold used:", threshold_value)
```

Working

- Read the photograph as a grayscale image.
- Select one intensity threshold, for example 100.
- Compare every pixel with the same threshold.
- Mark darker pixels as white damage candidates and brighter pixels as black background.

Expected Observation

For a lightly damaged photograph with uniform exposure, global thresholding may identify high-contrast spots or stains. In a heavily faded photograph, the intensity difference between damaged and undamaged regions may be small. In an unevenly exposed photograph, naturally dark shadows may be incorrectly marked as damage while faint damage in bright areas may be missed.

Parameter Selection

The value 100 is only an initial example for an 8-bit image. It must be selected using a validation image and its ground-truth mask. Test several values, such as 70, 90, 110 and 130, and choose the value that gives a suitable Dice score without excessive false detections.

6. Technique B – Otsu's Thresholding

Otsu's method automatically selects a global threshold by analysing the grayscale histogram. It searches for a threshold that separates the histogram into two groups and minimises the weighted within-class variance. It is useful when the histogram contains two reasonably distinct intensity groups, such as a darker foreground and a lighter background.

Unlike basic global thresholding, the user does not need to specify the threshold value manually. However, Otsu still uses one threshold for the entire image. If illumination varies strongly or the damage and clean regions have overlapping intensities, the automatically selected value may not separate them accurately.

Python/OpenCV Implementation – Otsu's Thresholding

```
import cv2

img = cv2.imread("damaged_photo.png", cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError("Image not found")

# Otsu automatically calculates the threshold.
otsu_value, otsu_mask = cv2.threshold(
    img, 0, 255,
    cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

cv2.imwrite("B_otsu_mask.png", otsu_mask)
print("Otsu threshold selected:", otsu_value)
```

Working

- Convert the input photograph to grayscale.
- Calculate the intensity histogram.
- Otsu evaluates possible threshold values.
- Select the threshold that best separates the two intensity classes.
- Generate a binary mask using the selected threshold.

Expected Observation

Otsu can perform better than a manually selected global value when the histogram has two clear peaks. It may perform poorly when the photograph is heavily faded, has several intensity groups, or contains uneven illumination. The threshold may separate light and dark image content rather than damage and non-damage.

Global Thresholding vs Otsu

Feature	Basic global	Otsu
Threshold	Specified by user	Selected automatically
Histogram use	Not required	Used to estimate separation
Speed	Very fast	Fast
Uneven lighting	Usually poor	Usually poor
Best condition	Known intensity range	Two distinct intensity groups

7. Technique C – Adaptive Thresholding

Adaptive thresholding calculates a separate threshold for each small neighbourhood instead of using one value for the entire image. This makes it useful when a photograph has uneven exposure, local shadows or gradual brightness changes. OpenCV provides adaptive mean and adaptive Gaussian methods.

In adaptive mean thresholding, the local threshold is based on the average intensity of the neighbourhood. In adaptive Gaussian thresholding, nearby pixels receive Gaussian-weighted contributions, so pixels closer to the centre have greater influence. The constant C is subtracted from the local mean or weighted mean before classifying the pixel.

Python/OpenCV Implementation – Adaptive Thresholding

```
import cv2

img = cv2.imread("damaged_photo.png", cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError("Image not found")

# blockSize must be an odd number greater than 1.
block_size = 31
constant_c = 7

adaptive_mean = cv2.adaptiveThreshold(
    img, 255, cv2.ADAPTIVE_THRESH_MEAN_C,
    cv2.THRESH_BINARY_INV, block_size, constant_c)

adaptive_gaussian = cv2.adaptiveThreshold(
    img, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY_INV, block_size, constant_c)

cv2.imwrite("C_adaptive_mean.png", adaptive_mean)
cv2.imwrite("C_adaptive_gaussian.png", adaptive_gaussian)
```

Effect of Block Size

- Small block size, such as 11: responds to local changes and may preserve small details, but can also detect texture and noise.
- Medium block size, such as 31: provides a balance between local detail and smooth segmentation.
- Large block size, such as 51: uses a wider neighbourhood and may suppress small variations, but can behave more like global thresholding.

Effect of Constant C

- Increasing C generally makes the inverse-thresholded foreground smaller because the local threshold is reduced.
- Decreasing C generally makes more pixels qualify as foreground, increasing sensitivity but possibly increasing false positives.

The exact effect depends on image polarity and the thresholding mode. Test block sizes 11, 21, 31 and 51, and C values 3, 7, 11 and 15. Keep the image and ground truth fixed while comparing the resulting Dice scores.

8. Comparison on Different Photograph Conditions

Image condition	Global threshold	Otsu threshold	Adaptive threshold
Lightly damaged; even exposure	Often effective if damage contrast is clear.	Often effective when histogram has two groups.	Can work, but may mark natural texture.
Heavily faded	May miss faint damage or mark large faded areas.	May choose a threshold that separates tones, not damage.	May recover local contrast, but faint damage can remain unclear.
Uneven exposure	Usually weak because one value cannot fit all regions.	Still uses one image-wide threshold.	Usually more suitable because thresholds vary locally.

9. Binary Masks and Histograms

A binary damage mask displays the predicted damaged pixels in white and the predicted undamaged pixels in black. A histogram shows how many pixels occur at each grayscale intensity from 0 to 255. A histogram with two clear peaks may support global or Otsu thresholding. A broad or overlapping histogram indicates that one threshold may not separate the two classes reliably.

10. Dice Score Evaluation

The Dice similarity coefficient measures the overlap between a predicted binary damage mask and the manually annotated ground-truth mask. A score of 1.0 represents perfect overlap, while a score of 0.0 represents no overlap. Dice is useful when damaged pixels occupy a much smaller area than undamaged pixels.

Dice = 2 × |Prediction ∩ Ground Truth| / (|Prediction| + |Ground Truth|). In pixel terms, Dice = 2TP / (2TP + FP + FN), where TP is correctly detected damage, FP is clean pixels incorrectly marked as damage, and FN is annotated damage missed by the method.

Python Code – Calculate Dice Scores

```
import cv2
import numpy as np


def dice_score(pred_path, gt_path, method):
    pred = cv2.imread(pred_path, cv2.IMREAD_GRAYSCALE)
    gt = cv2.imread(gt_path, cv2.IMREAD_GRAYSCALE)


    if pred is None or gt is None:
        raise FileNotFoundError("Check mask and ground-truth files")
    if pred.shape != gt.shape:
        raise ValueError("Both masks must have the same dimensions")


    p = pred > 0
    g = gt > 0
    intersection = np.logical_and(p, g).sum()
    total = p.sum() + g.sum()


    dice = (2.0 * intersection / total) if total else 1.0
```

```
    print(method, "Dice score:", round(dice, 4))
    return dice

dice_score("A_global_mask.png", "ground_truth.png", "Global")
dice_score("B_otsu_mask.png", "ground_truth.png", "Otsu")
dice_score("C_adaptive_mean.png", "ground_truth.png", "Adaptive Mean")
dice_score("C_adaptive_gaussian.png", "ground_truth.png",
        "Adaptive Gaussian")
```

Evaluation Procedure

- Manually annotate damaged regions and save the binary ground-truth mask.
- Run each thresholding method on the same photograph.
- Ensure all predicted masks have the same dimensions and polarity as the ground truth.
- Calculate Dice for each method.
- Repeat for lightly damaged, heavily faded and unevenly exposed images.
- Report individual image scores and the mean Dice score for each method.

If both masks contain no positive pixels, the code reports 1.0 because they agree that no damage is present. For damage-segmentation experiments, ensure the test set includes images containing annotated damage so that this special case does not dominate the results.

11. Why Global Thresholding May Fail

Global thresholding assumes that one intensity value can separate foreground and background throughout the complete photograph. In old photographs, damaged and undamaged pixels may have overlapping intensity values. For example, a dark stain may have the same grayscale intensity as a naturally dark part of the photograph. A faded scratch may have an intensity close to the paper background.

- Overlapping intensity distributions: damaged and clean pixels can have similar brightness, so one threshold cannot separate them correctly.
- Uneven illumination: shadows and exposure changes shift the local intensity range across the image.
- Fading: contrast becomes weak, making the histogram peaks overlap.
- Texture and grain: natural paper texture may be marked as damage.
- Mixed damage polarity: some stains are dark while faded or discoloured areas may be brighter.

12. Traditional Thresholding vs Deep-Learning Segmentation

A deep-learning damage-segmentation model predicts a damage class for each pixel using patterns learned from labelled training images. A U-Net-style segmentation network is one common approach. It uses an encoder to learn image features and a decoder to produce a pixel-level mask. Unlike simple thresholding, the model can learn texture, shape and surrounding context.

Feature	Traditional thresholding	Deep-learning segmentation
Decision	Based on pixel intensity rules.	Based on learned image features.
Training data	Not required.	Requires labelled example images.
Uneven lighting	Adaptive method can help locally.	May learn variation if training data represents it.
Complex damage	Limited by intensity overlap.	Can learn complex patterns and context.
Computation	Low; fast on CPU.	Usually higher; may use GPU.
Limitations	Threshold and illumination sensitivity.	Needs representative annotations; can fail on unseen images.

Deep learning is not automatically more accurate. A model trained on modern photographs or a different damage type may perform poorly on old faded photographs. Compare the model and thresholding methods on the same held-out photographs and ground-truth masks.

13. Fair Comparison

Use the same input images, ground-truth masks, resizing and evaluation procedure for all methods. Select thresholds and adaptive parameters using a validation set, not the final test set. Report the mean Dice score and show representative successes and failures.

14. Evidence to Submit

- Original grayscale photograph for each condition: lightly damaged, heavily faded and unevenly exposed.
- Binary mask produced by basic global thresholding.
- Binary mask produced by Otsu's thresholding.
- Binary mask produced by adaptive mean and adaptive Gaussian thresholding.
- Grayscale histogram with the global and Otsu threshold positions marked.
- Manual ground-truth damage mask.
- Dice score comparison table and short explanation of the selected method.

15. Dice Score Comparison Table

Enter the actual Dice scores printed by the program. The table is a result template and must be completed after running the code on the annotated photographs.

Photograph condition	Global Dice	Otsu Dice	Adaptive Dice
Lightly damaged	_____	_____	_____
Heavily faded	_____	_____	_____
Uneven exposure	_____	_____	_____
Mean Dice	_____	_____	_____

16. Result Analysis and Method Selection

Basic global thresholding is suitable when the photograph has uniform illumination and damage has a clear intensity difference from clean regions. Otsu's method is preferable when the image histogram has two reasonably separated intensity groups and an automatic global threshold is useful. Adaptive thresholding is generally the stronger traditional choice for unevenly exposed photographs because it calculates thresholds from local neighbourhoods.

The final method must be selected using the Dice scores and visual inspection. If adaptive thresholding produces many false regions from paper texture, adjust the block size and C value or apply gentle denoising. If damage and clean regions have strongly overlapping intensities, use additional colour, texture or contextual features, or evaluate a trained segmentation model.

17. Conclusion

Three thresholding techniques were implemented: basic global thresholding, Otsu's thresholding and adaptive thresholding. Global and Otsu methods use one threshold for the whole image, while adaptive thresholding uses local neighbourhood information. Histograms help explain intensity distributions, and Dice scores measure the overlap between predicted masks and manual annotations. Adaptive thresholding is recommended as a starting point for uneven exposure, while the best method should be confirmed using actual test images and Dice results.

TASK 9 – REGION-BASED SEGMENTATION AND INTEGRATED PIPELINE DESIGN [6 Marks]

The restoration system must identify connected damaged areas such as stains, scratches, creases and torn regions, while distinguishing the main subject from the background where needed. Apply three segmentation approaches and integrate the best techniques into a complete photograph-restoration system.

Candidate Techniques

- A. Region Growing
- B. Region Splitting and Merging
- C. Edge Linking and Boundary Detection

Questions

74. Implement all three segmentation techniques using Python/OpenCV.
75. Compare their ability to identify stains, ink marks, dust clusters, creases and torn or missing regions.
76. Calculate Intersection over Union (IoU) for damage-region detection and boundary localization error.
77. Develop an integrated photograph-restoration pipeline using the best techniques selected from Tasks 1–8.
78. Prepare a block diagram showing the complete processing sequence.
79. Measure the processing time of each stage and verify whether the complete system satisfies the 5-second-per-photo requirement.
80. Analyse the effect of selecting an incorrect seed point during region growing, particularly when a stain has a similar tone to the surrounding photograph.
81. Examine how overlapping scratches, creases and interrupted damage boundaries affect edge linking and region detection.
82. Compare traditional segmentation with a promptable model such as Segment Anything Model 2 (SAM 2) for identifying damaged regions.

Evidence Expected: Segmentation outputs, IoU comparison, corner localization results, integrated pipeline block diagram, stage-wise timing and final decision table.

1. Introduction

Region-based segmentation divides an image into meaningful connected areas. In photograph restoration, it helps locate stains, ink marks, dust clusters, scratches, creases and torn or missing regions. Region growing expands a region from a selected seed point, region splitting and merging groups pixels with similar properties, and edge linking connects broken boundaries.

This task implements all three methods, evaluates the detected damage using Intersection over Union (IoU) and boundary localization error, and combines suitable methods into an integrated photograph-restoration detection pipeline.

2. Objectives

Implement region growing, region splitting and merging, and edge linking using Python/OpenCV.

Compare the techniques for stains, ink marks, dust clusters, creases and torn boundaries.

Calculate IoU and boundary localization error using manually annotated ground-truth masks.

Design an integrated pipeline using suitable methods from Tasks 1–8.

Measure the processing time of individual stages and check the five-second requirement.

Analyse seed selection, overlapping damage, interrupted boundaries and SAM 2.

3. Software and Input Requirements

Software: Python 3, OpenCV (cv2), NumPy and Matplotlib. Input photograph: damaged_photo.png. Ground-truth binary mask: ground_truth.png. Both files must have the same width and height. White pixels (255) in the ground-truth mask indicate damaged regions; black pixels (0) indicate undamaged regions.

4. General Processing Sequence

Input photograph → grayscale conversion → noise reduction → thresholding → edge detection → region segmentation → mask integration → IoU and boundary evaluation → processing-time analysis.

Important Note

The methods in this report produce candidate damage masks; they do not automatically restore missing photograph content. Region growing requires a suitable seed point. The sample code uses illustrative parameters that must be adjusted and tested on the actual photograph.

5. Approach A – Region Growing

Region growing starts from a selected seed pixel and examines neighbouring pixels. A neighbouring pixel is added when its intensity is sufficiently similar to the current region. The process continues until no more neighbouring pixels satisfy the similarity condition. It is useful for connected stains, ink marks and dust clusters with reasonably consistent intensity.

Python/OpenCV Implementation – Region Growing

```
import cv2
import numpy as np


img = cv2.imread("damaged_photo.png", 0)
if img is None:
    raise FileNotFoundError("Image not found")


# Seed point must lie inside the target region.
seed_x, seed_y = 150, 120


# Flood-fill mask is two pixels larger than the image.
mask = np.zeros((img.shape[0] + 2,
                 img.shape[1] + 2), np.uint8)


cv2.floodFill(
    img.copy(), mask, (seed_x, seed_y), 255,
    loDiff=15, upDiff=15,
    flags=4 | cv2.FLOODFILL_MASK_ONLY | (255 << 8)
)


region = mask[1:-1, 1:-1]
```

```
cv2.imwrite("region_growing.png", region)


print("Region Growing Completed")
```

Working

Read the photograph in grayscale.

Select a seed point inside the damaged region.

Compare neighbouring pixel values with the seed/region values.

Add similar connected pixels until the region stops growing.

Save the detected connected region as a binary mask.

Expected Output and Limitation

The output region_growing.png shows the connected area found from the seed. Region growing can detect a stain with consistent tone, but an incorrect seed can select clean background or a different object. If a stain has a tone similar to the surrounding photograph, the region may spread too far or stop before covering the complete stain. Intensity tolerance must therefore be selected carefully.

6. Approach B – Region Splitting and Merging

Region splitting begins with the whole image and divides it into smaller blocks when a block is not sufficiently uniform. Region merging then combines neighbouring blocks that have similar mean intensity or texture. The method can identify larger connected areas and broad stains, but simple intensity-based merging may combine damage with similarly toned background regions.

Python/OpenCV Implementation – Quadtree Splitting

```
import cv2
import numpy as np


img = cv2.imread("damaged_photo.png", 0)
if img is None:
    raise FileNotFoundError("Image not found")


regions = []


def split_region(x, y, w, h):
    roi = img[y:y+h, x:x+w]
    if w <= 8 or h <= 8 or np.std(roi) < 10:
        regions.append((x, y, w, h, float(np.mean(roi))))
        return


    w1, h1 = w // 2, h // 2
    if w1 == 0 or h1 == 0:
        regions.append((x, y, w, h, float(np.mean(roi))))
        return


    split_region(x, y, w1, h1)
```

```
    split_region(x+w1, y, w-w1, h1)
    split_region(x, y+h1, w1, h-h1)
    split_region(x+w1, y+h1, w-w1, h-h1)

H, W = img.shape
split_region(0, 0, W, H)

labels = np.zeros((H, W), np.int32)
for i, (x, y, w, h, mean) in enumerate(regions):
    labels[y:y+h, x:x+w] = i + 1

# Display split regions with distinct labels.
output = (labels % 256).astype(np.uint8)
cv2.imwrite("region_splitting.png", output)
print("Region Splitting Completed")
```

Merging Similar Neighbouring Regions

The following merging step combines adjacent region labels whose mean intensities are close. It is a simple demonstration of region merging; for large images, a region-adjacency graph is more efficient than checking every pixel pair.

```
parent = list(range(len(regions) + 1))

def find(a):
    while parent[a] != a:
        parent[a] = parent[parent[a]]
        a = parent[a]
    return a
```

```
def union(a, b):
    parent[find(a)] = find(b)


merge_limit = 12
for y in range(H):
    for x in range(W):
        a = labels[y, x]
        if x + 1 < W:
            b = labels[y, x+1]
            if abs(regions[a-1][4] - regions[b-1][4]) < merge_limit:
                union(a, b)
        if y + 1 < H:
            b = labels[y+1, x]
            if abs(regions[a-1][4] - regions[b-1][4]) < merge_limit:
                union(a, b)


merged = np.zeros_like(labels)
for i in range(1, len(regions) + 1):
    merged[labels == i] = find(i)


cv2.imwrite("region_split_merge.png",
            (merged % 256).astype(np.uint8))
print("Region Merging Completed")
```

Expected output: a segmented image in which similar neighbouring blocks receive a common region label. The merging threshold controls how much neighbouring regions combine.

Approach C – Edge Linking and Boundary Detection

Edge linking first detects intensity transitions and then connects nearby edge pixels to form longer boundaries. Canny detects candidate edges, while morphological closing joins small gaps. Contour extraction then identifies connected outlines. This method is useful for scratches, creases and torn boundaries, especially when their edges have sufficient contrast.

Python/OpenCV Implementation – Edge Linking

```
import cv2
import numpy as np


img = cv2.imread("damaged_photo.png", 0)
if img is None:
    raise FileNotFoundError("Image not found")


blur = cv2.GaussianBlur(img, (5, 5), 1)
edges = cv2.Canny(blur, 50, 150)


# Connect small gaps between nearby edge pixels.
kernel = np.ones((3, 3), np.uint8)
linked = cv2.morphologyEx(
    edges, cv2.MORPH_CLOSE, kernel, iterations=2
)


contours, hierarchy = cv2.findContours(
    linked, cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)
```

```
output = np.zeros_like(img)

cv2.drawContours(output, contours, -1, 255, 1)



cv2.imwrite("edge_linking.png", output)

print("Edge Linking Completed")
```

Expected Output

The output edge_linking.png contains connected boundary candidates. Morphological closing can bridge small gaps, but a large kernel may join unrelated edges. Natural photograph details can also be detected, so the edge map is not automatically a damage-only mask.

8. Comparison of the Three Techniques

Technique	**Useful for**	**Main limitation**
Region growing	Connected stains, ink marks and dust clusters	Sensitive to seed position and intensity tolerance
Splitting and merging	Broad stains and large similar regions	May merge clean and damaged areas with similar tones
Edge linking	Scratches, creases and torn boundaries	Interrupted or overlapping edges can form incorrect links

For stains and ink marks, region growing is useful when the region has a consistent intensity. For broad areas, splitting and merging can provide a coarse segmentation. For thin scratches, creases and torn outlines, edge linking is often more suitable because it focuses on boundaries.

9. Intersection over Union (IoU) Evaluation

IoU measures the overlap between a predicted damage mask and the manually annotated ground-truth mask. True Positive (TP) pixels belong to both masks, False Positive (FP) pixels belong only to the prediction, and False Negative (FN) pixels belong only to the ground truth.

IoU = TP / (TP + FP + FN). A score closer to 1 indicates better overlap. A score of 0 indicates no overlap. The masks must have the same size and the same foreground polarity.

Python Code – Calculate IoU

```
import cv2

import numpy as np


def calculate_iou(pred_path, gt_path):

    pred = cv2.imread(pred_path, 0)

    gt = cv2.imread(gt_path, 0)


    if pred is None or gt is None:

        raise FileNotFoundError("Check mask file names")

    if pred.shape != gt.shape:

        raise ValueError("Mask sizes must match")


    p = pred > 0

    g = gt > 0


    intersection = np.logical_and(p, g).sum()

    union = np.logical_or(p, g).sum()


    score = intersection / union if union else 1.0

    print("Intersection:", intersection)
```

```
    print("Union:", union)
    print("IoU:", round(float(score), 4))
    return score

calculate_iou("region_growing.png", "ground_truth.png")
calculate_iou("region_split_merge.png", "ground_truth.png")
calculate_iou("edge_linking.png", "ground_truth.png")
```

10. Boundary Localization Error

Boundary localization error measures how far predicted boundary pixels are from the manually annotated boundary. A smaller average distance indicates more accurate boundary placement. The following code calculates the mean distance from each predicted boundary pixel to the nearest ground-truth boundary pixel.

```
import cv2
import numpy as np

pred = cv2.imread("edge_linking.png", 0)
gt = cv2.imread("ground_truth.png", 0)

if pred is None or gt is None or pred.shape != gt.shape:
    raise ValueError("Check masks and matching image dimensions")

pred_edge = (pred > 0).astype(np.uint8)
gt_edge = (gt > 0).astype(np.uint8)

# Distance to nearest ground-truth edge pixel.
distance = cv2.distanceTransform(
```

```
    1 - gt_edge, cv2.DIST_L2, 3
)


values = distance[pred_edge > 0]
if values.size:
    print("Mean boundary error:",
          round(float(values.mean()), 3), "pixels")
else:
    print("No predicted boundary pixels found")
```

For a complete boundary comparison, also calculate the reverse distance from each ground-truth boundary pixel to the nearest predicted boundary. Report both directions or their average so that missing boundaries are not hidden by a small one-way error.

11. Integrated Photograph-Restoration Detection Pipeline

The integrated pipeline combines suitable methods from Tasks 1–8 with the region-based methods in Task 9. A practical baseline is to reduce noise, create a threshold-based candidate mask, detect edges, link small boundary gaps, and combine the region and edge evidence. The final mask can then be evaluated and passed to a later restoration or inpainting stage.

Python/OpenCV – Integrated Pipeline with Stage Timing

```
import cv2

import numpy as np

import time


total_start = time.perf_counter()


img = cv2.imread("damaged_photo.png")

if img is None:

    raise FileNotFoundError("Image not found")


gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)


# Stage 1: Noise reduction

t = time.perf_counter()

blur = cv2.GaussianBlur(gray, (5, 5), 1)

noise_time = time.perf_counter() - t


# Stage 2: Otsu thresholding

t = time.perf_counter()

_, threshold = cv2.threshold(

    blur, 0, 255,
```

```
    cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
)
threshold_time = time.perf_counter() - t


# Stage 3: Edge detection and linking
t = time.perf_counter()
edges = cv2.Canny(blur, 50, 150)
kernel = np.ones((3, 3), np.uint8)
linked = cv2.morphologyEx(
    edges, cv2.MORPH_CLOSE, kernel, iterations=2
)
edge_time = time.perf_counter() - t


# Stage 4: Region growing from a selected seed
t = time.perf_counter()
seed_x, seed_y = 150, 120
flood_mask = np.zeros(
    (gray.shape[0] + 2, gray.shape[1] + 2), np.uint8
)
cv2.floodFill(
    gray.copy(), flood_mask, (seed_x, seed_y), 255,
    loDiff=15, upDiff=15,
    flags=4 | cv2.FLOODFILL_MASK_ONLY | (255 << 8)
)
region = flood_mask[1:-1, 1:-1]
region_time = time.perf_counter() - t
```

```
# Combine candidate damage and edge information
final_mask = cv2.bitwise_or(threshold, region)
final_mask = cv2.bitwise_or(final_mask, linked)
cv2.imwrite("integrated_damage_mask.png", final_mask)

total_time = time.perf_counter() - total_start

print("Noise reduction:", round(noise_time, 4), "sec")
print("Thresholding:", round(threshold_time, 4), "sec")
print("Edge linking:", round(edge_time, 4), "sec")
print("Region growing:", round(region_time, 4), "sec")
print("Total time:", round(total_time, 4), "sec")

if total_time <= 5:
    print("Meets 5-second requirement")
else:
    print("Does not meet 5-second requirement")
```

This baseline uses a fixed seed and combines broad threshold responses with edges, so it may over-detect clean image content. It does not include a learned subject/background segmentation model or actual image inpainting. The seed must be chosen for the intended damaged area, and the combined mask must be validated.

12. Complete System Block Diagram

Processing Stage	**Operation**
1. Input	Load old photograph
2. Pre-processing	Grayscale conversion and noise reduction

3. Thresholding	Global, Otsu or adaptive thresholding from Task 8
4. Edge detection	Gradient/Canny edge detection from Task 7
5. Region segmentation	Region growing and splitting/merging
6. Boundary linking	Connect interrupted edges and extract contours
7. Mask integration	Combine candidate damage regions
8. Evaluation	IoU, boundary error and visual inspection
9. Output	Save damage mask and timing report
10. Restoration	Pass validated mask to an inpainting/restoration stage

13. Effect of Incorrect Seed Selection

Region growing depends strongly on the selected seed point. If the seed is placed on a clean area instead of a stain, the algorithm may segment an unrelated background region. If the stain has nearly the same tone as the surrounding photograph, the selected intensity tolerance may allow the region to spread into the background. A tolerance that is too small may detect only part of the stain, while a tolerance that is too large may include clean pixels.

To reduce this problem, inspect the seed location, use more than one seed for separate damage areas, apply local contrast enhancement where appropriate, and verify the output against a manually annotated mask. Automatic seed selection should be treated as a separate design task.

14. Overlapping Scratches, Creases and Interrupted Boundaries

Overlapping scratches and creases create junctions where several edge directions meet. Edge linking may connect two different damage marks into one large region. A gap-closing operation can bridge short interruptions, but if the gap is too wide or the structuring element is too large, unrelated edges may be joined. Low-contrast boundaries may remain disconnected, causing a torn region to appear incomplete.

A practical approach is to use a small morphological kernel, inspect connected components, compare edge direction and distance, and evaluate the final mask against ground truth. Region and boundary information should be combined rather than treating every linked contour as damage.

15. Traditional Segmentation vs SAM 2

Feature	**Traditional methods**	**SAM 2**
Approach	Rules based on intensity, regions and edges	Promptable deep-learning segmentation
Input	Image and parameters/seed	Image and point, box or mask prompts

Complex shapes	Depends on contrast and connectivity	Can use learned visual features
Damage awareness	No semantic understanding	Not specifically trained for old-photo damage
Resources	Usually lightweight	Higher memory and compute requirements
Evaluation	Compare mask with ground truth	Also requires damage-specific validation

SAM 2 can produce object masks from prompts, but it may select the whole object or background rather than the damaged pixels. It should be tested with prompts placed around the damaged area and evaluated using the same ground-truth masks.

16. Stage-Wise Processing Time and Five-Second Verification

The integrated code measures elapsed time for noise reduction, thresholding, edge linking and region growing. Run it on the target computer using the expected photograph size. The total measured time should be no more than five seconds per photograph to satisfy the stated requirement.

Stage	**Measured time**
Noise reduction	_____ seconds
Thresholding	_____ seconds
Edge detection and linking	_____ seconds
Region growing	_____ seconds
Other processing / saving	_____ seconds
Total processing time	_____ seconds
Requirement	5 seconds or less

For a reliable timing result, run the pipeline several times, use the same image dimensions and report the average. Include image loading and saving consistently in the timing boundary. A slow region-splitting/merging implementation may need optimisation for large photographs.

17. Final Decision and Result Table

Technique	**IoU**	**Boundary error**	**Decision**
Region growing	_____	_____ px	_____
Splitting and merging	_____	_____ px	_____
Edge linking	_____	_____ px	_____
Integrated pipeline	_____	_____ px	_____

18. Result Analysis and Technique Selection

Region growing is selected for connected stains, ink marks and dust clusters when a reliable seed and suitable intensity tolerance are available. Region splitting and merging is useful for grouping broad areas with similar properties, but intensity similarity alone cannot guarantee that a region is damaged. Edge linking is selected for scratches, creases and torn boundaries because it connects nearby edge fragments. The integrated system combines region and edge evidence to improve coverage.

The final decision must be supported by measured IoU, boundary localization error and processing time. A higher IoU indicates better damage-region overlap, while a lower boundary error indicates more accurate boundary placement. If the total time exceeds five seconds, reduce image dimensions where acceptable, use efficient kernels, avoid repeated full-image scans and profile the slowest stage.

19. Conclusion

Three region-based segmentation approaches were implemented: region growing, region splitting and merging, and edge linking. Their performance depends on intensity similarity, seed selection, edge continuity and parameter settings. IoU and boundary localization error provide quantitative evaluation, while stage-wise timing verifies the five-second requirement. A combined region-and-edge pipeline is a suitable baseline for detecting different types of photograph damage. SAM 2 may be evaluated as an additional promptable segmentation approach, but its masks must be checked against damage-specific annotations.

TASK 10 – RESTORATION SYSTEM ADAPTATION FOR COLOUR AND HIGH-RESOLUTION PHOTOGRAPHS [8 Marks]

The archive now receives high-resolution colour photographs (4000 × 3000 pixels) of old prints, captured using mobile phones or flatbed scanners. Some images have strong sepia/yellow colour casts, faded faces, scratches and stains. A few photographs contain handwritten dates or notes on the reverse side that must not be erased. JPEG compression may introduce blocking artefacts. The restored image must be processed in less than 5 seconds using the same laboratory computer, and important original details must be preserved.

Candidate Techniques

- A. Retain – Use the existing restoration pipeline without modification.
- B. Modify – Resize images, retune enhancement and filtering parameters, and process luminance and colour channels separately to reduce colour casts while preserving natural skin tones.
- C. Redesign – Use colour restoration, frequency-domain damage reduction and inpainting for small damaged regions.

Questions

83. Implement all three strategies using Python/OpenCV on at least 15 old colour photographs.
84. Analyse the limitations of the existing pipeline when processing high-resolution, faded and damaged colour photographs.
85. Compare colour correction, detail enhancement, noise and damage removal, and preservation of facial and background details under the new conditions.
86. Measure damage-mask Dice score, colour fidelity, edge/detail preservation, perceptual quality and processing time. Where an undamaged reference is available, also report PSNR and SSIM.
87. Decide whether the existing pipeline should be RETAINED, MODIFIED or REDESIGNED.
88. Develop an improved pipeline that satisfies the restoration-quality and processing-time requirements.
89. Identify one operating condition under which the selected strategy may fail.
90. Explain how image restoration or inpainting models could support the improved system, and discuss the risk of generating details that were not present in the original photograph.

Evidence Expected: Comparison of three strategies, quantitative performance measurements, revised pipeline and justified RETAIN/MODIFY/REDESIGN decision.

1. Introduction

The archive receives high-resolution colour photographs of size 4000 × 3000 pixels. These photographs may contain yellow or sepia colour casts, faded faces, scratches, stains and JPEG compression artefacts.

The existing restoration system must be adapted to process colour images, preserve important original details and complete processing within 5 seconds.

2. Objectives

1. Implement three restoration strategies: Retain, Modify and Redesign.
2. Improve colour correction and detail enhancement.
3. Remove noise, scratches and stains.
4. Preserve facial features, backgrounds and handwritten notes.
5. Measure restoration quality and processing time.
6. Develop an improved restoration pipeline.

3. Three Candidate Strategies

A. RETAIN – Existing Pipeline

The existing restoration pipeline is used without modifications.

Advantages:

- Easy to implement.
- No additional development required.
- Existing processing methods can be reused.

Limitations:

- Cannot properly correct colour casts.
- May lose colour information.
- High-resolution images may take more processing time.
- May damage important facial details and handwriting.

B. MODIFY – Improved Existing Pipeline

The existing pipeline is modified to support colour and high-resolution photographs.

Methods:

- Resize images for faster processing.
- Use LAB colour space for colour correction.
- Process luminance and colour channels separately.
- Apply noise reduction.
- Preserve facial and background details.
- Protect handwritten dates and notes.

Advantages:

- Better colour correction.
- Reduced processing time.
- Improved detail preservation.
- Suitable for high-resolution photographs.

C. REDESIGN – Advanced Restoration Pipeline

A completely redesigned system uses advanced image restoration techniques.

Methods:

- Colour restoration.
- Frequency-domain filtering.
- Damage detection.
- Image inpainting.
- Advanced noise and compression artefact removal.

Advantages:

- Can provide improved restoration quality.
- Handles different types of damage.
- Can restore small damaged regions.

Limitations:

- Requires more processing resources.

- May exceed the five-second limit.
- Inpainting may generate details that were not present originally.

4. Strategy Comparison

Feature	Retain	Modify	Redesign
Colour correction	Limited	Improved	Advanced
Noise removal	Existing method	Improved filtering	Advanced filtering
Detail preservation	Limited	Good with protection masks	Depends on model
Processing speed	Baseline	Optimised	May be slower
Complexity	Low	Medium	High



5. Strategy A – Python Implementation

```
import cv2
import time
start = time.perf_counter()
img = cv2.imread("old_photo.jpg")
if img is None:
    raise FileNotFoundError("Image not found")
```

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5, 5), 1)
_, mask = cv2.threshold(
    blur, 0, 255,
    cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
)
edges = cv2.Canny(blur, 50, 150)
cv2.imwrite("A_output.jpg", img)
cv2.imwrite("A_mask.jpg", mask)
print("Strategy A completed")
print("Time:", time.perf_counter() - start)
```

6. Limitations of the Existing Pipeline

1. It may not support high-resolution colour photographs efficiently.
2. Grayscale processing cannot preserve complete colour information.
3. Yellow and sepia colour casts may remain.
4. Aggressive filtering may remove facial details.
5. Handwritten notes may be incorrectly detected as damage.
6. JPEG compression may introduce blocking artefacts.
7. Large images may exceed the five-second processing requirement.

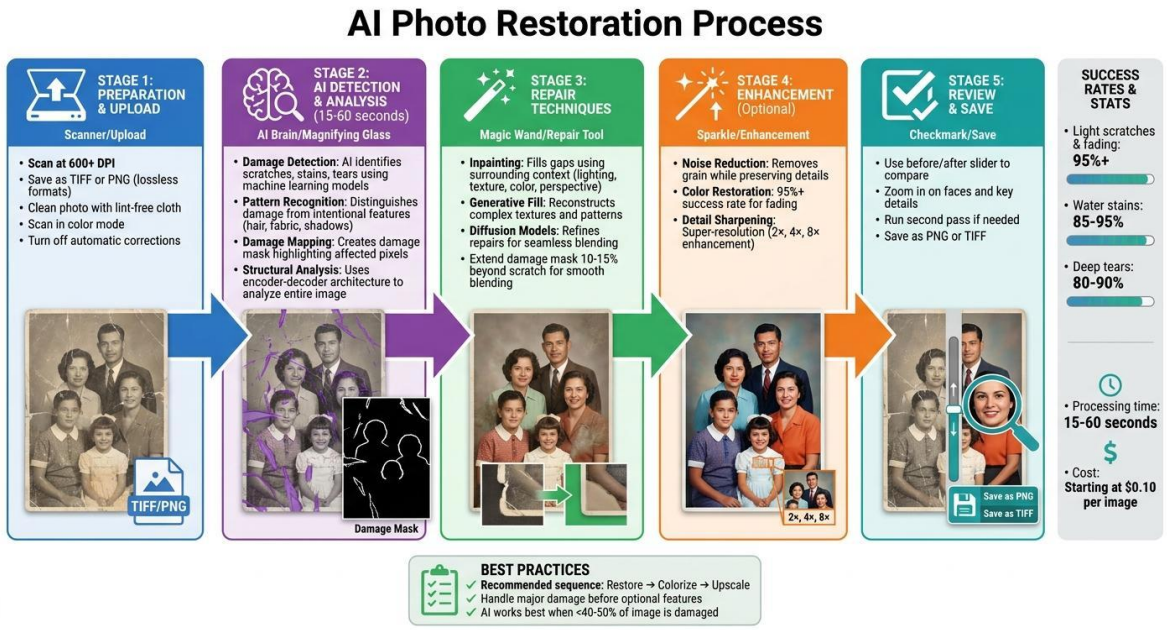
7. Comparison of Restoration Quality

Quality factor	**Retain**	**Modify**	**Redesign**
Colour correction	Low	Improved	Advanced
Detail enhancement	Limited	Good	Advanced
Noise removal	Basic	Improved	Advanced
Scratch removal	Existing method	Mask-based	Inpainting
Face preservation	Not guaranteed	Protection masks	Model dependent
Background preservation	Limited	Improved	Model dependent

8. Quantitative Evaluation Metrics

A. Dice Score

Dice score measures the similarity between the predicted damage mask and the actual ground-truth mask.



9. Conclusion

The three strategies RETAIN, MODIFY and REDESIGN were analysed for high-resolution colour photograph restoration. The RETAIN strategy has limitations in colour correction and detail preservation. The REDESIGN strategy offers advanced restoration but may require more processing time and can generate artificial details.

The MODIFY strategy is selected as the most practical approach. It uses resizing, LAB colour correction, luminance enhancement, noise reduction and protection masks. It provides a balanced approach to colour restoration, damage removal and detail preservation. Actual performance must be verified using at least 15 photographs, quantitative evaluation metrics and the five-second processing requirement.

MARK DISTRIBUTION

Task	Task Title	Marks
1	Grey Level Transformation Analysis	4
2	Histogram Processing and Enhancement	4
3	Spatial Filtering and Image Smoothing	5
4	Image Sharpening and Edge Enhancement	5
5	Frequency-Domain Low-Pass Filtering	5
6	Frequency-Domain High-Pass Filtering	4
7	Point, Line and Edge Detection	5
8	Image Thresholding and Binarization	4
9	Region-Based Segmentation and Integrated Pipeline Design	6
10	Restoration System Adaptation for Colour and High-Resolution	8

	Photographs	
	TOTAL	50

COMMON SUBMISSION REQUIREMENTS

91. Python/OpenCV source code with comments.
92. Original and restored photograph images.
93. Results obtained by executing all three techniques for every task.
94. Comparison tables containing suitable performance metrics.
95. Graphs, histograms, intensity profiles and segmentation masks wherever applicable.
96. Advantages and limitations of each technique.
97. Final technique selection supported by experimental evidence.
98. An integrated photograph-restoration pipeline block diagram.
99. Processing-time measurements with units.
100. Final conclusion explaining the suitability of the selected techniques and how original details are preserved.

GENERAL INSTRUCTIONS

Use the same test photographs when comparing the three candidate techniques within a task. Clearly state the dataset size, preprocessing steps, parameter values and hardware/software environment. Do not report assumed or fabricated measurements as experimental results. If a pretrained model cannot be executed, clearly identify the comparison as a reasoned discussion rather than an experimental result. Do not invent facial features or historical details during restoration; clearly label any inpainting or generative reconstruction and retain the unedited original.